La Plateforme - 3W Academy

Titre « Expert en Informatique et Systèmes d'Information » Option « Concevoir et déployer des infrastructures DevOps automatisées »

RNCP40573BC06 - Niveau 7 (Bac+5)

DOSSIER PROFESSIONNEL

Kroma

Comment offrir une plateforme de gestion de photos qui combine la confidentialité et la souveraineté des solutions auto-hébergées avec l'expérience utilisateur moderne des services grand public, tout en proposant une valeur d'usage que ces derniers n'adressent pas ?

Projet de fin de cycle

Candidat : Tony Mascate

Supervisé par : Joris Verguldezoone

Blocs communs BC1 · BC2 · BC3

Bloc optionnel retenu : BC4C - Concevoir et déployer des infrastructures DevOps automatisées

Session 2026

Remerciements

Je souhaite tout d'abord exprimer un grand merci à Sofiane BERKANI, mon tuteur, qui m'a accueilli avec bienveillance et m'a rapidement intégré au sein de l'équipe. Sa disponibilité, son écoute attentive et ses conseils avisés ont grandement facilité mon apprentissage et mon évolution tout au long de cette alternance.

Je tiens également à remercier Marceau CRISTIANINI, développeur avec qui j'ai eu le plaisir de collaborer au sein du service digital. Au-delà d'un simple collègue, il est devenu un ami, avec qui j'ai partagé de nombreux échanges tant sur le plan professionnel que personnel. Mon expérience au sein des Mutuelles du Soleil a été enrichissante.

L'entreprise accorde une réelle importance aux alternants, qui sont pleinement impliqués dans les projets et la vie d'équipe. La réalisation d'un projet commun ainsi que les différentes journées d'intégration organisées ont été autant d'occasions précieuses pour rencontrer et échanger avec plusieurs collègues. Je repars de cette expérience avec de solides compétences, un réseau professionnel élargi et une motivation renforcée pour la suite de mon parcours.

Je tiens enfin à saluer Joris pour son accompagnement et son soutien constant auprès de notre promotion tout au long de l'année.

Résumé

Ce dossier présente la conception et la réalisation d'une plateforme web de gestion de photos et d'albums, menée comme projet de fin de cycle dans le cadre du titre « Expert en Informatique et Systèmes d'Information » (RNCP 40573, niveau 7).

Le projet répond à un constat : les solutions grand public (Google Photos, iCloud, Amazon Photos) centralisent et exploitent les données des utilisateurs au sein d'écosystèmes fermés et soumis aux lois extraterritoriales américaines (comme le *Cloud Act*). À l'inverse, les alternatives open source respectueuses de la vie privée (Immich, Nextcloud) exigent des compétences techniques complexes en auto-hébergement, ce qui les rend inaccessibles au grand public.

Kroma se positionne comme une alternative intermédiaire : une plateforme clé en main offrant une expérience utilisateur moderne, tout en garantissant un contrôle strict de la confidentialité et une conformité au RGPD grâce à une architecture découplée (traitement des données sur VPS et stockage externalisé via une API S3).

La plateforme propose les fonctions attendues comme l'inscription sécurisée, upload, galerie chronologique, albums, partage par lien public et albums collaboratifs autour d'une fonctionnalité différenciante : **l'exploration chromatique, une navigation de la bibliothèque par ambiance colorimétrique**. À l'upload, une palette de couleurs est extraite de chaque

photo, puis classée dans un atlas chromatique fixe : l'utilisateur explore alors un nuancier où chaque case révèle ses photos d'une teinte donnée.

Sur le plan technique, le système repose sur une architecture conteneurisée orchestrée par Docker Swarm : une API NestJS (TypeScript, PostgreSQL via PgBouncer, Redis, file BullMQ, stockage objet Amazon S3 distribué par CloudFront), un frontend Next.js 16 (React Server Components), le tout dans un monorepo Turborepo. La chaîne DevOps est entièrement automatisée (GitHub Actions, registre ghcr.io, déploiement Ansible / Docker Secrets, migrations au démarrage) et observée par une stack Prometheus / Grafana / Loki. La qualité est garantie par une suite de tests (unitaires, intégration sur base réelle via testcontainers, E2E API et navigateur) bloquant le déploiement sous 80 % de couverture.

Abstract

This professional report details the design, development, and deployment of Kroma, a privacy-first photo management and chromatic exploration platform, realized as a final project for the "Expert in Information Systems Engineering" (EISI) degree. The project addresses a major gap in the current market: the dichotomy between mainstream cloud services (which compromise user privacy through extraterritorial laws and data profiling) and self-hosted open-source alternatives (which suffer from complex technical barriers for the general public).

Kroma positions itself as a turnkey solution offering a modern user experience while strictly adhering to GDPR standards and data sovereignty principles through a decoupled architecture. Its standout differentiating feature is "chromatic exploration," an innovative navigation system that extracts a perceptual color palette (using the OKLab color space) from each uploaded image and indexes it within a fixed, deterministic chromatic atlas.

Technically, the platform is built on a full-stack TypeScript architecture utilizing Next.js and NestJS, backed by a PostgreSQL database and Redis. The project emphasizes a comprehensive DevOps culture: it is containerized via Docker Swarm, orchestrated with Ansible (Infrastructure as Code), and features a fully automated CI/CD pipeline with stringent testing and quality gates.

Ultimately, this project demonstrates the feasibility for a solo developer to architect, develop, and industrialize a secure, eco-designed, and highly observable web platform, validating the full spectrum of advanced IT engineering competencies.

Keywords: *Photo Management, Data Sovereignty, GDPR, Chromatic Exploration, DevOps, TypeScript, Docker Swarm, Next.js, NestJS.*

Sommaire

1 - LISTE DES ABREVIATIONS 1

2 - GLOSSAIRE 2

3 - PRESENTATION PERSONNELLE ET INTRODUCTION 3

3.1 - PRESENTATION ET PARCOURS 3

3.2 - OBJECTIFS DE LA PRESENTATION 4

4 - INTRODUCTION GENERALE 4

5 - PRESENTATION DE LA PROBLEMATIQUE 5

5.1 - CONTEXTE DU PROJET 5

5.2 - DESCRIPTION DE LA PROBLEMATIQUE 5

5.3 - OBJECTIFS DU PROJET 6

5.4 - IMPORTANCE ET IMPACT DE LA PROBLEMATIQUE 6

5.5 - QUESTIONS DE RECHERCHE ET HYPOTHESES 6

5.6 - ÉTUDE D'OPPORTUNITE : DECISION GO / NO-GO 7

6 - ANALYSE ET DEFINITION DE LA STRATEGIE DU SYSTEME D'INFORMATION 8

6.1 - TECHNOLOGIQUE ET REGLEMENTAIRE 8

6.2 - CONFORMITE AU RGPD 12

7 - BESOINS FONCTIONNELS ET CAHIER DES CHARGES FONCTIONNEL 13

7.1 - DESCRIPTION DES FONCTIONNALITES REQUISES 13

7.2 - METHODOLOGIE DE COLLECTE DES BESOINS 14

7.3 - CONTRAINTES, EXIGENCES NON FONCTIONNELLES ET ACCESSIBILITE 16

7.4 - ANALYSE STRATEGIQUE ET CARTOGRAPHIE DU SI 17

7.5 - ARCHITECTURE CIBLE : COMPARAISON DES STYLES D'ARCHITECTURE 22

8 - IMPLEMENTATIONS TECHNIQUES 23

8.1 - INFRASTRUCTURE ET ENVIRONNEMENT DE DEPLOIEMENT 23

8.2 - TECHNOLOGIES ET OUTILS UTILISES 24

8.3 - SURVEILLANCE ET MONITORING DE LA SECURITE 26

8.4 - PLANS DE SAUVEGARDE ET DE RECUPERATION 28

9 - GESTION DU PROJET ET MANAGEMENT 29

9.1 - NOTE DE CADRAGE 29

9.2 - METHODOLOGIE, DECOUPAGE ET PLANIFICATION 31

9.3 - SUIVI DE L'AVANCEMENT ET GESTION DES RISQUES 34

9.4 - MANAGEMENT ET COMMUNICATION AVEC LES PARTIES PRENANTES 35

10 - CONCEPTION ET DEVELOPPEMENT DE L'APPLICATION INFORMATIQUE 37

10.1 - CONCEPTION 37
10.2 - DEVELOPPEMENT DE L'APPLICATION ET DE SON ENVIRONNEMENT 43
10.3 - STRATEGIE DE TESTS ET SUIVI DE LA QUALITE 50
10.4 - MODALITES DE LIVRAISON 52

11 - CONCEVOIR ET DEPLOYER DES INFRASTRUCTURES DEVOPS AUTOMATISEES 54

11.1 - ANALYSE DE L'EXISTANT ET STRATEGIE DEVOPS 54
11.2 - CONCEPTION ET IMPLEMENTATION DES PIPELINES CI/CD 56
11.3 - ORCHESTRATION CONTENEURISEE ET INFRASTRUCTURE AS CODE 57
11.4 - STRATEGIE DE MONITORING ET D'OBSERVABILITE 59
11.5 - SOBRIETE ENERGETIQUE ET USAGE DE L'INTELLIGENCE ARTIFICIELLE 60
11.6 - RETOUR D'EXPERIENCE ET PLAN D'AMELIORATION CONTINUE 61

12 - ANALYSES ET PERSPECTIVES 62

12.1 - ANALYSE DES RESULTATS ET DES PERFORMANCES 62
12.2 - RETOUR D'EXPERIENCE ET LEÇONS APPRISES 63
12.3 - PERSPECTIVES ET RECOMMANDATIONS 64

13 - CONCLUSION ET REMERCIEMENTS 64

14 - BIBLIOGRAPHIE ET WEBOGRAPHIE 66

14.1 - DOCUMENTATIONS TECHNIQUES OFFICIELLES 66
14.2 - SECURITE ET REGLEMENTATION 66
14.3 - DOCUMENTATION PRODUITE DANS LE CADRE DU PROJET 67

15 - ANNEXES 68

1 - Liste des abréviations

Les sigles et abréviations employés tout au long de ce dossier sont rassemblés ci-dessous.

Sigle	Signification
ADR	Architecture Decision Record - fiche de décision d'architecture
API	Application Programming Interface - interface de programmation
CDN	Content Delivery Network - réseau de diffusion de contenu (CloudFront)
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete - opérations de base sur les données
CSRF	Cross-Site Request Forgery - falsification de requête inter-sites
DTO	Data Transfer Object - objet de transfert de données
E2E	End-to-End - test de bout en bout
IaC	Infrastructure as Code - infrastructure décrite sous forme de code
JWT	JSON Web Token - jeton d'authentification signé
KPI	Key Performance Indicator - indicateur clé de performance
MVP	Minimum Viable Product - produit minimum viable
ORM	Object-Relational Mapping - correspondance objet-relationnel (TypeORM)
OWASP	Open Worldwide Application Security Project
RBAC	Role-Based Access Control - contrôle d'accès basé sur les rôles
RGAA	Référentiel Général d'Amélioration de l'Accessibilité
RGPD	Règlement Général sur la Protection des Données
RSC	React Server Components
SI	Système d'Information
SPOF	Single Point Of Failure - point unique de défaillance
SSR	Server-Side Rendering - rendu côté serveur
UUID	Universally Unique Identifier - identifiant unique universel
VPS	Virtual Private Server - serveur privé virtuel
WCAG	Web Content Accessibility Guidelines

2 - Glossaire

Les termes techniques structurants du projet sont définis ci-dessous, par ordre alphabétique.

Terme	Définition
Argon2	Algorithme de hachage de mots de passe « memory-hard », première recommandation OWASP, résistant aux attaques par GPU.
BullMQ	File de messages basée sur Redis, utilisée pour traiter l'optimisation des images de façon asynchrone, hors du cycle requête/réponse.
CloudFront	CDN d'Amazon qui distribue les fichiers stockés sur S3 ; les images sont servies via des URL signées à durée de vie limitée.
Docker Secrets	Mécanisme natif de Docker Swarm stockant les valeurs sensibles chiffrées dans le raft du cluster et les montant en mémoire dans les conteneurs.
Exploration chromatique	Fonctionnalité différenciante du projet : navigation de la galerie par la couleur, via une palette extraite de chaque photo et classée dans un atlas chromatique fixe.
OKLab / OKLCH	Espace colorimétrique perceptuellement uniforme (une même distance ≈ une même différence perçue). OKLCH en est la forme cylindrique : clarté, chroma, teinte. Utilisé pour extraire et ranger les couleurs fidèlement à la perception.
Atlas chromatique	Grille fixe de 53 cellules (12 teintes × 4 clartés + 5 neutres) dans laquelle chaque couleur d'une photo est rangée de façon déterministe.
PgBouncer	Pooler de connexions PostgreSQL mutualisant les connexions ouvertes par les réplicas de l'API.
Raft	Algorithme de consensus utilisé par Docker Swarm pour partager un état cohérent et chiffré entre les nœuds managers du cluster.
Sharp	Bibliothèque Node.js de traitement d'images (redimensionnement, conversion WebP, extraction de couleur) utilisée dans le pipeline d'upload.
Testcontainers	Bibliothèque démarrant de vrais conteneurs (ici PostgreSQL) le temps des tests d'intégration, pour valider le SQL réel et non un mock.
Turborepo	Orchestrateur de build pour monorepo, parallélisant et mettant en cache les tâches (build, test, lint) des différents paquets.

3 - Présentation personnelle et introduction

3.1 - Présentation et parcours

Dans cette partie, je présenterai mon parcours académique ainsi que mon projet professionnel.

Après avoir obtenu un baccalauréat Économique et Social, je me suis dirigé vers un DUT en Gestion des Entreprises et des Administrations (GEA). Le fonctionnement de l'entreprise m'intéressait particulièrement et je souhaitais développer mes compétences dans ce domaine. Au cours de cette formation, j'ai étudié des matières variées telles que le marketing, la comptabilité, le contrôle de gestion, la fiscalité et l'informatique bureautique.

Une fois diplômé, j'ai débuté ma carrière professionnelle en tant que chargé de clientèle à La Banque Postale. Mes missions consistaient à gérer les opérations bancaires courantes, mais aussi à assurer la relation client afin de proposer des offres adaptées aux besoins de chacun.

En parallèle, j'ai commencé à me former en autodidacte au développement web, un domaine qui m'a toujours attiré. En utilisant des plateformes d'apprentissage (Udemy, OpenClassrooms, YouTube) et en m'appuyant sur les documentations officielles, j'ai développé des compétences en développement front-end (HTML, CSS, JavaScript) et acquis de premières bases en back-end (MongoDB, Node.js).

En mars 2022, j'ai créé une micro-entreprise pour exercer en tant que développeur web front-end. Cette expérience m'a permis de réaliser plusieurs projets concrets, principalement des sites vitrines. En avril 2023, j'ai décidé de mettre fin à cette activité pour me concentrer sur l'apprentissage du développement back-end, dans le but de pouvoir intervenir sur des projets de plus grande envergure.

J'ai alors cherché une formation complète, ce qui m'a conduit à intégrer l'école ENI. J'y ai suivi une formation à distance pour valider le titre professionnel de Développeur Web et Web Mobile, puis j'ai poursuivi avec eux en alternance afin d'obtenir le titre de Concepteur Développeur d'Applications.

Aujourd'hui, j'achève ce parcours avec un Master 2 en Ingénierie du Web à La Plateforme, à Marseille, pour valider le titre d'Expert en Informatique et Systèmes d'Information.

Concernant mon projet professionnel, mon objectif à court terme est d'intégrer une entreprise en CDI afin d'acquérir une solide expérience dans le développement web. À plus long terme, j'envisage de relancer une activité de freelance, avec un niveau d'expertise technique bien supérieur à celui de mes débuts.

3.2 - Objectifs de la présentation

Ce dossier poursuit un double objectif. Le premier est de démontrer, à travers un projet complet et fonctionnel, la maîtrise des compétences attendues par le titre : définir une stratégie de système d'information, piloter un projet informatique, concevoir et développer une application, et déployer une infrastructure DevOps automatisée. Le second est de présenter une démarche d'ingénierie rigoureuse, dans laquelle chaque choix technique est justifié, les compromis sont assumés et documentés, et les limites actuelles sont clairement identifiées comme des axes d'évolution.

Le fil conducteur retenu est la réalisation, de bout en bout, d'une plateforme de gestion de photos. Ce projet couvre l'ensemble du cycle de vie de l'application : de l'analyse du besoin et de la veille technologique jusqu'au déploiement en production et au monitoring, en passant par la conception, le développement et la stratégie de tests. Le choix d'un projet unique, traité dans toute sa profondeur, permet d'illustrer concrètement l'intégralité des quatre blocs de compétences.

4 - Introduction générale

La photographie numérique a fait exploser le volume d'images que chacun produit et conserve. Un particulier accumule aujourd'hui des milliers de clichés par an, un photographe amateur bien davantage. Pour les stocker, les organiser et les partager, deux familles de solutions coexistent. D'un côté, les services grand public comme Google Photos, Apple iCloud, Amazon Photos qui offrent une expérience fluide et des fonctions d'intelligence artificielle puissantes, mais au prix d'un abonnement croissant et d'un hébergement des données sur des clouds américains, soumis à des législations extraterritoriales. De l'autre, les solutions libres et auto-hébergées comme Immich, Nextcloud qui garantissent la confidentialité, mais imposent une expérience technique qui les réserve de fait à un public averti.

Entre ces deux mondes subsiste un espace inoccupé : une plateforme à la fois respectueuse de la vie privée et agréable à utiliser. C'est l'hypothèse de départ de ce projet, et le sujet du chapitre consacré à la problématique. La réponse proposée n'ambitionne pas de rivaliser fonctionnalité pour fonctionnalité avec des géants disposant de moyens nettement supérieurs. Elle cherche plutôt à démontrer qu'une petite structure peut, avec des choix techniques sobres et bien maîtrisés, livrer un produit confidentiel, accessible, éco-responsable et porteur d'une vraie valeur d'usage différenciante.

Le présent document retrace l'intégralité de la démarche. Après avoir posé la problématique, il expose la stratégie de système d'information et la veille qui la fonde, détaille les implémentations techniques, la gestion de projet, la conception et le développement de l'application accompagnés de sa stratégie de tests, puis l'option DevOps qui industrialise le

déploiement. Il se referme sur une analyse des résultats, un retour d'expérience critique et les perspectives d'évolution.

5 - Présentation de la problématique

5.1 - Contexte du projet

Le projet s'inscrit dans le marché de la gestion de photothèques personnelles. La photographie numérique a démocratisé la production massive d'images : entre le smartphone, les réseaux sociaux et les sauvegardes automatiques, un utilisateur ordinaire conserve plusieurs milliers de photos, éparpillées entre plusieurs services. Le besoin n'est plus de capturer, mais de retrouver, d'organiser et de partager sans renoncer à la maîtrise de ses données.

Ce projet est mené dans une posture de porteur de projet, il ne s'agit pas de répondre à la commande d'un client unique, mais de concevoir un produit destiné à un marché, en s'appuyant sur une étude d'opportunité.

En l'absence d'un commanditaire traditionnel, les fonctionnalités et les besoins ont été définis à partir d'une analyse concurrentielle et de l'élaboration de personas représentatifs.

Cette démarche correspond à une situation très fréquente en entreprise, où une équipe doit être capable de transformer une opportunité de marché en spécifications fonctionnelles et techniques, puis en un produit livrable.

5.2 - Description de la problématique

Le marché de la gestion de photos se partage entre deux familles de solutions, chacune résolvant une moitié du problème de l'utilisateur tout en aggravant l'autre.

Famille de solutions	Points forts	Points faibles
Services grand public (Google Photos, iCloud, Amazon Photos)	Expérience soignée, IA puissante, mobilité	Données hébergées hors UE (Cloud Act), abonnement croissant, exploitation publicitaire/IA des contenus
Solutions auto-hébergées (Immich, Nextcloud Photos)	Confidentialité, souveraineté des données, gratuité logicielle	Mise en place et UX réservées à un public technique, ergonomie en retrait

Tableau 5.2 - Le compromis insatisfaisant du marché.

La problématique du projet se formule ainsi : **Comment offrir une plateforme de gestion de photos qui combine la confidentialité et la souveraineté des solutions auto-hébergées avec l'expérience utilisateur moderne des services grand public, tout en proposant une valeur d'usage que ces derniers n'adressent pas ?**

5.3 - Objectifs du projet

Pour répondre à cette problématique, le projet poursuit quatre objectifs structurants.

Souveraineté des données : Héberger les données identifiantes (comptes, métadonnées, albums) exclusivement sur une infrastructure maîtrisée en Europe, et conformer le traitement au RGPD (droit à l'oubli, minimisation des données).

Expérience moderne et accessible : Atteindre une ergonomie comparable aux services grand public, avec une exigence d'accessibilité (RGAA 4.1 / WCAG 2.1 niveau AA visé).

Valeur différenciante : Proposer une fonctionnalité absente du marché : l'exploration de la bibliothèque par la couleur des photos (exploration chromatique).

Industrialisation et fiabilité : Démontrer une chaîne complète de production logicielle : architecture conteneurisée, déploiement automatisé, observabilité et tests garantissant la non-régression.

5.4 - Importance et impact de la problématique

La concentration des photothèques chez quelques acteurs extra-européens pose une question de souveraineté numérique : des données personnelles intimes comme des visages, lieux, habitudes sont soumises à des législations comme le Cloud Act, qui autorise les autorités américaines à y accéder. Par ailleurs, l'entraînement de modèles d'IA sur les contenus des utilisateurs alimente une inquiétude légitime sur le devenir de ces images. Offrir une alternative crédible, respectueuse du RGPD et techniquement sobre (démarche d'éco-conception), répond donc à un besoin sociétal et réglementaire, pas seulement ergonomique.

L'impact attendu, à l'échelle de ce projet de fin de cycle, est double : sur le plan produit, valider l'existence d'un segment de marché par une démonstration fonctionnelle et sur le plan des compétences, prouver la capacité à mener un système d'information de la stratégie au déploiement.

5.5 - Questions de recherche et hypothèses

La démarche est guidée par trois hypothèses, vérifiées tout au long du dossier.

#	Hypothèse	Où elle est vérifiée
H1	Une petite structure peut livrer une UX moderne sans renoncer au RGPD.	Choix d'une stack moderne (Next.js, shadcn/ui) sur infrastructure auto-hébergée.
H2	Une organisation des photos par la couleur apporte une valeur d'usage réelle et inédite.	Benchmark et réalisation de l'exploration chromatique.

H3	Un développeur seul peut industrialiser un déploiement fiable et observable.	Docker Swarm, CI/CD, IaC Ansible, observabilité, tests bloquants.

Tableau 5.5 - Hypothèses de travail et lieux de vérification.

5.6 - Étude d'opportunité : décision Go / No-Go

Avant tout cadrage, une étude d'opportunité a tranché une seule question « faut-il lancer ce projet ? » en se plaçant du point de vue d'un décideur. Le « retour sur investissement » se mesure ici en valeur produit et en valeur de certification, le projet n'étant pas commercial mais démonstratif.

Axe du ROI	Éléments
Quantitatif	Coût d'abonnement cloud photo évité pour l'utilisateur (~30 à 100 €/an). Coût d'exploitation maîtrisé (1 VPS EU + stockage objet). Temps de tri manuel évité par la recherche par couleur.
Qualitatif	Souveraineté des données (métadonnées et comptes en Europe, aucune exploitation algorithmique). Différenciation produit (exploration chromatique unique sur le marché). Valeur de démonstration (système réel exploitable comme preuve de compétences). Réutilisabilité du socle technique.

Tableau 5.6 - Retour sur investissement estimé.

Quatre risques stratégiques ont été pesés contre cette valeur.

Conformité RGPD (risque élevé) : Les photographies constituant des données personnelles, un hébergement européen souverain est indispensable pour la mise en production. L'utilisation actuelle d'un stockage S3 chez AWS est un compromis assumé, accepté uniquement pour la phase de prototypage (MVP).

Dépendance cloud / Cloud Act (risque moyen) : Bien que l'infrastructure de stockage AWS soit soumise au droit américain, ce risque est fortement atténué par le découplage de l'architecture. Les fichiers bruts stockés sur le S3 sont séparés de la base de données (qui est hébergée sur le VPS). Sans ces métadonnées pour faire le lien, les photos stockées perdent leur contexte et sont inexploitables.

Développement solo (risque élevé) : Porter seul la réalisation complète de cette plateforme constitue un goulot d'étranglement naturel. Pour y faire face, une priorisation stricte des fonctionnalités a été appliquée en utilisant la méthode MoSCoW.

Adoption de la fonction couleur (risque faible) : L'intérêt réel des utilisateurs pour la recherche de photos par couleur reste à valider. Ce point sera évalué lors des premiers tests d'usage.

Go / No-Go - Décision : GO. Le besoin est réel et non couvert, la différenciation (exploration chromatique) est forte, les risques sont maîtrisables et la valeur de certification est élevée (cycle complet + bloc DevOps). Conditions du Go : périmètre cadré strictement (MoSCoW), conformité RGPD traitée dès la conception, périmètre exclu explicitement acté dans la note de cadrage.

6 - Analyse et définition de la stratégie du système d'information

Ce chapitre couvre le premier bloc de compétences : structurer une veille technologique et réglementaire, en exploiter les résultats pour formuler une stratégie informatique, cartographier le système d'information et en analyser les risques. Il fonde tous les choix techniques détaillés dans les chapitres suivants.

6.1 - Technologique et réglementaire

6.1.1 - Introduction

Tout choix technique de ce projet a été précédé d'une veille structurée, consignée dans un dossier dédié. L'objectif était triple : sélectionner des technologies pérennes et adaptées à un développeur seul, anticiper les contraintes réglementaires et d'accessibilité, et privilégier les options limitant l'empreinte environnementale. La veille ne s'est pas arrêtée au démarrage, elle a accompagné tout le cycle et plusieurs décisions ont été révisées grâce à de nouvelles informations.

6.1.2 - Méthodologie de la veille

La veille s'appuie sur des sources hiérarchisées par fiabilité, et sur une méthode de collecte et d'évaluation explicite afin de ne retenir que des informations vérifiables.

Type de source	Exemples mobilisés	Niveau d'exploitation
Sources primaires	Documentations officielles, spécifications, dépôts GitHub des projets	Fiabilité maximale car référence faisant autorité
Sources institutionnelles	CNIL, ANSSI, OWASP	Réglementaire et sécurité car autorité reconnue
Sources techniques de qualité	Blogs d'ingénierie, conférences, benchmarks reproductibles	Recoupées avant adoption
Veille passive	Newsletters spécialisées, agrégateurs, releases GitHub suivies	Déclenche une investigation ciblée pour approfondir le sujet

Tableau 6.1.2 - Sources de veille et critères d'évaluation.

Chaque information candidate est évaluée selon sa fiabilité, sa date de sortie, son impact sur l'environnement, et le rapport bénéfice/risque de son adoption. Les normes de référence retenues pour l'évaluation sont les standards ISO et W3C, le RGPD et le RGAA pour l'accessibilité.

6.1.3 - Cadre réglementaire identifié

Quatre cadres réglementaires structurent les contraintes du projet. Leur identification a directement orienté l'architecture (notamment le choix d'héberger les données identifiantes en Europe).

Cadre	Objet	Implications pour le projet
RGPD	Protection des données personnelles	Minimisation des données, droit à l'oubli (suppression du compte et des données), hébergement UE, consentement, sécurité des traitements
IA Act	Encadrement des systèmes d'IA	Vigilance sur l'usage de modèles. Le projet écarte volontairement tout traitement ou profilage des contenus visant à entraîner des modèles d'intelligence artificielle.
Cloud Act	Accès extraterritorial des autorités US aux données	Risque sur les fichiers stockés chez un acteur américain. Mitigé par région UE en production et séparation des données identifiantes.
RGAA / WCAG	Accessibilité numérique	Niveau AA visé : contrastes, navigation clavier, sémantique HTML, alternatives textuelles

Tableau 6.1.3 - Grandes orientations réglementaires retenues.

6.1.4 - Résultats : synthèse des technologies retenues

La veille a abouti à un socle technologique cohérent, dont chaque brique a fait l'objet d'une fiche de décision. Le tableau ci-dessous synthétise les choix et leur justification.

Domaine	Choix	Raison principale
Langage	TypeScript (front et back)	Un seul langage sur toute la chaîne, types partagés
Frontend	Next.js 16 + React 19 (RSC)	SSR, bundle réduit, SEO, optimisation d'images
UI	shadcn/ui + Radix UI + Tailwind v4	Accessibilité native, code possédé, design sobre
Backend	NestJS 11 + TypeORM	Architecture modulaire, écosystème intégré, migrations
Base de données	PostgreSQL + PgBouncer	Relationnel robuste, pooling multi-réplicas
Cache / file	Redis + BullMQ	Cache partagé et traitement asynchrone des images
Stockage objet	Amazon S3 + CloudFront (pour le MVP)	Stockage binaire scalable, diffusion CDN signée
Orchestration	Docker Swarm	Réplication et rolling updates sans la lourdeur de K8s
CI/CD	GitHub Actions + ghcr.io + Ansible	Pipeline as code, déploiement reproductible
Observabilité	Prometheus + Loki + Grafana	Métriques et logs unifiés, faible empreinte

Tableau 6.1.4 - Socle technologique issu de la veille.

6.1.5 - Choix de la fonctionnalité différenciante (matrice de décision)

La veille a aussi servi à arbitrer la fonctionnalité différenciante. Trois pistes ont été évaluées au regard de leur valeur d'usage, de leur innovation, de leur faisabilité pour un développeur seul et de leur conformité réglementaire. Une matrice de décision les a départagées.

Option	Valeur	Innov.	Faisabilité	Contrainte réglementaire	Score
Exploration chromatique	5	5	4	Aucune (pas de profilage)	36/40
Carte interactive (géo)	4	3	3	Métadonnées de localisation	-

Recherche sémantique par IA	5	5	2	Forte (IA Act, contenus)	-

Tableau 6.1.5 - Matrice de décision de la fonctionnalité différenciante.

L'exploration chromatique apporte une valeur d'usage immédiate et inédite, reste réalisable seul et n'implique aucun profilage ni analyse de contenu sensible, point décisif au regard de l'IA Act. Elle se fonde sur une simple caractéristique physique de l'image (ses teintes dominantes), pas sur la reconnaissance de personnes ou de scènes. C'est un exemple concret de veille réglementaire orientant un choix produit.

6.1.6 - Démarche d'éco-conception (Green IT)

L'éco-conception n'a pas été traitée comme une contrainte isolée, mais comme un critère de sélection transverse intégré à chaque décision technique. La démarche repose sur un principe de sobriété : à service rendu équivalent, l'option la moins consommatrice de ressources énergétiques et matérielles est systématiquement privilégiée.

Domaine	Choix éco-responsable	Bénéfice
Orchestration	Docker Swarm plutôt que Kubernetes	CPU/RAM nettement réduits à cette échelle
Observabilité	Loki plutôt qu'ELK (pas de JVM)	Empreinte mémoire et stockage moindres
Rendu frontend	React Server Components	Moins de JavaScript transmis et exécuté côté client
Images	Conversion WebP + diffusion CDN	Poids réseau réduit, moins de retransmissions
Build	Turborepo (cache) + turbo prune	Moins de calcul redondant en CI, images plus légères

Tableau 6.1.6 - Déclinaison de l'éco-conception dans les choix techniques.

6.1.7 - Recommandations, perspectives et conclusion

La veille technologique a confirmé la pertinence d'une pile technique moderne basée sur TypeScript pour un projet mené en solo. Elle a également mis en évidence certains points de vigilance, notamment la jeunesse de certaines briques logicielles comme Tailwind v4 ou le runtime Bun. Les axes d'évolution identifiés, tels que l'étude d'un stockage objet souverain compatible avec l'API S3 (par exemple via un fournisseur européen), sont détaillés dans la section dédiée aux perspectives.

Posture de veille : La veille n'a pas été un préalable figé mais un processus continu : plusieurs décisions (URLs CloudFront signées, durcissement des secrets) sont nées de veilles de sécurité menées en cours de projet.

6.2 - Conformité au RGPD

Le traitement de photos personnelles place la protection des données au cœur du projet. Cette section fait le point sur ce qui est en place et sur ce qui reste à implémenter.

6.2.1 - Principes appliqués

Minimisation : Seules les données strictement nécessaires sont collectées : email, nom, prénom, mot de passe (haché), et les métadonnées techniques des photos. Aucune donnée de géolocalisation, aucun profilage.

Localisation européenne : Les données identifiantes (comptes, albums, métadonnées) résident exclusivement sur le VPS européen, les binaires sont sur S3 région Paris (eu-west-3) seulement pour le MVP.

Sécurité des traitements : Mots de passe hachés en Argon2id, transport chiffré (HTTPS), images servies par URL CloudFront signées, secrets chiffrés (Docker Secrets), aucune donnée sensible dans les logs.

Absence de profilage : L'exploration chromatique repose sur une caractéristique physique de l'image (teinte dominante), pas sur la reconnaissance de personnes ou de scènes : aucun traitement de données biométriques ni de profilage.

6.2.2 - Registre des traitements (extrait)

Traitement	Finalité	Données	Base légale
Gestion de compte	Permettre l'authentification	Email, nom, prénom, mot de passe haché	Exécution du service
Stockage de photos	Héberger et organiser les médias	Fichiers, métadonnées, couleur dominante	Exécution du service
Partage	Diffuser une ressource choisie	Jeton de partage, membres d'album	Consentement de l'utilisateur
Journalisation technique	Exploitation et sécurité	Logs techniques (sans données sensibles)	Intérêt légitime

Tableau 6.2.2 - Extrait du registre des traitements.

6.2.3 - Droits des personnes - état et écart assumé

Sur ce point, le dossier établit une distinction claire entre les solutions déjà opérationnelles et les perspectives d'évolution futures.

Droit RGPD	État	Détail
Droit d'accès	Partiel	L'utilisateur consulte ses données via l'application (galerie, profil).
Droit de rectification	Partiel	Renommage des albums, suppression des images.
Droit à l'effacement (droit à l'oubli)	À implémenter	La suppression de compte n'est pas encore développée. Le modèle de données y est toutefois préparé : toutes les clés étrangères sont en suppression cascade (ON DELETE CASCADE). Un futur endpoint de suppression effacera automatiquement photos, albums, partages et jetons.
Droit à la portabilité	À implémenter	Export des données envisagé en évolution.

Tableau 6.2.3 - Droits des personnes : niveau de prise en charge.

Point important : Le droit à l'oubli n'est pas encore développé, c'est un écart assumé du MVP. Le choix de conception (cascades en base) a néanmoins été fait dès le départ pour que son implémentation future soit rapide et fiable. Sur un produit réellement mis en production, cette fonctionnalité serait un prérequis de conformité.

7 - Besoins fonctionnels et cahier des charges fonctionnel

7.1 - Description des fonctionnalités requises

Les besoins ont été formalisés en 29 user stories réparties en six domaines fonctionnels. La priorisation MoSCoW distingue le périmètre du MVP (Must Have) des évolutions souhaitables (cf Annexe 21 : Matrice MoSCoW).

Domaine fonctionnel	Stories	Contenu
1. Authentification & compte	5	Inscription, connexion, déconnexion, changement de mot de passe, suppression du compte (RGPD)
2. Upload & gestion des photos	6	Upload multiple résilient, galerie chronologique, suppression, favori, détails, tags
3. Albums	6	Création, ajout/retrait de photos, liste, renommage, suppression, photo de couverture
4. Partage	4	Lien public révocable, désactivation, lecture seule, expiration
5. Exploration chromatique	5	Extraction de palette à l'upload, classement dans un atlas fixe, navigation par teinte
6. Recherche & filtres	3	Recherche par date, filtre favoris, recherche par tag

Tableau 7.1 - Cartographie des 29 user stories par domaine.

Répartition par priorité : 16 « Must Have », 9 « Should Have », 4 « Could Have ». Le MVP correspond aux 16 Must Have.

7.1.1 - Exemples de user stories

US-06 : En tant qu'utilisateur, je veux importer une ou plusieurs photos depuis mon appareil afin de les stocker sur la plateforme. **[Must]**

US-18 : En tant qu'utilisateur, je veux partager un album via un lien public afin que mes proches puissent le consulter sans créer de compte. **[Must]**

US-22 : En tant que photographe, je veux pouvoir regrouper mes photos par couleur dominante afin de visualiser la cohérence de mes séries. **[Must]**

7.2 - Méthodologie de collecte des besoins

En l'absence d'un client unique, les besoins ont été dérivés d'une analyse de marché (benchmark concurrentiel) et formalisés par des personas représentatifs, puis traduits en user stories priorisées selon la méthode MoSCoW (Must / Should / Could have). Cette démarche garantit que chaque fonctionnalité répond à un besoin identifié.

7.2.1 - Personas

Deux personas complémentaires cadrent les attentes et structurent les arbitrages produit (cf Annexe 20 : Personas).

Persona	Profil et besoin principal
Léa, 28 ans : la particulière connectée	Non technique. Elle Veut retrouver et partager ses photos de voyage simplement, sans abonnement croissant, et garder ses données privées. Fonctions clés : galerie, albums, partage.
Maxime, 24 ans : le photographe amateur	Il organise mentalement ses séries par ambiance colorimétrique. Frustré que ses outils trient par date mais jamais par couleur. Fonction clé : l'exploration chromatique.

Tableau 7.2.1 - Personas directeurs du produit.

7.2.2 - Analyse concurrentielle (benchmark)

Six solutions ont été comparées sur quatre axes (fonctionnalités, technique, expérience utilisateur, prix) : Google Photos, iCloud Photos, Amazon Photos, Flickr, Immich et Nextcloud Photos. Le benchmark révèle deux gaps de marché exploitables.

Gap n°1 - Organisation visuelle par la couleur : Aucune des six solutions ne propose de naviguer sa photothèque par dominante chromatique. C'est l'espace que vient occuper l'exploration chromatique.

Gap n°2 - Privacy-first avec UX moderne : D'un côté, les solutions auto-hébergées garantissent la vie privée mais s'avèrent trop techniques. De l'autre, les outils grand public offrent une expérience fluide mais s'approprient les données des utilisateurs. Le segment combinant à la fois le respect de la vie privée et une interface moderne reste donc totalement inoccupé.

Solution	Fonct.	Tech.	UX	Prix	Total /20
Google Photos	4	3	5	3	15
iCloud Photos	4	4	5	3	16
Amazon Photos	3	3	3	4	13
Flickr	3	3	3	3	12
Immich	3	5	4	5	17
Nextcloud Photos	2	4	2	5	13
Kroma (cible)	5	4	5	5	19

Tableau 7.2.2 - Score global du benchmark (notation 1 à 5 par axe).

7.2.3 - Analyse SWOT de l'opportunité

La synthèse de l'étude d'opportunité prend la forme d'une matrice SWOT, qui croise les facteurs internes (forces, faiblesses du projet) et externes (opportunités, menaces du marché). Elle ne fait que structurer des constats déjà établis (benchmark, contraintes du projet).

Axe	Éléments
Forces (interne)	Fonctionnalité différenciante (exploration chromatique) Stack moderne et accessible Contrôle de la qualité (tests, CI/CD, observabilité) Architecture agnostique Éthique et conformité
Faiblesses (interne)	Projet solo (capacité limitée) Points uniques de défaillance (PostgreSQL, VPS, manager Swarm) Dépendance à Amazon S3 (Cloud Act) pour le MVP Fonctions encore absentes (droit à l'oubli, alerting, recherche avancée) Pas d'application mobile
Opportunités (externe)	Quadrant de marché vide (confidentialité + UX moderne) Sensibilité croissante à la souveraineté et au RGPD Valorisation de l'éco-conception (numérique responsable)
Menaces (externe)	Acteurs installés aux moyens massifs (Google, Apple…) Dépendance réglementaire/technique à un fournisseur américain Evolution rapide de l'écosystème technique

Tableau 7.2.3 - Analyse SWOT de l'opportunité.

7.3 - Contraintes, exigences non fonctionnelles et accessibilité

Au-delà des fonctions, le cahier des charges fixe des exigences transverses.

Exigence	Déclinaison
Sécurité	Mots de passe hachés (Argon2), JWT en cookies HTTP-only, protection CSRF, rate limiting
Performance	Galerie en défilement infini, images optimisées (WebP), diffusion via CDN, pagination côté serveur

Disponibilité	Réplication des services, rolling updates sans interruption, restart automatique, sauvegardes quotidiennes
Accessibilité	RGAA 4.1 / WCAG 2.1 AA : navigation clavier, contrastes ≥ 4.5:1, alternatives textuelles, sémantique HTML
Éco-conception	Choix de briques légères (Swarm vs K8s, Loki vs ELK, RSC), réduction du JavaScript client et de la consommation serveur
Conformité	RGPD : droit à l'oubli, minimisation, hébergement UE des données identifiantes

Tableau 7.3 - Exigences non fonctionnelles.

L'accessibilité a fait l'objet d'une attention particulière. Le choix de Radix UI (socle de shadcn/ui) apporte nativement les bons attributs ARIA, la gestion du focus et la navigation clavier sur tous les composants interactifs. Les critères WCAG appliqués comme les alternatives textuelles sur les vignettes, contrastes vérifiés, langue de page déclarée, identification des erreurs de formulaire sont détaillés dans la cartographie du SI.

7.4 - Analyse stratégique et cartographie du SI

7.4.1 - Introduction

La stratégie de système d'information traduit les besoins en une architecture cible, cartographiée puis soumise à une analyse de risques. La cartographie distingue le découpage fonctionnel (diagramme de composants) de l'infrastructure physique (diagramme de déploiement).

7.4.2 - Cartographie applicative (diagramme de composants)

L'application est découpée en modules métier qui communiquent via des contrats explicites. L'architecture s'appuie sur un frontend en Next.js qui consomme une API REST développée avec NestJS. Côté infrastructure, PostgreSQL (optimisé par PgBouncer) gère les données structurées, Redis assure la gestion du cache et des files d'attente, tandis qu'Amazon S3 et CloudFront prennent en charge le stockage et la distribution des fichiers binaires.

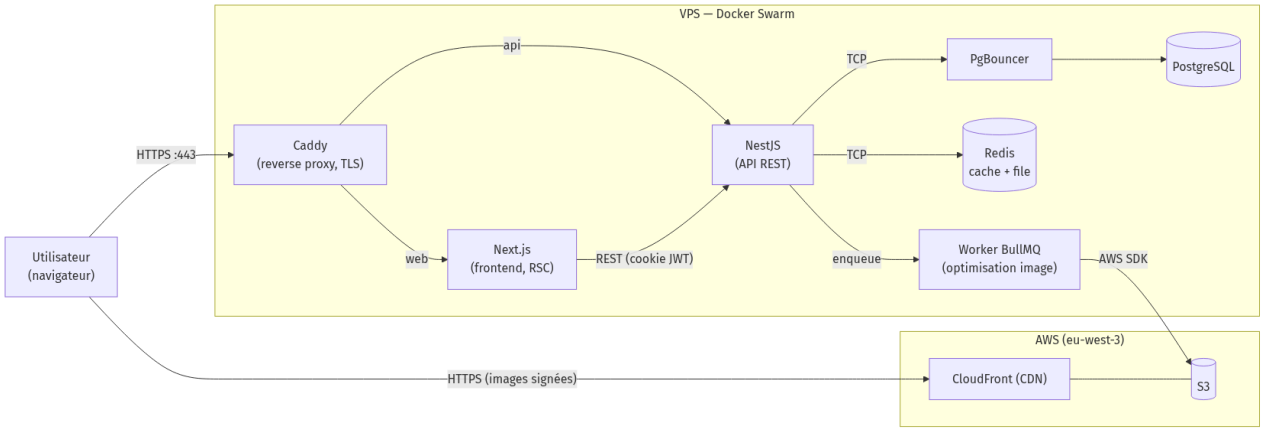
La représentation visuelle détaillant les interactions entre ces différentes briques logicielles est consultable en annexe *(cf. Annexe 1 : Diagrammes d'architecture)*.

Module	Responsabilité
Auth	Authentification (JWT) et autorisation (ownership), gestion des refresh tokens
Users	Comptes utilisateurs, profil, rôles

Photos	Upload, métadonnées, statut de traitement, couleur dominante, partage public
Albums	Albums, photos d'album, membres (albums collaboratifs), couvertures
Storage (AWS)	Abstraction du stockage objet : multipart S3, URLs CloudFront signées
Observabilité	Exposition des métriques Prometheus et des logs structurés (Pino → Loki)

Tableau 7.4.2 - Les briques métier de l'architecture applicative.

Le schéma de flux résume les protocoles et données échangés.



7.4.3 - Cartographie d'infrastructure (diagramme de déploiement)

L'infrastructure repose sur un cluster Docker Swarm hébergé sur un VPS européen. Les services tournent en plusieurs réplicas derrière le load balancer intégré de Swarm. Le code est construit et publié par GitHub Actions sur le registre ghcr.io, puis déployé par SSH via « docker stack deploy ». Amazon S3 / CloudFront est le seul composant externe.

La représentation globale de cette infrastructure réseau ainsi que la mécanique de déploiement sont schématisées en annexe *(cf. Annexe 2 : Diagramme de déploiement)*.

Service	Port / protocole	Accessible depuis	Réplicas
Frontend (Next.js)	:3000 (HTTP) / :443 (reverse proxy)	Internet	2 réplicas
API (NestJS)	:4000 (HTTP)	Réseau interne Swarm	4 réplicas
PgBouncer	:5432 (TCP)	Réseau interne Swarm	1
PostgreSQL	:5432 (TCP)	PgBouncer uniquement	1 (nœud unique)
Redis	:6379 (TCP)	Réseau interne Swarm	1
Prometheus / Loki / Grafana	:9090 / :3100 / :3001	Administrateur (protégé)	1 chacun

Tableau 7.4.3 - Plan de déploiement : services, ports et exposition.

7.4.4 - Diagnostic du SI et analyse de risques

Le diagnostic du système d'information se lit d'abord à travers ses forces et ses faiblesses, avant d'être approfondi par l'analyse de risques.

Dimension	Forces	Faiblesses
Architecture	Modulaire, découplée, testable Déploiement automatisé reproductible	Monolithe sur nœuds uniques (PostgreSQL, VPS, manager Swarm) Pas de haute disponibilité actuellement avec un seul VPS
Sécurité	Argon2, JWT HTTP-only, CSRF, ownership, secrets chiffrés, URLs signées	Alerting de sécurité non configuré Rate limiting non partagé entre réplicas
Données	Modèle relationnel cohérent, cascades RGPD, sauvegardes quotidiennes	Droit à l'oubli non implémenté Binaires sur S3 pour le MVP
Qualité	Tests pluri-niveaux, couverture bloquante en CI, observabilité (métriques + logs)	Alerting non configuré Pas d'environnement de pré-production
Exploitation	IaC complète (Ansible), rolling updates, CI/CD automatisé	Manager Swarm unique Pas de mesure d'empreinte énergétique

Tableau 7.4.4 - Diagnostic du SI : forces et faiblesses.

Ce diagnostic oriente directement l'analyse de risques : les faiblesses identifiées (points uniques de défaillance, dépendance externe, absence d'alerting) se retrouvent parmi les menaces cotées ci-dessous.

La méthode d'analyse de risques retenue est une cotation Probabilité × Impact (échelle 1 à 3), inspirée des méthodes EBIOS et ISO 27005 (*cf. Annexe 12 : Analyse des risques*). Les actifs critiques ont été inventoriés, puis chaque menace cotée avant et après mitigation pour faire apparaître le risque résiduel.

Actifs critiques identifiés : PostgreSQL (données structurées), S3/CloudFront (binaires), service d'authentification (mots de passe, refresh tokens), Redis (cache), et le VPS (socle de l'ensemble).

#	Actif	Menace	Brut	Mitigation	Résiduel
R1	Auth	Brute force sur le login	9	Throttler, Argon2id, refresh token révocable	2
R2	PostgreSQL	Panne (nœud unique, SPOF)	9	Backups quotidiens, restart Swarm, supervision Grafana (métriques)	6
R3	Photos	Accès non autorisé à des photos privées	6	ownership, HTTPS, URLs CloudFront signées	2
R4	Tokens JWT	Vol de token (XSS)	6	Cookie HTTP-only, durée 15 min, HTTPS	2
R5	VPS	Indisponibilité (panne matérielle)	6	Rolling updates, snapshots, supervision Grafana	3
R7	API	Injection SQL	3	TypeORM (requêtes paramétrées) + validation Zod	1
R8	S3 / AWS	Cloud Act (accès autorités US)	2	Région UE (Paris), aucune donnée identifiante sur S3	1

Tableau 7.4.4 - Extrait de la matrice de risques (score = P × I).

Trois risques résiduels sont explicitement acceptés dans le contexte d'un projet solo : le nœud PostgreSQL unique (la réplication primaire/secondaire dépasse la charge utile pour le MVP), le VPS unique (un second serveur de failover n'est pas justifié économiquement mais peut être ajouté facilement via Ansible), et l'absence de WAF (le rate limiting applicatif et le pare-feu réseau constituent la protection en place).

7.4.5 - Accessibilité du SI

L'accessibilité est traitée comme une exigence d'architecture, non comme une retouche finale. Le tableau ci-dessous relie des critères WCAG à leur application concrète dans le produit.

Critère WCAG	Application dans Kroma
1.1.1 Contenu non textuel	Attribut alt descriptif sur les vignettes alt vide sur le décoratif
1.4.3 Contraste minimum	Ratio ≥ 4.5:1 (texte)
2.1.1 Clavier	Navigation Tab / Entrée / Échap sur tous les composants Radix UI
2.4.7 Focus visible	Outline conservé sur l'élément actif
3.1.1 Langue de la page	lang="fr" sur l'élément html
3.3.1 Identification des erreurs	Messages Zod localisés, associés au champ via react-hook-form
4.1.2 Nom, rôle, valeur	Rôles ARIA natifs fournis par Radix UI

Tableau 7.4.5 - Critères WCAG appliqués.

Un point de vigilance est à noter sur les interfaces visuelles riches (comme l'explorateur chromatique, fondé sur un nuancier de cases colorées) qui posent des défis d'accessibilité spécifiques comme l'identification par la seule couleur. Des alternatives textuelles (étiquette du nom de la teinte, déjà présente en infobulle et aria-label, description de la répartition) sont la priorité retenue pour atteindre le niveau AA sur ces écrans.

Pour garantir le respect des critères WCAG, je ne me suis pas contenté d'une approche théorique. J'ai mis en place une double stratégie de contrôle. D'un côté, j'ai intégré Pa11y pour réaliser des tests d'accessibilité automatisés, ce qui me permet de détecter les régressions techniques (comme un problème de contraste ou un attribut alt manquant) directement lors des phases de build. D'un autre côté, j'ai croisé ces résultats avec des contrôles manuels en utilisant Ara (ara.numerique.gouv.fr), la plateforme officielle de l'État pour l'évaluation RGAA. Cette démarche m'a permis de valider concrètement mon niveau d'accessibilité AA, notamment sur les interfaces complexes comme l'explorateur chromatique.

7.4.6 - Élaboration de la stratégie et préconisations

Le diagnostic conduit à une stratégie informatique en trois axes, intégrant les dimensions sécurité, DevOps et éco-conception.

Axe 1 - Socle robuste avant fonctionnalités. Investir d'abord dans l'authentification, l'architecture modulaire et le pipeline de déploiement, pour bâtir les fonctionnalités sur des fondations sûres.

Axe 2 - Différenciation par l'usage. Concentrer l'effort d'innovation sur l'exploration chromatique, seul élément réellement distinctif face à des concurrents.

Axe 3 - Industrialisation et observabilité. Automatiser le déploiement et instrumenter le système dès le départ, pour déployer souvent et sans crainte.

Fil directeur stratégique : La stratégie assume un principe d'économie de moyens : préférer systématiquement la brique la plus simple qui satisfait le besoin (Swarm plutôt que Kubernetes, Loki plutôt qu'ELK), ce qui sert à la fois le délai, la maintenabilité et le Green IT.

7.5 - Architecture cible : comparaison des styles d'architecture

Au-delà du choix des briques, un choix plus structurant porte sur le style d'architecture lui-même. Quatre grands styles ont été comparés au regard des contraintes du projet : un développeur seul, un délai court, des besoins avec état (file de traitement, connexions base), et une exigence d'éco-conception.

Critère	Monolithe	Monolithe modulaire	Microservices	Serverless
Complexité opérationnelle	Faible	Faible-moyenne	Élevée	Moyenne
Scalabilité	Globale (tout le bloc)	Globale + extraction possible	Fine (par service)	Automatique
Observabilité	Simple	Simple	Complexe (distribuée)	Dépend du fournisseur
Besoins avec état (file, DB)	Naturel	Naturel	Naturel	Contraint (cold start)
Adapté à un projet solo	Oui	Oui	Non (surcoût)	Partiel
Dépendance fournisseur	Aucune	Aucune	Aucune	Forte (lock-in)
Verdict	Possible	Retenu	Surdimensionné	Inadapté ici

Tableau 7.5 - Comparaison des styles d'architecture.

Le style retenu est le monolithe modulaire avec une API NestJS unique mais découpée en modules au périmètre net (auth, users, photo, album, aws), complétée par un worker asynchrone (BullMQ) pour le traitement d'image, et un frontend Next.js distinct. Ce choix se justifie point par point.

Contre les microservices. Découper en services indépendants imposerait une complexité réseau (communication inter-services, cohérence distribuée) et une observabilité distribuée disproportionnées pour un développeur seul. Le coût d'exploitation dépasserait le bénéfice à cette échelle.

Contre le serverless. Le projet a des besoins avec état mal servis par les fonctions éphémères : une file de traitement persistante, des connexions base de données mutualisées (PgBouncer),

et un worker de longue durée. S'y ajoutent les démarrages à froid et une forte dépendance au fournisseur, contraire à l'objectif de souveraineté.

Pourquoi modulaire et non monolithe « classique ». Les frontières de modules (injection de dépendances, contrats explicites) préservent la maintenabilité et, surtout, rendent une extraction future en services possible sans réécriture : le module le plus sollicité (le traitement d'image, déjà isolé derrière une file) pourrait devenir un service autonome le jour où la charge le justifierait.

Du point de vue des qualités non fonctionnelles, ce style offre une bonne performance (un seul saut réseau interne, cache partagé), une sécurité maîtrisée (une seule surface d'API à protéger plutôt qu'un maillage de services) et une sobriété supérieure (moins de conteneurs, moins de trafic interne). La scalabilité, globale, est assurée par la réplication du conteneur d'API sous Docker Swarm, suffisante à l'échelle visée.

Style d'architecture : Le monolithe modulaire est le point d'équilibre entre la simplicité du monolithe et l'évolutivité des microservices : on déploie une seule application, mais conçue de façon à pouvoir être décomposée plus tard si le besoin réel apparaît, sans payer la complexité distribuée avant l'heure.

8 - Implémentations techniques

Ce chapitre décrit l'environnement d'exécution du système : l'infrastructure matérielle et logicielle, l'environnement de déploiement, les technologies et outils mobilisés, la surveillance de la sécurité, et la stratégie de sauvegarde et de reprise. Les choix sont ici présentés et leur justification détaillée figure dans les ADR (cf Annexe 17 : Architecture Decision Records) et leur industrialisation dans le chapitre DevOps.

8.1 - Infrastructure et environnement de déploiement

8.1.1 - Infrastructure matérielle

La plateforme est hébergée sur un VPS loué chez un hébergeur européen, sur lequel tourne un cluster Docker Swarm mono-nœud (configuré pour évoluer vers le multi-nœuds). Ce choix d'un hébergement maîtrisé en Europe est une décision d'architecture directement issue de la contrainte RGPD : les données identifiantes ne quittent pas le territoire.

Rôle	Composant	Données / fonction
Calcul / orchestration	VPS Linux, Docker Swarm	Exécution des conteneurs, réplication, réseau overlay
Données structurées	PostgreSQL + PgBouncer	Comptes, métadonnées photos, albums, partages

Cache & file	Redis	Cache partagé entre réplicas et file de traitement BullMQ
Stockage binaire	Amazon S3 (eu-west-3, Paris)	Fichiers photos (raw + optimisés)
Diffusion	Amazon CloudFront (CDN)	Distribution des images via URLs signées
Registre d'images	ghcr.io (GitHub)	Stockage des images Docker construites en CI

Tableau 8.1.1 - Infrastructure matérielle et services.

8.1.2 - Environnement de déploiement (cloud, on-premise, hybride)

L'architecture est hybride et assumée comme telle. Le cœur du système comme le calcul, la base de données, le cache, les données identifiantes est auto-hébergé sur le VPS. Seuls les fichiers binaires des photos sont externalisés sur un cloud public (Amazon S3 pour le MVP, région Paris), pour bénéficier d'un stockage scalable et d'une diffusion CDN performante sans gérer soi-même un serveur de fichiers et sa bande passante.

Choix d'hébergement - Externaliser uniquement les binaires sur S3 (région UE) introduit une dépendance à un acteur américain (Cloud Act), mais les données identifiantes restent exclusivement sur le VPS. Un fichier S3 seul, sans la base, n'est pas exploitable. L'évolution vers un stockage objet souverain est identifiée en perspective pour la mise en production (OVH, Scaleway).

Les conteneurs sont décrits de façon déclarative dans un fichier « stack.yml » (mode Swarm), qui joue le rôle d'infrastructure as code : il fixe le nombre de réplicas, les politiques de redémarrage, la configuration de mise à jour (rolling update) et le montage des secrets. Tout changement d'infrastructure passe par ce fichier versionné.

8.1.3 - Réseau et exposition

Seuls trois ports sont ouverts sur internet (22 pour l'administration SSH, 80 et 443 pour le trafic web), via un pare-feu UFW en politique « deny all » par défaut. Les ports internes du cluster Swarm (2377, 7946, 4789) sont restreints au réseau privé entre nœuds. La base de données PostgreSQL n'est jamais exposée directement : elle n'est accessible que par PgBouncer, lui-même interne au réseau overlay du cluster.

8.2 - Technologies et outils utilisés

8.2.1 - Frameworks et bibliothèques

L'ensemble du projet est écrit en TypeScript, du frontend au backend, ce qui permet de partager des types et des schémas de validation entre les deux applications via un paquet commun. Le tableau suivant récapitule la stack par couche, avec sa version.

Couche	Technologies	Rôle
Frontend	Next.js 16, React 19, TailwindCSS v4, shadcn/ui, TanStack Query v5, Axios, Uppy	Rendu (RSC + SSR), UI accessible, data fetching, upload S3
Backend	NestJS 11, TypeORM 0.3, Passport (JWT), BullMQ, Sharp, nestjs-zod	API REST modulaire, ORM, file de traitement, images
Partagé	@repo/shared (Zod 4), types et constantes	Contrats d'API et permissions, source unique front/back
Données	PostgreSQL, PgBouncer, Redis (ioredis)	Persistance relationnelle, pooling, cache & file
Stockage	AWS SDK v3 (S3, CloudFront signer)	Upload multipart, signature d'URLs
Outillage	Turborepo, Bun, Prettier, ESLint, Swagger	Build monorepo, formatage, documentation API

Tableau 8.2.1 - Technologies et outils par couche.

8.2.2 - Outils de développement et de gestion de projet

Monorepo Turborepo + Bun : Les deux applications et les paquets partagés cohabitent au sein d'un unique dépôt. Turborepo parallélise et met en cache l'exécution des tâches (build, test, lint), tout en permettant de générer un sous-ensemble minimal de fichiers. Cela permet de construire chaque image Docker de manière isolée, sans embarquer le code des autres applications du monorepo.

Documentation API auto-générée (Swagger) : Chaque point d'accès (endpoint) est documenté par des annotations dans le code. La spécification OpenAPI est ainsi générée automatiquement, servant à la fois de documentation vivante et d'interface pour les tests manuels.

Gestion de version et suivi (Git / GitHub) : L'organisation du code s'appuie sur des branches thématiques et des commits directement articulés autour des user stories. Les flux de travail (workflows) de la CI/CD via GitHub Actions sont également versionnés et stockés directement avec le code source.

8.2.3 - Algorithmes notables

Deux algorithmes clés portent la valeur différenciante du produit et méritent d'être détaillés :

Extraction de palette en espace perceptuel (Sharp + OKLab) : Lors de l'upload, l'image est réduite en une vignette de 50×50 pixels, puis analysée par un algorithme de k-means déterministe au sein de l'espace colorimétrique OKLab. Cela permet d'extraire une palette de 3 à 5 couleurs pondérées plutôt qu'une simple moyenne globale. L'espace OKLab étant

perceptuellement uniforme, ce regroupement respecte fidèlement la perception humaine, contrairement à l'espace RVB classique.

Classement dans un atlas chromatique fixe : Chaque couleur extraite est ensuite associée de manière déterministe à une grille fixe composée de 53 cellules (12 teintes $\times$ 4 clartés, complétées par les tons neutres). Cette classification garantit des résultats stables d'un chargement à l'autre, facilement nommables et adaptés à la mise en cache ainsi qu'à l'accessibilité. À l'inverse, un calcul de partitionnement (clustering) dynamique exécuté à chaque requête aurait généré des catégories instables et impossibles à indexer.

8.3 - Surveillance et monitoring de la sécurité

8.3.1 - Outils de monitoring

L'observabilité repose sur une stack unifiée et légère, consultable depuis une seule interface (Grafana) : Prometheus collecte les métriques (modèle pull, chaque service expose un endpoint /metrics), Loki agrège les logs (indexés par labels, alimentés par Promtail), et cAdvisor expose les métriques des conteneurs. Cette unification permet de corréler, sur un même écran, un pic de latence avec les logs d'erreur émis au même instant.

Outil	Rôle	Ce qui est observé
Prometheus	Métriques (pull / scrape)	Latence, débit, taux d'erreur, ressources conteneurs
Loki + Promtail	Logs structurés (JSON)	Logs applicatifs corrélés par identifiant de requête
Grafana	Visualisation	Tableaux de bord unifiés métriques + logs (alerting : évolution)
cAdvisor	Métriques conteneurs	CPU, mémoire, I/O par conteneur

Tableau 8.3.1 - Stack d'observabilité.

Côté API, le logging est assuré par Pino via nestjs-pino. Chaque requête HTTP produit un log JSON avec un identifiant unique (reqId) propagé à tous les logs de la requête. En cas d'incident, ce reqId permet de reconstituer dans Grafana/Loki l'intégralité d'une requête. Les sérialiseurs sont configurés pour ne jamais journaliser de données sensibles (mots de passe, tokens).

8.3.2 - Techniques de détection et de réponse aux incidents

La sécurité opérationnelle combine prévention, détection et réaction.

Étape	Moyens en place	Objectif
Prévention	Argon2, JWT HTTP-only, CSRF, ownership, rate limiting, requêtes paramétrées	Réduire la surface d'attaque applicative
Durcissement système	SSH par clé uniquement, pas de root, UFW deny-all, fail2ban	Limiter l'accès au serveur et bannir les tentatives d'intrusion
Détection	Métriques (taux de 4xx/5xx, 429), logs corrélés, tableaux de bord Grafana	Repérer un comportement anormal en temps quasi réel
Réaction	Révocation de refresh token, rotation de secret, restart Swarm, rollback image	Contenir et corriger rapidement un incident

Tableau 8.3.2 - Détection et réponse aux incidents.

Un point important de la sécurité opérationnelle concerne la gestion des secrets. Aucun secret n'existe en clair dans le dépôt ni sur le disque du serveur. Les valeurs sensibles sont stockées dans GitHub Secrets puis transformées en Docker Secrets (chiffrés dans le raft Swarm, montés en mémoire dans les conteneurs).

8.3.3 - Tableaux de bord et indicateurs de supervision

Les métriques et les logs sont centralisés et exploités à travers deux tableaux de bord Grafana dédiés : l'un pour la télémétrie de l'API et l'autre pour l'analyse des logs applicatifs.

Les indicateurs détaillés ci-dessous permettent de détecter les anomalies de manière proactive, avant qu'elles ne se transforment en incidents visibles pour l'utilisateur. Bien que ces données soient aujourd'hui consultables visuellement sur les tableaux de bord, la mise en place de règles d'alerte automatiques (définition de seuils déclenchant des notifications) constitue une perspective d'évolution à court terme. La pile technique actuelle permet nativement cette configuration, bien qu'aucune règle n'ait encore été déployée à ce stade.

Indicateur	Source / mesure	Signal de vigilance
Taux d'erreur HTTP	Part de réponses 5xx	Hausse anormale à surveiller
Latence des requêtes	Temps de réponse (percentiles)	Dégradation soutenue à surveiller
Tentatives bloquées	Réponses 429 (rate limiting)	Pic = possible attaque / abus
Santé des réplicas	Conteneurs up/restarts	Redémarrages répétés = investigation
Ressources conteneurs	CPU / mémoire (cAdvisor)	Saturation = mise à l'échelle
File d'optimisation	Jobs en attente / échoués (BullMQ)	Accumulation = goulot d'étranglement

Tableau 8.3.3 - Indicateurs de supervision suivis sur les tableaux de bord.

La corrélation est la valeur ajoutée du dispositif : face à un pic de latence, l'opérateur bascule, dans la même interface, vers les logs émis au même instant pour ce service, retrouvés par identifiant de requête. Le diagnostic passe ainsi de la supposition à l'observation.

8.4 - Plans de sauvegarde et de récupération

8.4.1 - Stratégie de sauvegarde des données

La donnée critique est la base PostgreSQL (comptes, métadonnées, albums, partages). Elle fait l'objet de sauvegardes automatisées quotidiennes, pilotées par un workflow GitHub Actions dédié (« backup.yml ») et restaurables par un workflow symétrique (« restore.yml »). Les fichiers binaires sur S3 bénéficient quant à eux de la durabilité native du service et des règles de cycle de vie configurées sur le bucket (expiration des uploads incomplets et des fichiers bruts temporaires).

Élément	Fréquence / nature	Contenu	Récupération
PostgreSQL	Quotidienne (workflow planifié)	Dump complet de la base	Restauration via restore.yml
Fichiers S3	Durabilité native du service	Objets versionnables	Indépendant du VPS
Infrastructure	Versionnée (Git)	stack.yml, playbooks Ansible	Recréable à l'identique
Secrets	GitHub Secrets (chiffré)	Valeurs sensibles	Recréés par Ansible au déploiement

Tableau 8.4.1 - Plan de sauvegarde par type de donnée.

8.4.2 - Plan de reprise après sinistre

La stratégie de reprise après sinistre repose sur un principe fondamental : l'infrastructure étant entièrement définie par le code (*Infrastructure as Code*), un nouveau serveur peut être reconstruit à l'identique en quelques commandes.

En cas de perte du serveur, le scénario de restauration type est **entièrement orchestré par des playbooks Ansible**, découpé selon les étapes suivantes :

1. **Provisionnement :** Déploiement d'un nouveau VPS vierge avec injection automatique de la clé SSH de déploiement dès l'initialisation (*boot*).
2. **Configuration de l'OS (Ansible) :** Exécution du premier playbook pour durcir et sécuriser le système d'exploitation, puis installer le moteur Docker.
3. **Orchestration du cluster (Ansible) :** Exécution du deuxième playbook dédié à l'initialisation et à la configuration du cluster Docker Swarm.
4. **Gestion des secrets (Ansible) :** Exécution du troisième playbook pour l'injection sécurisée des variables d'environnement indispensables via les *Docker Secrets*.

5. **Déploiement applicatif (Ansible) :** Lancement automatisé de la commande docker stack deploy pour instancier les services et les conteneurs.
6. **Restauration des données (Ansible) :** Réintégration automatique du dernier *dump* de la base de données PostgreSQL via le playbook de restauration dédié (restore.yml).

Cette reproductibilité automatisée réduit drastiquement le temps de reprise par rapport à une configuration manuelle. Les limites actuelles de cette infrastructure comme le nœud PostgreSQL unique et le manager Swarm unique sont identifiées comme des axes d'évolution majeurs (réplication, cluster à trois managers).

Sauvegarde & reprise - L'approche «Infrastructure as Code » n'est pas qu'un confort de déploiement, c'est aussi le plan de reprise après sinistre. Le code Ansible remplace une documentation qui devient vite obsolète et garantit une reconstruction fidèle du serveur.

9 - Gestion du projet et management

Ce chapitre couvre le pilotage du projet : note de cadrage, méthodologie, découpage et planification, suivi de l'avancement et gestion des risques, et communication avec les parties prenantes. Le projet ayant été mené individuellement, les outils classiques de management d'équipe (RACI, instances de gouvernance) sont présentés dans leur forme transposée à ce contexte, en explicitant l'adaptation.

9.1 - Note de cadrage

La note de cadrage fixe le contexte, les objectifs, les contraintes et les ressources du projet avant tout développement. Elle constitue le contrat que le projet se donne à lui-même (cf Annexe 13 : Extrait de la note de cadrage).

Rubrique	Contenu
Contexte	Projet de fin de cycle (titre EISI, RNCP 40573). Démontrer la maîtrise des quatre blocs de compétences sur un produit complet et déployé.
Objectif	Concevoir, développer et déployer une plateforme de gestion de photos privacy-first intégrant une fonctionnalité différenciante (exploration chromatique).
Méthode	Cycle itératif inspiré de l'agilité (incréments fonctionnels, priorisation MoSCoW), adapté à un travail individuel.

Périmètre (MVP)	Les user stories « Must Have » : authentification, upload, galerie, albums, partage, exploration chromatique.
Hors périmètre	Application mobile native, reconnaissance de visages/objets, édition photo avancée, expiration des liens de partage.
Ressources	Un développeur (profil full-stack / DevOps). Outils gratuits ou open source. Un VPS et un compte AWS pour le stockage objet.
Contraintes	Délai limité, travail solo, conformité RGPD et accessibilité, démarche d'éco-conception, budget maîtrisé.
Indicateurs	Couverture de tests ≥ 80 %, déploiement automatisé fonctionnel, temps de réponse galerie acceptable, zéro secret en clair dans le dépôt.

Tableau 9.1 - Synthèse de la note de cadrage.

9.1.1 - Indicateurs clés de performance (KPI)

Le pilotage s'appuie sur des indicateurs mesurables, suivis tout au long du projet.

Axe	Indicateur	Cible	Résultat
Qualité	Couverture de tests + quality gate	≥ 80 % (4 métriques) + SonarQube	Seuil Jest bloquant + gate SonarQube en CI
Fiabilité	Suites de tests au vert en CI	100 % des suites	Atteint (blocage CI si échec)
Sécurité	Secrets en clair dans le dépôt	0	Atteint (GitHub + Docker Secrets)
Livraison	Déploiement automatisé sur push main	Sans intervention manuelle	Atteint
Performance	Galerie en défilement infini, images optimisées	Chargement fluide	Atteint (WebP + CDN + pagination)

Tableau 9.1.1 - KPI du projet et niveau d'atteinte.

9.1.2 - Budget estimatif

Le budget est présenté à deux niveaux complémentaires : une valorisation « conditions professionnelles » (le projet comme s'il était facturé dans une entreprise), puis le coût réellement décaissé dans le cadre de la certification. La charge totale (74 jours homme, qui est détaillée lot par lot dans le WBS) est valorisée à un TJM de 550 €/jour (développeur full-stack, fourchette junior/intermédiaire du marché français).

Poste	Contenu	Estimation
Développement technique (~48 j)	Infra/CI-CD, auth & sécurité, fonctionnalités cœur, killer feature, pages, observabilité ; tests unitaires & intégration inclus	26 400 €
Conception & documentation (~21 j)	Cadrage, personas, user stories, modélisation, maquettes, ADR..	11 550 €
Recette & qualité (~5 j)	Recette manuelle de bout en bout, tests E2E, reprise d'anomalies	2 750 €
Infrastructure & hébergement (1 an)	VPS EU + stockage objet + CDN	< 1 K€
Licences / outils	Stack 100 % open source	0 €
Contingence (15 %)	Aléas techniques, dérive de périmètre	6 105 €
TOTAL (valorisation pro)	≈ 74 j·h × 550 €/j + infrastructure + contingence	≈ 48 000 €

Tableau 9.1.2 - Budget global (valorisation en conditions professionnelles).

En réalité, le travail étant fourni par le porteur dans un cadre de formation, le coût monétaire effectivement décaissé se limite à l'infrastructure : VPS EU (50 €/an), stockage S3/CloudFront (quelques euros à quelques dizaines d'euros par an selon le volume), nom de domaine (15 €/an, TLS gratuit via Caddy), outillage 0 € (stack open source). Soit un total réel de l'ordre de 65 à 100 €/an. Le seul véritable investissement est le temps d'ingénierie. L'infrastructure mutualisée sur un VPS unique minimise les coûts récurrents. Le détail figure dans le document budget-estimatif.md du dépôt.

9.2 - Méthodologie, découpage et planification

9.2.1 - Méthodologie de conduite

La démarche adoptée repose sur un cycle de développement itératif et incrémental. À l'opposé d'un cycle en V rigide qui est peu adapté aux exigences d'un produit exploratoire mené en solo, le projet a progressé par jalons fonctionnels directement livrables. Lors de chaque itération, une tranche verticale complète (allant de la modélisation de la base de données jusqu'à l'interface utilisateur) a été conçue, développée, testée puis intégrée. La méthode de priorisation MoSCoW a ainsi servi de fil conducteur, garantissant le déploiement prioritaire des fonctionnalités indispensables (Must Have) et la stabilisation du socle technique avant l'intégration des options de confort.

9.2.2 - Découpage du projet (WBS)

Le projet a été décomposé en dix lots de travail (Work Breakdown Structure), chaque feuille étant une tâche estimable et assignable. Le WBS distingue les lots de conception/documentation, de socle technique (DevOps), de fonctionnalités, et de qualité (cf Annexe 18 : WBS - Découpage par lots).

Lot	Intitulé	Contenu	Charge
1	Cadrage & avant-projet	Initialisation dépôt, étude d'opportunité, note de cadrage, matrice de décision killer feature	3 j
2	Conception & documentation	Personas, benchmark, user stories, veille, cartographie SI, modélisation, ADR, UML, maquettes, Swagger, EBIOS	12 j
3	Socle technique & infrastructure (DevOps)	Monorepo, Docker Swarm, CI/CD, migrations, Ansible, Docker Secrets, backup/restore	14 j
4	Authentification & sécurité	JWT + Argon2, middleware, throttling, sécurité upload, URLs signées, CSRF	8 j
5	Fonctionnalités cœur	Upload, galerie, détail/suppression, partage photo, albums, partage d'album	10 j
6	Killer feature exploration Chromatique	R&D OKLCH, extraction de palette, atlas fixe, page d'exploration, rework v2	8 j
7	Pages vitrine (frontend)	Landing, fonctionnalités, tarifs, login/inscription, responsive	5 j
8	Observabilité	Métriques Prometheus, logs Loki/pino, dashboards Grafana	3 j
9	Qualité & recette	Tests unitaires, intégration, E2E, web, quality gate SonarQube, recette manuelle	5 j
10	Pilotage & livraison	Documents de pilotage, préparation soutenance	6 j

Tableau 9.2.2 - WBS : lots de travail et charge estimée (total ≈ 74 j·h).

9.2.2.1 - Charge théorique, durée et effort réel

Trois grandeurs distinctes ne doivent pas être confondues. La charge théorique (≈ 74 j·h) est la base de chiffrage du budget. La durée calendaire (≈ 24 semaines, du 5 janvier au 24 juin 2026) est le temps écoulé, à temps partiel. L'effort réellement consenti (≈ 56 j·h) est inférieur à l'estimation professionnelle.

Grandeur	Valeur	Nature
Durée calendaire	≈ 24 semaines (05/01 au 24/06/2026)	Temps écoulé, à temps partiel
Charge théorique	≈ 74 j·h	Base de chiffrage du budget (conditions pro)
Effort réel cumulé	≈ 56 j·h	8 sem. intensives (≈ 40 j·h) + 16 sem. à ≈ 1 j (≈ 16 j·h)

Tableau 9.2.2.1 - Charge théorique, durée calendaire et effort réel.

L'écart (~25 %) entre charge théorique et effort réel est cohérent : un chiffrage en conditions professionnelles intègre une marge de coordination d'équipe et de montée en compétence qu'un développeur solo, seul concepteur de son architecture, n'a pas à supporter.

9.2.3 - Phases du projet

Phase	Activités principales	Livrables
Initiation	Veille, benchmark, personas, user stories, ADR	Cahier des charges, 15 ADR
Conception	Architecture, modèle de données, maquettes, choix techniques	Cartographie SI, schéma BDD
Réalisation	Développement incrémental front + back, fonctionnalité chromatique	MVP fonctionnel
Industrialisation	CI/CD, IaC, secrets, observabilité	Déploiement automatisé observé
Vérification	Tests pluri-niveaux, couverture, durcissement	Suite de tests bloquante
Clôture	Bilan, documentation, rédaction du dossier	Dossier professionnel

Tableau 9.2.3 - Phases du projet et livrables associés.

9.2.4 - Planning (diagramme de Gantt)

La fenêtre calendaire s'étend du 5 janvier au 24 juin 2026. La modélisation technique du planning est versionnée en langage Mermaid au sein du dépôt, tandis que sa restitution graphique sous forme de diagramme de Gantt est consultable en annexe (*cf. Annexe 3 : Diagramme de Gantt*).

Plusieurs tâches se chevauchent volontairement : la veille, les ADR et la documentation ont été menés en continu, et l'observabilité comme la sécurité ont été traitées au fil de l'eau plutôt qu'en fin de parcours.

9.2.5 - Séquençage et chemin critique (logique PERT)

Le chemin critique représente la plus longue séquence de tâches interdépendantes dont le retard impacterait directement la date de livraison finale du projet. Pour Kroma, cette chaîne lie de manière séquentielle le socle d'industrialisation, l'authentification, les fonctionnalités cœur, et enfin la validation de la qualité.

Séquence du chemin critique : Initialisation du dépôt → CI/CD → Monorepo & Docker Swarm → Authentification JWT → Téléversement des images → Galerie → Gestion des albums → Exploration Chromatique (v2) → Tests applicatifs → Quality Gate SonarQube.

Tâches à flottement (hors chemin critique, menées en parallèle) : documentation technique, pages vitrines, observabilité (Prometheus / Grafana) et fonctionnalités de partage.

Justification : Ce séquencement s'explique par des dépendances techniques strictes. Aucun déploiement n'est réalisable sans l'infrastructure Swarm et la chaîne CI/CD préalablement configurées. De même, l'accès aux fonctionnalités applicatives reste conditionné par le module d'authentification. L'exploration chromatique, quant à elle, dépend directement du téléversement car c'est à l'ingestion que chaque photo est indexée (palette OKLCH et cellules d'atlas), l'album ne constituant qu'un filtre optionnel. Par défaut, l'exploration porte sur l'ensemble de la bibliothèque. La galerie et les albums ne sont donc pas des prérequis de la killer feature, mais font partie du cœur applicatif que la campagne de tests finale doit couvrir. Enfin, cette phase de validation (tests et passerelle de qualité bloquante) garantit l'intégrité du code avant toute livraison sur la branche principale. Les tâches à flottement, bénéficiant d'une marge de manœuvre temporelle, ont pu être menées en parallèle sans jamais ralentir le flux principal

9.3 - Suivi de l'avancement et gestion des risques

9.3.1 - Outils de suivi

À défaut d'une équipe, la rigueur est portée par l'outillage. Le suivi des tâches s'est appuyé sur un backlog Kanban Trello (qui tient lieu de « daily », cf Annexe 16 : Trello) et sur le flux Git où chaque évolution passe par une branche et une pull request, ce qui trace les décisions et impose une auto-revue structurée avant fusion sur la branche principale. La CI a joué un rôle de garde-fou permanent : tant qu'une suite de tests échoue, que la couverture passe sous le seuil ou que le quality gate SonarQube est rouge, la pull request est bloquée. L'avancement « terminé » d'une tâche n'est réel qu'une fois la CI au vert. Le quality gate SonarQube agit ainsi comme un relecteur automatisé.

9.3.2 - Gestion des risques projet

Au-delà des risques de sécurité du SI, le pilotage a suivi des risques propres à la conduite du projet, avec leur stratégie de mitigation.

Risque	Description	Gravité	Mitigation
Risque de périmètre	Ambition trop large pour un délai solo	Élevé	Priorisation MoSCoW stricte, MVP limité aux 16 Must Have
Risque technique	Blocage sur une brique récente (Tailwind v4, Bun)	Moyen	Veille continue, repli possible vers une alternative éprouvée
Risque de qualité	Régressions invisibles sans tests	Élevé	Seuil de couverture bloquant en CI, tests à plusieurs niveaux
Risque de sécurité	Fuite de secrets	Élevé	Docker Secrets, audit du dépôt, aucun secret en clair
Risque de déploiement	Déploiement cassé en production	Moyen	Rolling updates, migrations idempotentes, rollback d'image

Tableau 9.3.2 - Matrice des risques projet.

9.4 - Management et communication avec les parties prenantes

9.4.1 - Parties prenantes et matrice de responsabilités (RACI)

La gouvernance du projet s'articule autour de cinq rôles distincts. En tant que chef de projet et développeur, j'ai incarné une double casquette, matérialisant ainsi la frontière nécessaire entre les décisions de pilotage et les impératifs de réalisation. Cette structure est complétée par trois autres piliers : le formateur/mentor (La Plateforme), garant du conseil et de la validation pédagogique, les testeurs (mon entourage), qui ont assuré les retours d'usage et enfin le jury, chargé de la validation finale.

Livrable	Chef projet	Dev/Archi	Formateur	Jury	Testeurs
Étude d'opportunité / note de cadrage	R	C	A	-	-
User stories / modélisation / maquettes	C	R	A	-	-
ADR / choix techniques	A	R	C	-	-
Analyse de risques (EBIOS)	C	R	A	-	-
Planning (WBS / Gantt)	R	C	A	-	-

Fonctionnalités cœur / killer feature	A	R	C	-	C
Infrastructure / CI-CD / DevOps	A	R	C	-	-
Tests & qualité	A	R	C	-	-
Dossier professionnel / soutenance	R	C	C	A	-

Tableau 9.4.1 - Matrice RACI (R réalise · A approuve · C consulté · - non concerné).

Deux lectures sont décisives. Le formateur / l'école valide les livrables au fil de l'année. C'est la validation continue qui rythme réellement le projet. Le jury, lui, n'intervient qu'en fin de parcours : il n'a aucune visibilité pendant le projet et n'apparaît que sur ce qu'il évalue effectivement, le dossier et la soutenance. Les livrables techniques sont réalisés par le développeur et approuvés par le chef de projet. Ici, l'auto-arbitrage est encadré par la relecture du formateur et le contrôle qualité automatisé.

9.4.2 - Gouvernance et instances (transposées)

En milieu professionnel, la gestion de projet s'appuie sur des rituels collectifs tels que les comités de pilotage (COPIL), les comités de projet, les *dailies* ou les *sprint reviews*. Dans le cadre d'un projet en autonomie, ces instances collaboratives sont adaptées sous forme de points de validation personnels, d'auto-organisation et de contrôles automatisés.

Instance (modèle pro)	Transposition dans le projet	Objet
COPIL (décisions stratégiques)	Points de validation avec le formateur aux jalons	Go / No-Go de phase
Comité projet (suivi)	Auto-revue de fin de lot + retours formateur	Suivi & réajustement
Daily stand-up	Auto-organisation : backlog Trello + commits réguliers	Suivi des tâches
Sprint review	Jalons fonctionnels testés avec l'entourage	Recette informelle
Soutenance	« COPIL simulé » : le jury joue le rôle du décideur	Décision de certification

Tableau 9.4.2 - Instances de gouvernance transposées au contexte solo.

9.4.3 - Plan de communication

Travaillant seul sur ce projet, j'ai misé sur une organisation écrite et structurée pour suivre mon avancée et partager mon travail efficacement.

Documentation vivante : Pour garder une trace claire de mes choix techniques, j'ai tout consigné par écrit. Le projet intègre un historique de mes décisions d'architecture (ADR), des notes de développement et une documentation interactive Swagger pour tester l'API.

Retours d'usage : Ne disposant pas d'une équipe de testeurs professionnels, j'ai demandé à des proches de prendre en main l'application. Leurs retours concrets m'ont aidé à repérer les petits défauts visuels et à améliorer la navigation (menus, boutons). Ces avis humains ont complété mes tests techniques automatisés (unitaires, intégration, E2E).

Restitution au jury : Ce dossier de synthèse et la soutenance orale constituent les deux livrables officiels pour présenter et valider le résultat de mon travail devant le jury.

9.4.4 - Capitalisation des compétences

Une attention particulière a été portée à la capitalisation : chaque décision d'architecture est tracée dans un ADR, chaque sujet technique délicat (pipeline d'upload, URLs signées, secrets, tests) fait l'objet d'une note de développement détaillée. Cette documentation remplit deux fonctions : garantir la passation des connaissances en cas de changement d'équipe, et servir de support de montée en compétence réutilisable.

Management du savoir : Dans un projet solo, le risque n'est pas la coordination d'équipe mais la perte de contexte. La réponse retenue est la documentation systématique des décisions (ADR) et des points techniques difficiles (notes de dev), qui transforme l'expérience individuelle en savoir transmissible.

10 - Conception et développement de l'application informatique

Ce chapitre est le cœur technique du dossier. Il détaille la conception de l'application (architecture, modèle de données, environnement, patterns), son développement (pipeline d'upload, fonctionnalité différenciante, sécurité), la stratégie de tests qui en garantit la qualité, et enfin les modalités de livraison. Les décisions structurantes sont formalisées en ADR (Architecture Decision Records), synthétisées ici et accessible sur GitHub au sein du répertoire docs/.

10.1 - Conception

10.1.1 - Introduction et principes

La conception applique trois principes directeurs : modularité (un module = une responsabilité métier), découplage par contrats (interfaces et injection de dépendances), et sécurité par conception (authentification, autorisation et validation pensées dès la structure, pas ajoutées

après coup). L'architecture est de type client-serveur en couches, avec un frontend Next.js et une API REST NestJS partageant des types via un paquet commun.

10.1.2 - Décomposition de l'application

Le backend est découpé en modules NestJS au périmètre clair. Chaque module encapsule son controller (exposition HTTP), son service (logique métier), ses entités, ses repositories et ses DTO. Les dépendances entre modules passent par des services exportés, jamais par accès direct aux données d'un autre module.

Module	Composants	Responsabilité
auth	AuthController, AuthService, JwtStrategy, guards (JWT, CSRF, Roles)	Authentification, émission/rotation des tokens, autorisation
users	UsersController, UsersService	Comptes, profil, rôles
photo	PhotoController, PublicPhotoController, PhotoService, PhotoProcessor	Upload, métadonnées, statut, exploration chromatique, partage
album	AlbumController, AlbumService	Albums, photos d'album, membres
aws	AwsService	Multipart S3, URLs CloudFront signées, flux d'upload/download
redis	RedisService	Cache et propriété des uploads en cours
common	Filtres d'exception, DTO d'erreur, exceptions applicatives	Format d'erreur homogène sur toute l'API

Tableau 10.1.2 - Décomposition modulaire du backend.

Côté frontend, l'organisation suit l'App Router de Next.js : un groupe de routes publiques (accueil, connexion, inscription, page de partage public) et un groupe de routes authentifiées (galerie, albums, exploration), protégées au niveau du middleware avant tout rendu de page.

10.1.3 - Données utilisées (modèle de données)

Le modèle relationnel est volontairement simple et normalisé. Les identifiants sont des UUID (non devinables, utiles pour le partage public). Les suppressions en cascade garantissent la cohérence et facilitent le droit à l'oubli : supprimer un utilisateur supprime ses photos, albums et tokens.

Table	Colonnes principales	Relations
users	id (UUID), email (unique), password (haché, non sélectionné), role, firstName, lastName, createdAt	-
photos	id, s3Key, originalName, status (enum), fileSizeBytes, dominantColor (#RRGGBB), palette (jsonb), colorCells (text[], index GIN), shareToken (unique), userId, createdAt, updatedAt	user (N-1, cascade)
albums	id, name, userId, createdAt, updatedAt	user (N-1, cascade)
album_photos	albumId, photoId, addedAt (clé composite)	album & photo (cascade)
album_members	albumId, userId, createdAt (clé composite)	album & user (cascade)
refresh_token	id, tokenHash, isRevoked, expiresAt, userId, createdAt	user (N-1, cascade)

Tableau 10.1.3 - Schéma de la base de données (entités TypeORM).

Quelques choix méritent d'être soulignés. Le mot de passe est marqué « non sélectionné » par défaut : il n'est jamais renvoyé par une requête ordinaire. Le « shareToken » des photos est un jeton aléatoire unique permettant un accès public révocable sans authentification. La table « album_members » matérialise les albums collaboratifs (partage avec d'autres utilisateurs identifiés par email). La couleur d'une photo n'est pas réduite à une seule dominante : une palette pondérée (jsonb) et la liste des cellules d'atlas couvertes (tableau indexé en GIN) alimentent l'exploration chromatique et permettent à une photo d'appartenir à plusieurs couleurs. Afin d'offrir une vision globale de la structure de la base de données, la modélisation conceptuelle ainsi que sa traduction physique applicative sont consultables en annexe *(cf. Annexe 4 : Modèle conceptuel de données et Annexe 5 : Modèle physique de données)*.

10.1.4 - Environnement informatique proposé

Du point de vue de la conception, le point notable est l'homogénéité TypeScript : le paquet partagé « @repo/shared » contient les schémas de validation Zod et les constantes (rôles, permissions, codes d'erreur) utilisés à la fois par l'API et le frontend, garantissant qu'un contrat ne peut pas diverger entre les deux applications.

10.1.5 - Spécifications techniques et API

L'API REST expose des ressources cohérentes, documentées via Swagger (OpenAPI). Toutes les routes modifiant des données exigent un token CSRF (à l'exception des routes d'upload

multipart, voir plus bas) et toutes les routes privées passent par le guard JWT. Le tableau ci-dessous donne un aperçu représentatif des endpoints.

Méthode	Endpoint(s)	Rôle
POST	/auth/register · /auth/login · /auth/refresh · /auth/logout	Cycle d'authentification (cookies)
POST	/photos/uploads/multipart (+ sign-part, list-parts, complete, abort)	Upload multipart résilient
GET	/photos · /photos/colors[/:cellId] · /photos/quota · /photos/:id/status	Galerie, exploration chromatique, quota, statut
POST/DELETE	/photos/:id/share	Activer / désactiver le partage public d'une photo
GET/POST/PATCH/DELETE	/albums · /albums/:id (+ /photos, /members)	CRUD albums, contenu et membres
GET	/share/:token	Accès public à une ressource partagée (sans authentification)

Tableau 10.1.5 - Aperçu des principaux endpoints de l'API.

10.1.6 - Patterns de conception mobilisés

Plusieurs patterns structurent le code, chacun répondant à un besoin précis du projet plutôt qu'appliqué pour lui-même.

Pattern	Où dans le projet	Pourquoi
Injection de dépendances	NestJS (constructeurs)	Découpler les composants, faciliter le mock en test
Repository	Repositories TypeORM dédiés (photo, album-photo, album-member)	Isoler les requêtes SQL complexes
Strategy	PassportStrategy (JwtStrategy)	Brancher la vérification du JWT de façon interchangeable
Guard / Decorator	JwtAuthGuard, RolesGuard, CsrfGuard, @CurrentUser	Centraliser l'autorisation hors de la logique métier
Producer/Consumer	File BullMQ + PhotoProcessor	Sortir le traitement d'image du cycle requête/réponse
DTO + validation	Schémas Zod partagés (nestjs-zod)	Valider les entrées et garantir les contrats

Tableau 10.1.6 - Patterns de conception et justification.

10.1.7 - Architecture du frontend

Le frontend ne se réduit pas à une couche d'affichage : c'est une application à part entière, avec sa propre stratégie de rendu, d'authentification et de gestion d'état. Quatre décisions en structurent l'architecture.

Rendu hybride RSC + Client Components : Par défaut, les pages sont des React Server Components : elles récupèrent leurs données côté serveur, sans exposer d'endpoint intermédiaire ni alourdir le bundle JavaScript envoyé au navigateur. Seuls les composants réellement interactifs (upload, mutations, explorateur) sont des Client Components. Ce partage réduit le JavaScript client et offre un bénéfice de performance et d'éco-conception.

Protection des routes au niveau du middleware : La vérification du JWT s'effectue dans le middleware Next.js, avant même le rendu de la page donc un utilisateur non authentifié est redirigé vers la connexion sans qu'aucun rendu protégé n'ait lieu.

État serveur via TanStack Query + Axios : Dans les Client Components, TanStack Query gère le cache et l'invalidation après mutation. Un intercepteur Axios centralise l'attache des cookies et le renouvellement automatique du token, évitant toute duplication de la logique d'authentification.

Contrats partagés via Zod : Les mêmes schémas Zod valident les formulaires côté client (react-hook-form) et les entrées côté API : une règle de validation ne peut pas diverger entre les deux applications, puisqu'elle est définie une seule fois dans le paquet partagé.

Afin de visualiser la structure de navigation qui découle de cette logique, l'organisation complète des vues de l'application est cartographiée en annexe *(cf. Annexe 8 : Arborescence des pages)*.

L'interface s'appuie sur shadcn/ui (Radix UI couplé à Tailwind v4). Cette approche permet de s'approprier entièrement le code des composants au lieu de dépendre d'un paquet logiciel opaque, rendant ainsi chaque élément librement modifiable. En parallèle, Radix gère nativement toutes les exigences d'accessibilité (attributs ARIA, gestion du focus, navigation au clavier).

Afin de visualiser les différents éléments de l'interface, les maquettes conceptuelles de l'application sont consultables en partie en annexe *(cf. Annexe 10 : Maquettes)*.

10.1.8 - Justification des choix techniques structurants

Chaque brique majeure a été retenue après comparaison explicite d'alternatives, formalisée en ADR. Les tableaux ci-dessous synthétisent les choix les plus structurants. La logique est de retenir l'option qui satisfait le besoin au moindre coût de complexité.

10.1.8.1 - Framework backend (ADR-003)

Critère	NestJS	Express	Spring Boot
Architecture	Modulaire, opinionnée	Minimaliste, libre	Modulaire, opinionnée
TypeScript natif	Oui	Possible, non natif	Non (Java)
Injection de dépendances	Intégrée	Non	Intégrée
Cohérence avec Next.js	Forte (même langage)	Forte	Faible
Verdict	Retenu	Trop bas niveau	Stack différente

Tableau 10.1.8.1 - Choix du framework backend.

NestJS impose une structure claire (modules, controllers, services), fournit un écosystème intégré (authentification, validation, documentation) et son injection de dépendances native facilite les tests. Autant d'atouts décisifs pour un projet seul devant rester maintenable.

10.1.8.2 - Base de données et ORM (ADR-002, ADR-003)

Critère	PostgreSQL	MySQL	MongoDB
Modèle	Relationnel	Relationnel	Documents (NoSQL)
Types avancés (UUID, JSON)	Excellents	Bons	Bons
Intégrité (FK, transactions)	Forte	Bonne	Faible (pas de FK natif)
Adéquation aux données photos	Oui (relations claires)	Oui	Moins naturel
Verdict	Retenu	Alternative valable	Inadapté ici

Tableau 10.1.8.2 - Choix de la base de données.

Les données du projet (utilisateurs, photos, albums, partages) ont des relations claires et des contraintes d'intégrité fortes donc le relationnel s'impose. PostgreSQL a été préféré pour ses types natifs (UUID, JSON), sa maturité et son support TypeORM de référence. L'ORM TypeORM (approche code-first par décorateurs, migrations versionnées, intégration NestJS native) a été retenu face à Prisma, dont l'avantage de typage ne compensait pas l'intégration moins directe.

10.1.8.3 - Authentification : stockage du token et hachage (ADR-006)

Critère	Cookie HTTP-only	localStorage	sessionStorage
Accessible par JavaScript	Non (protégé)	Oui (vulnérable XSS)	Oui (vulnérable XSS)
Envoi automatique	Oui (cookie)	Non (manuel)	Non (manuel)
Protégé contre XSS	Oui	Non	Non
Verdict	Cookie HTTP-only	Écarté	Écarté

Tableau 10.1.8.3a - Stockage du token côté client.

Critère	Argon2id	bcrypt	PBKDF2
Résistance aux GPU	Excellente (memory-hard)	Bonne	Moyenne
Recommandation OWASP	1re recommandation	Acceptable	Acceptable
Verdict	Retenu	Alternative	Insuffisant

Tableau 10.1.8.3b - Algorithme de hachage des mots de passe.

L'architecture de sécurité retenue combine des jetons JWT en cookies HTTP Only, une protection CSRF par double soumission et un hachage Argon2id. Cette alliance garantit un haut niveau de protection tout en s'adaptant idéalement à une API multi instances. De plus, la conservation en base de données du jeton de rafraîchissement haché apporte une fonctionnalité cruciale. Elle rend possible sa révocation explicite lors d'une déconnexion, d'un changement de mot de passe ou d'une faille de sécurité, une action irréalisable avec un JWT intégralement sans état.

10.1.8.4 - Observabilité (ADR-005)

Critère	Prometheus + Loki + Grafana	Datadog	ELK
Coût	Gratuit (open source)	Payant	Gratuit mais lourd
Collecte	Pull (scrape)	Push (agent)	Push (agent)
Consommation ressources	Légère	Légère (cloud)	Lourde (JVM)
Logs + métriques unifiés	Oui (Grafana)	Oui	Kibana séparé
Verdict	Retenu	Coût	Trop lourd

Tableau 10.1.8.4 - Choix de la stack d'observabilité.

La stack Prometheus, Loki et Grafana rassemble les métriques et les logs sur une seule plateforme de supervision. Elle requiert beaucoup moins de ressources qu'un environnement ELK puisqu'elle s'affranchit de la JVM et n'indexe les journaux que par labels. Cette sobriété technique valide pleinement les objectifs d'éco-conception fixés pour l'application.

10.2 - Développement de l'application et de son environnement

Trois réalisations illustrent particulièrement les compétences de développement : le pipeline d'upload résilient, la fonctionnalité différenciante d'exploration chromatique, et le dispositif de sécurité applicative.

10.2.1 - Pipeline d’upload résilient et asynchrone

Le transfert fiable de fichiers volumineux représente un défi technique important. Une connexion mobile peut s'interrompre à tout moment et les opérations lourdes sur les images ne doivent pas saturer la boucle d'événements de l'application. La solution mise en œuvre repose sur un upload fragmenté directement vers le stockage S3 via des URLs pré-signées, couplé à un traitement asynchrone géré par une file de messages.

Le flux opérationnel s'articule selon la séquence suivante :

1. **Initialisation :** Le client initie le protocole de transfert auprès de l'API NestJS. Celle-ci crée l'identifiant d’upload et associe la clé de l'opération à l'identifiant de l'utilisateur au sein d'une base Redis.
2. **Transfert fragmenté :** Pour chaque fragment de fichier, le client demande une URL de dépôt signée à l'API, puis transmet directement la portion de données vers le stockage S3 sans surcharger le serveur d'application.
3. **Validation et planification :** Lors de la finalisation, l'API demande l'assemblage des pièces à S3, enregistre l'entité en base de données avec le statut en attente et ajoute une tâche d'optimisation dans la file BullMQ.
4. **Notification :** Le client interroge régulièrement l'API pour connaître l'état d'avancement. Le statut bascule à « validé » dès que le processeur termine ses calculs.

L'orchestration complète de ce flux entre le client, l'API, le stockage S3 et la file d'attente asynchrone est modélisée dans le diagramme de séquence disponible en annexe *(cf. Annexe 6 : Diagramme de séquence de l'upload d'une image)*.

Plusieurs mécanismes de protection encadrent ce processus :

Contrôle strict de la propriété : L'association entre l'identifiant du transfert et celui de l'utilisateur est conservée dans Redis avec une durée de vie d'une heure. Chaque action ultérieure requiert une vérification d'identité pour interdire toute manipulation malveillante.

Validation des formats : Une vérification stricte par schéma Zod limite les formats acceptés aux extensions JPEG, PNG, WebP et HEIC. L'extension finale découle uniquement du type MIME et ignore le nom de fichier d'origine.

Gestion des quotas et tolérance aux pannes : Le système valide l'espace disponible par rapport à la limite de 500 Mo par utilisateur avant de démarrer. Si l'enregistrement final échoue, une procédure de nettoyage supprime automatiquement le fichier sur S3 pour éviter l'accumulation de données orphelines.

Protection alternative contre les injections de requêtes : Les routes de ce protocole sont exemptées de la protection CSRF traditionnelle car le mécanisme de double soumission générait des erreurs d'authentification bloquantes lors des envois multiples. La sécurité reste

maximale puisque l'authentification est déléguée au système de validation d'identité basé sur Redis, tandis que les autres actions de l'application conservent la protection CSRF par défaut.

10.2.2 - Fonctionnalité différenciante : l'exploration chromatique

L'exploration chromatique est la fonctionnalité unique du produit : naviguer dans sa bibliothèque par la couleur. Elle a fait l'objet d'une matrice de décision parmi trois propositions (exploration chromatique, carte interactive, recherche sémantique par IA) et a été retenue pour son rapport valeur/innovation et sa faisabilité. Sa conception se décompose en trois temps : l'extraction d'une palette perceptuelle, son classement dans un atlas fixe, puis une restitution mise en cache.

10.2.2.1 - Extraction d'une palette en espace perceptuel (à l'upload)

La réduction d'une photographie à une unique couleur moyenne engendre généralement une teinte grisâtre et peu représentative, à l'image d'un coucher de soleil orange et bleu qui se transformerait en un marron terne. Afin de préserver la richesse visuelle des images, le système extrait une palette de trois à cinq couleurs pondérées. Ce calcul s'effectue au sein de l'espace colorimétrique OKLab, reconnu pour son uniformité perceptuelle, plutôt qu'en RVB dont la structure s'accorde mal avec la perception de l'œil humain.

Le protocole algorithmique suit les étapes suivantes :

Échantillonnage : L'image est d'abord réduite à une vignette de 50 par 50 pixels afin de travailler sur une base stable de 2500 pixels.

Conversion de l'espace : Les pixels subissent une transformation mathématique en trois temps pour passer du sRGB à l'espace OKLab. La première étape consiste à supprimer la correction gamma du sRGB pour obtenir un espace linéaire, indispensable pour effectuer des calculs physiques de lumière conformes. Ces valeurs sont ensuite projetées via une matrice dans l'espace LMS, un modèle qui simule la réaction physique des trois types de cônes récepteurs de la rétine humaine. Enfin, le système applique une racine cubique pour imiter la sensibilité de l'œil humain face à la lumière. Cela permet de séparer proprement l'image en trois informations distinctes : la luminosité pure (la clarté) d'un côté, et la teinte de l'autre, répartie sur deux axes de couleurs opposées (un axe allant du rouge au vert, et un autre allant du jaune au bleu).

Note technique : Cette implémentation algorithmique se base sur les travaux et les formules de conversion d'espace de Björn Ottosson (Oklab).

Segmentation : L'algorithme des k-means regroupe les pixels de la vignette en trois à cinq paquets selon leur ressemblance colorimétrique. Par nature, cet algorithme est instable car il choisit ses points de départ au hasard, ce qui signifie qu'une même image pourrait générer deux palettes légèrement différentes d'un clic à l'autre. Pour neutraliser ce problème, le système

utilise l'initialisation par le point le plus éloigné. Le premier point est fixé de manière stricte (le pixel en haut à gauche), puis le point suivant est choisi comme étant le pixel dont la couleur est la plus mathématiquement distante de la première, et ainsi de suite. Ce choix élimine complètement le hasard.

Pondération : Les couleurs retenues pour composer la palette finale sont issues des centroïdes de chaque cluster, calculés en intégrant un facteur de pondération proportionnel au volume de pixels associés. Cette approche permet de donner plus d'importance visuelle aux teintes dominantes du paysage. Pour finaliser ce traitement, l'algorithme applique un seuil de coupure à 6%. Toutes les nuances de couleur qui couvrent une surface inférieure à cette valeur critique sont éliminées pour éviter que des éléments isolés ne viennent polluer la palette.

Indexation : Chaque couleur retenue est ensuite répertoriée dans une cellule spécifique de l'atlas chromatique.

Ce choix technologique apporte une garantie cruciale de déterminisme. Une même image produira rigoureusement la même palette, ce qui permet de mettre les résultats en cache et d'assurer la stabilité de la carte de navigation. Concernant l'impact sur les performances, cette extraction est directement intégrée au pipeline d'optimisation existant par le biais d'un clone du flux Sharp. L'image n'est récupérée qu'une seule fois depuis le stockage S3 et le calcul des couleurs s'exécute en parallèle de la compression, évitant ainsi toute surcharge matérielle.

10.2.2.2 - Classement dans un atlas chromatique fixe

Au lieu de générer des catégories dynamiques par partitionnement (qui s'avèrent instables d'un chargement à l'autre, anonymes et impossibles à mettre en cache), le système distribue chaque couleur de manière déterministe dans une grille structurelle immuable. Cet atlas se compose de 12 teintes réparties sur le cercle chromatique par secteurs de 30 degrés, combinées à 4 niveaux de clarté. Une rangée additionnelle de 5 nuances neutres allant du noir au blanc complète ce dispositif pour atteindre un total de 53 cellules uniques.

L'algorithme de classification s'appuie sur les règles suivantes :

- Si la chrominance est inférieure au seuil de 0.045, la couleur rejoint une cellule neutre définie uniquement par sa clarté.
- Dans le cas contraire, la couleur est affectée à un secteur de teinte spécifique calculé par le ratio de son angle sur 30 degrés, associé à son niveau de clarté.

Une photographie se trouve ainsi rattachée à l'ensemble des cellules correspondant aux couleurs de sa palette. Cette association est enregistrée dans la colonne des cellules de couleur et bénéficie d'un index de type GIN, permettant à une même image de ressortir lors de recherches sur différentes teintes.

Cet atlas fixe garantit une parfaite stabilité visuelle, permet de nommer explicitement les catégories pour améliorer l'accessibilité comme le bleu clair ou le vert sombre, et s'avère entièrement cacheable. Enfin, l'isolation des seuils sous forme de constantes applicatives facilite tout recalibrage ultérieur sans altérer la logique du code.

10.2.2.3 - Restitution : API mise en cache et interface intégrée

Deux endpoints servent la feature. « GET /photos/colors » renvoie l'atlas complet avec le nombre de photos par cellule (agrégat SQL léger, mis en cache Redis et invalidé à chaque ajout/suppression). « GET /photos/colors/:cellId » renvoie les photos d'une cellule, paginées, en ne signant les URLs S3 que pour la page visible. Les deux acceptent un paramètre « albumId » optionnel pour restreindre l'exploration à un album. L'accès aux données est entièrement porté par la couche repository, le service se limitant à l'orchestration (cache, assemblage, mapping).

Le détail technique des requêtes, illustrant le rôle joué par le cache Redis pour garantir la fluidité de cette fonctionnalité particulièrement sollicitée, est schématisé en annexe *(cf. Annexe 9 : Diagramme de séquence de l'exploration chromatique)*.

L'interface (ChromaticExplorer) s'intègre de manière harmonieuse au design global de l'application à travers un nuancier interactif dont l'aspect visuel dépend directement du contenu de la base de données. Les cellules associées à des photographies existantes apparaissent de façon vive, affichant une lueur dont l'intensité est proportionnelle au nombre d'images indexées, tandis que les cases vides restent estompées en arrière-plan. La collection d'images vient ainsi illuminer le nuancier. Sur le plan de l'expérience utilisateur, une pression sur une couleur ouvre immédiatement la grille de résultats filtrée, qui bénéficie d'un mécanisme de défilement infini, tandis qu'un clic sur une image ouvre la boîte de dialogue détaillée déjà utilisée au sein de la galerie principale.

Innovation maîtrisée - L'exploration chromatique illustre une démarche d'ingénierie itérative et maîtrisée : un POC (moyenne RVB puis k-means recalculé à chaque requête) a été identifié comme instable et perceptuellement faux. Il a été remplacé par une v2 fondée sur une palette en espace perceptuel OKLab et atlas fixe déterministe et cacheable.

10.2.3 - Sécurité applicative

La sécurité a été conçue en défense en profondeur, en s'appuyant sur les recommandations OWASP. Le tableau récapitule les mesures et la menace qu'elles adressent.

Mesure	Périmètre	Menace adressée
Hachage Argon2id	Mots de passe	Vol de base / brute force hors-ligne (résistant GPU)
JWT en cookie HTTP-only	Sessions	Vol de token par XSS (invisible au JavaScript)
Access 15 min + refresh révocable	Sessions	Fenêtre d'exploitation réduite, révocation possible
Protection CSRF (double-submit)	Mutations	Falsification de requête inter-sites
RBAC statique + ownership	Autorisation	Accès à une ressource d'autrui
Rate limiting (Throttler)	Endpoints	Brute force, abus automatisés (HTTP 429)
Validation Zod en entrée	Toutes les routes	Données malformées, injections
URLs CloudFront signées	Diffusion	Accès direct non autorisé aux fichiers

Tableau 10.2.3 - Mesures de sécurité applicative (conformité OWASP).

La gestion des autorisations s'appuie sur la vérification systématique de la propriété des données, permettant de valider qu'un utilisateur n'accède qu'aux ressources qui lui appartiennent. Cette règle de sécurité contrôle la correspondance exacte entre l'identifiant du propriétaire de la ressource et celui de l'utilisateur à l'origine de la requête. L'ensemble de cette logique est encapsulé dans des guards réutilisables, ce qui permet de l'isoler complètement du code métier de l'application. Il est important de noter qu'une tentative d'accès à une photographie appartenant à un tiers renvoie systématiquement une erreur 404 au lieu d'une erreur 403. Ce choix stratégique empêche la divulgation indirecte de l'existence d'un identifiant valide au sein du système.

Le cycle de vie d'une session utilisateur illustre la superposition de ces protections :

La phase de connexion : L'API valide le mot de passe à l'aide de l'algorithme Argon2. Si la vérification réussit, le serveur dépose trois cookies distincts, à savoir le jeton d'accès valable 15 minutes, le jeton de rafraîchissement configuré pour 7 jours et le jeton CSRF. Les deux premiers cookies reçoivent la directive HTTP Only afin de les rendre invisibles au code JavaScript, tandis que le jeton CSRF reste lisible par le client. L'empreinte du jeton de rafraîchissement est simultanément enregistrée en base de données pour permettre sa révocation future.

La validation des requêtes protégées : Lors d'un appel ultérieur, le guard d'authentification extrait le jeton d'accès. En parallèle, le guard de sécurité CSRF compare la valeur de l'en-tête envoyée par le client avec celle du cookie selon le patron de la double soumission, interdisant ainsi l'exécution de requêtes forgées par des sites tiers. Une fois ces barrières franchies, le système valide la propriété de la ressource avant d'autoriser le traitement.

Le renouvellement des accès : Lorsqu'un jeton d'accès expire, le client reçoit un code de retour 401. Un intercepteur configuré dans le client appelle alors automatiquement la route de rafraîchissement pour générer un nouveau couple de jetons. Si le jeton de rafraîchissement a été révoqué ou est arrivé à expiration, l'utilisateur est immédiatement redirigé vers l'écran de connexion.

L'intégration complète de ces barrières de sécurité, de la génération des cookies jusqu'à la vérification des requêtes via le mécanisme de double soumission, est modélisée dans le diagramme de séquence disponible en annexe *(cf. Annexe 7 : Diagramme de séquence de l'authentification)*.

Cette architecture de sécurité repose sur des principes stricts et complémentaires. La durée de vie réduite du jeton d'accès limite la fenêtre d'exploitation en cas de vol, la persistance hachée du jeton de rafraîchissement assure sa révocabilité et l'invisibilité des données face aux scripts neutralise les risques d'attaques XSS. Enfin, l'obligation d'utiliser le protocole HTTPS sécurise l'ensemble des échanges et des filtres spécifiques appliqués sur les outils de journalisation masquent automatiquement les mots de passe ainsi que les jetons afin de ne jamais inscrire de données sensibles dans les logs.

10.2.4 - Gestion des erreurs et robustesse

Une API exploitable doit échouer de façon prévisible. Le projet centralise le traitement des erreurs dans un filtre d'exception global : toute exception, attendue ou non, est convertie en une réponse au format homogène (code d'erreur applicatif, message, détails éventuels) afin que le frontend traite les erreurs de manière uniforme. Les erreurs inattendues sont ramenées à un 500 générique, sans fuite d'information technique vers le client.

Les codes d'erreur applicatifs sont partagés (dans le paquet commun), consommés par le frontend pour afficher des messages localisés et pertinents. Cette séparation entre code d'erreur (stable, exploité par le code) et message (présentation) évite de coupler l'interface à des chaînes de caractères fragiles.

Code / origine	HTTP	Signification
AUTH_INVALID_CREDENTIALS	401	Email ou mot de passe invalide
QUOTA_EXCEEDED	403	Quota de stockage atteint
FORBIDDEN	403/404	Accès refusé (404 si divulgation d'existence à éviter)
Validation (Zod)	400	Données d'entrée non conformes au schéma
Rate limit (Throttler)	429	Trop de requêtes, en-tête Retry-After

Tableau 10.2.4 - Extrait de la taxonomie d'erreurs de l'API.

La robustesse se joue aussi dans les détails du pipeline : rollback de l'objet S3 si l'enregistrement échoue, nettoyage du fichier brut si le job d'optimisation échoue, tentatives répétées et temporisées à la connexion à la base au démarrage (jusqu'à ce que la base soit joignable). Autant de mécanismes qui transforment des échecs partiels en situations récupérables plutôt qu'en états incohérents.

10.3 - Stratégie de tests et suivi de la qualité

La fiabilité du système repose sur une couverture de tests multiniveaux qui respecte les principes de la pyramide des tests. Ce pipeline de validation est automatisé au sein de l'intégration continue, ce qui permet de bloquer automatiquement les déploiements en cas d'anomalie ou de baisse des indicateurs de couverture de code.

10.3.1 - Objectifs et niveaux de tests

L'effort de test s'appuie sur une stratégie en pyramide composée de 3 niveaux fonctionnels cumulables (soit 258 tests au total, cf Annexe 15 : Pyramide des tests) :

- Les tests unitaires forment la base de la pyramide avec environ 210 tests (~81 %).
- Les tests d'intégration & E2E API constituent le niveau intermédiaire avec 34 tests (~13 %).
- Les tests E2E navigateur complètent le sommet avec 14 tests (~5 %).

En complément de ce volet fonctionnel, la tenue en charge de l'infrastructure est validée par des tests transversaux sous k6 sur l'environnement déployé :

- Scénario brut : Montée en charge sur les routes publiques jusqu'à 300 utilisateurs virtuels (VUs).
- Parcours authentifié : Simulation de 50 VUs réalisant des parcours complets en boucle (navigation, filtres chromatiques, requêtes paginées, calcul de quota) sur une durée de 6 minutes.

Les résultats sur le parcours authentifié (32 req/s pour un total de 11 741 requêtes) valident l'ensemble des critères de qualité avec 0 % d'erreur et un p95 de 698,75 ms (pour un seuil requis de moins de 800 ms). La charge est équilibrée sur 4 répliques d'API dont l'empreinte mémoire reste très stable (~160 MiB) et le processeur sollicité à seulement ~40 à 50 % lors des pics de trafic, garantissant une marge de sécurité confortable pour le MVP (cf Annexe 14 : Tests de charges).

Niveau	Outils	Volume	Ce qui est vérifié
Unitaires (API)	Jest 30	22 fichiers, ~180 tests	Logique des services, controllers, guards, processor, couleur
Intégration (API)	Jest + testcontainers	~20 tests	SQL réel sur PostgreSQL éphémère (agrégats couleur, atlas, jointures)
E2E API	Jest + Supertest	14 tests	Routage, validation des DTO, cookies, codes HTTP
Unitaires (web)	Vitest + Testing Library	~30 tests	Composants UI, formulaires, validation Zod côté client
E2E navigateur	Playwright (Chromium)	14 tests	Parcours réels : login, register, routes protégées
Tests de charge	K6s	2 scénarios, jusqu'à 300 VUs simultanés (~6 min de montée par palier)	Tenue en charge de l'API : latence p95 < 800 ms, taux d'erreur < 5 %, comportement du rate-limiting sous saturation

Tableau 10.3.1 - Déclinaison de la stratégie en 6 suites de tests.

10.3.2 - Couverture, seuil bloquant et quality gate

Au niveau de l'API, le taux de couverture du code est mesuré par l'outil Istanbul selon quatre indicateurs fondamentaux que sont les instructions, les branches, les fonctions et les lignes. La configuration du framework Jest impose un seuil minimal strict de 80% pour chacun de ces critères. Si l'un de ces indicateurs descend en dessous de cette limite, l'exécution de la suite de tests échoue automatiquement, ce qui entraîne l'interruption immédiate de la chaîne d'intégration continue et empêche le déclenchement de l'étape de construction de l'image Docker.

En complément de ces seuils locaux, un second dispositif de contrôle repose sur l'intégration d'un outil de mesure de qualité SonarQube au sein du pipeline d'intégration continue. Cet outil analyse de manière centralisée l'API et l'application frontend en agrégeant les rapports de couverture au format LCOV. Toute proposition de modification dont les indicateurs de qualité (couverture des lignes modifiées, taux de duplication, détection de vulnérabilités ou anomalies de code) s'avèrent insuffisants est bloquée avant de pouvoir être fusionnée sur la branche principale.

Afin d'attester de la validation effective de ces métriques lors du processus d'intégration continue, une extraction des rapports générés par l'outil SonarQube est consultable en annexe *(cf. Annexe 11 : Rapports d'analyse SonarQube)*.

10.3.3 - Tests d'intégration sur base réelle (testcontainers)

La validation de requêtes complexes impliquant des fonctions de fenêtrage SQL pour l'extraction des couvertures d'album, des agrégations basées sur les fonctions COALESCE et SUM pour le calcul des quotas, ou des jointures avancées avec alias, s'avère impossible à réaliser via de simples mocks de test. Les mocks renvoyant une valeur arbitraire ne permet pas de certifier la validité syntaxique ou logique d'une requête en base de données. Pour pallier cette limite, ces requêtes font l'objet de tests d'intégration face à une véritable instance PostgreSQL. Cette base de données est instanciée dynamiquement dans un conteneur Docker isolé grâce à la bibliothèque Testcontainers, qui gère automatiquement la création du schéma relationnel ainsi que la purge complète des tables entre chaque scénario. Ce processus apporte la certitude que les instructions SQL s'exécutent correctement en environnement réel.

10.3.4 - Intégration à la chaîne d'intégration continue

L'exécution des tests est entièrement automatisée lors de chaque soumission de code, en amont des phases de build et de déploiement. L'ordonnancement des tâches au sein du workflow garantit l'arrêt immédiat du processus et interdit la création de toute nouvelle image Docker dès lors qu'une seule vérification échoue. Du côté de l'application frontend, le flux de travail applique la même rigueur en enchaînant d'abord les validations unitaires avec Vitest, puis les scénarios de bout en bout avec Playwright, avant d'autoriser la phase de compilation.

10.4 - Modalités de livraison

10.4.1 - Mise en service et vérification de service régulier

La livraison est automatisée de bout en bout : un push sur la branche principale, une fois les tests au vert, déclenche le build, la publication de l'image, la création des secrets et le déploiement en rolling update sans interruption de service. Les migrations de base de données s'exécutent au démarrage des conteneurs, protégées par un verrou applicatif pour éviter les exécutions concurrentes entre réplicas.

La vérification de service régulier s'appuie sur l'observabilité : après chaque déploiement, les tableaux de bord Grafana (taux d'erreur, latence, santé des réplicas) confirment le bon fonctionnement. Les health checks de Swarm conditionnent l'arrivée du trafic sur les nouvelles.

10.4.2 - Plan de maintenance prévisionnel

Une précision de contexte s'avère indispensable concernant la portée de ce projet. S'agissant d'un livrable de fin de formation, cette application n'a pas vocation à être exploitée ni maintenue

à l'issue de la soutenance devant le jury. Le plan présenté dans cette section a donc un caractère strictement prévisionnel et conditionnel. Il détaille la stratégie opérationnelle qui serait déployée dans l'hypothèse d'une véritable mise en production. L'enjeu n'est pas de contracter un engagement de support sur le produit, mais de prouver la capacité technique à planifier l'intégralité de son cycle de vie logiciel.

Dans ce cadre théorique, la gestion du cycle de vie s'organiserait autour des quatre piliers :

- **La maintenance corrective :** Elle prendrait en charge la correction des anomalies identifiées en environnement de production grâce aux outils d'observabilité et aux signalements des utilisateurs.
- **La maintenance évolutive :** Elle assurerait l'intégration progressive des nouvelles fonctionnalités inscrites sur la feuille de route, comme le droit à l'oubli, la gestion de l'expiration des liens ou le moteur de recherche avancée.
- **La maintenance adaptative :** Elle piloterait la mise à jour régulière des dépendances logicielles, des runtimes et des images de conteneurs pour garantir l'alignement avec les derniers correctifs de sécurité.
- **La maintenance préventive :** Elle encadrerait les audits de sécurité, les vérifications d'accessibilité, l'analyse des journaux d'événements ainsi que les tests de restauration des sauvegardes afin d'anticiper les risques de défaillance.

Le calendrier prévisionnel établi par la suite exprime les fréquences d'intervention pour chaque cas :

Tâche	Cadence	Contenu / norme visée
Mise à jour des dépendances	Mensuelle	Correctifs de sécurité, audit `bun audit`
Test de restauration backup	Trimestrielle	Vérifier la restaurabilité réelle (workflow restore)
Revue de sécurité	Trimestrielle	Revue OWASP, rotation des secrets sensibles
Audit d'accessibilité	Semestrielle	Contrôle RGAA/WCAG (Axe, lecteur d'écran)
Renouvellement TLS	Automatique	Géré par Caddy (Let's Encrypt)
Revue des tableaux de bord	Continue	Surveillance latence, erreurs, ressources

Tableau 10.4.2 - Calendrier de maintenance prévisionnel (hypothèse de mise en production).

10.4.3 - Conclusion du chapitre

L'application a été conçue de façon modulaire et sécurisée, développée autour d'un pipeline d'upload robuste et d'une fonctionnalité différenciante aboutie, puis verrouillée par une stratégie de tests à cinq niveaux dont le seuil bloque tout déploiement non conforme. Le tableau ci-

dessous récapitule les décisions d'architecture formalisées en ADR (accessible sur Github dans le répertoire docs/) :

Réf.	Décision d'architecture
ADR-001	Orchestration : Docker Swarm (plutôt que Kubernetes)
ADR-002	Base de données : PostgreSQL + PgBouncer
ADR-003	Backend : NestJS + TypeORM
ADR-004	Cache : Redis
ADR-005	Observabilité : Prometheus + Grafana + Loki
ADR-006	Authentification : JWT + refresh tokens + Argon2
ADR-007	Autorisation : RBAC statique + ownership
ADR-008	Frontend : Next.js + React Server Components
ADR-009	Monorepo : Turborepo + Bun
ADR-010	CI/CD : GitHub Actions + ghcr.io + déploiement SSH
ADR-011	État & data fetching : TanStack Query + Axios
ADR-012	Validation : Zod + schémas partagés
ADR-013	Rate limiting : NestJS Throttler
ADR-014	Logging structuré : Pino
ADR-015	UI : shadcn/ui + Radix UI + Tailwind v4

Tableau 10.4.3 - Les 15 ADR du projet.

11 - Concevoir et déployer des infrastructures DevOps automatisées

Ce chapitre démontre la conception et l'implémentation d'une chaîne DevOps complète : analyse de l'existant et stratégie, pipelines CI/CD, orchestration conteneurisée et infrastructure as code, observabilité, et démarche d'amélioration continue. Il s'appuie sur une infrastructure réellement déployée et opérée.

11.1 - Analyse de l'existant et stratégie DevOps

11.1.1 - Analyse de l'infrastructure et des processus initiaux

Au démarrage, le projet ne disposait d'aucune automatisation : build manuel, déploiement par copie de fichiers, secrets regroupés puis utilisés dans un script shell au déploiement. Cette approche, fonctionnelle, présentait plusieurs faiblesses structurelles qui ont motivé la stratégie DevOps.

Faiblesse de l'existant	Impact
Secrets en bloc unique	Tout-ou-rien, risque de fuite dans les logs, rotation impossible sans tout réécrire
Interpolation shell des secrets	Valeurs sensibles transitant en clair dans des scripts, reconstituables depuis les logs
Parsing fragile (grep)	Job de migration cassant dès qu'une valeur contient un caractère spécial
Provisioning manuel	Serveur non reproductible, configuration non documentée, dérive de configuration

Tableau 11.1.1 - Diagnostic de l'infrastructure initiale.

11.1.2 - Stratégie DevOps

La stratégie vise quatre propriétés cibles, alignées sur les objectifs métier et sur l'éco-responsabilité.

Tout en code et versionné : Pipelines, infrastructure et configuration décrits dans le dépôt donc un clone suffit à reconstruire le système.

Zéro secret en clair : Les valeurs sensibles n'existent qu'à deux endroits : GitHub Secrets et le raft Swarm chiffré. Jamais dans le dépôt, jamais sur le disque du serveur.

Reproductibilité : Rejouer un playbook ou un déploiement est sans danger, ce qui permet d'ajouter un nouveau serveur en quelques commandes.

Sobriété : Docker Swarm plutôt que Kubernetes, Loki plutôt qu'ELK : des outils légers pour un résultat équivalent à cette échelle, donc moins de ressources consommées.

Docker Swarm a été retenu car il couvre l'ensemble des besoins (réplication, load balancing intégré, rolling updates, health checks, réseau overlay, secrets) avec une charge opérationnelle bien moindre, ce qui, sur un projet solo contraint en temps, libère du temps pour les fonctionnalités et la qualité. Les concepts d'orchestration (déclaratif, scalabilité, résilience) sont identiques et transposables vers Kubernetes.

L'exploitation applique les bonnes pratiques de la norme ISO/IEC 27001 pour sécuriser ses informations. Cela se traduit par un contrôle strict des accès (clés SSH individuelles, droits restreints, interdiction du compte root), le chiffrement des secrets (via le raft Swarm), le renforcement et le suivi des systèmes (durcissement, logs), ainsi que la résilience du service (sauvegardes, déploiements continus).

11.2 - Conception et implémentation des pipelines CI/CD

11.2.1 - Plateforme et choix techniques

L'infrastructure de CI/CD est entièrement gérée en tant que code (*Infrastructure as Code*) grâce à des workflows versionnés directement dans le dépôt GitHub. Le pipeline s'appuie sur GitHub Actions pour l'automatisation et utilise le registre ghcr.io (sécurisé nativement par le jeton GitHub Actions) pour stocker les images Docker. Enfin, le déploiement sur le VPS est automatisé par SSH via l'exécution de la commande docker stack deploy.

11.2.2 - Pipeline et pratiques GitOps

Afin d'éviter les déploiements superflus, l'API et le frontend possèdent des workflows indépendants. La modification de l'un ne déclenche pas le redéploiement de l'autre. Pour chaque pipeline, le cycle suit une logique stricte. La phase de build n'est initiée que si l'ensemble des critères de qualité et de tests est validé.

Concrètement, lors d'un push sur la branche main, le workflow lance d'abord les tests (unitaires avec une couverture minimale de 80 %, intégration et E2E). Une fois ces validations réussies, l'image Docker est optimisée (via turbo prune), construite, puis poussée vers ghcr.io. Pour l'API (api.yml), les secrets sont gérés de manière idempotente avec Ansible. Enfin, le déploiement s'effectue par SSH sur le serveur, le fichier de configuration est copié et la commande docker stack deploy est exécutée. Au démarrage du conteneur, l'application charge les secrets, applique les migrations de base de données, puis passe en production (cf Annexe 19 : Pipeline ci/cd et workflows).

Pratique	Mécanisme	Bénéfice
Build optimisé	turbo prune --scope --docker	Image ne contenant que le code nécessaire à l'application ciblée
Migrations sûres	Exécutées au démarrage, verrou applicatif	Une seule instance migre, les autres attendent Base à jour avant le trafic
Déploiement sans coupure	Rolling update Swarm + health checks	Les nouvelles instances ne reçoivent le trafic qu'une fois saines
Anti-concurrence	Groupe de concurrence GitHub Actions sur deploy	Deux déploiements simultanés du même stack sont sérialisés

Tableau 11.2.2 - Pratiques clés du pipeline de déploiement.

11.2.3 - Métriques de qualité dans le pipeline

Le pipeline fonctionne comme un véritable garant de la qualité. Il impose des critères éliminatoires stricts, comme une couverture de code d'au moins 80 %, des tests d'intégration sur une vraie base de données et des tests de bout en bout directement dans le navigateur. Si

jamais un test échoue, le système garde en mémoire un rapport Playwright complet pour aider à comprendre rapidement ce qui s'est passé. Grâce à cela, on s'assure qu'un code déclaré prêt pour la production a été réellement testé et validé, et qu'il n'est pas seulement capable de compiler.

Pour aller encore plus loin, un outil de contrôle de qualité SonarQube est directement branché sur l'API et le frontend. Il examine chaque nouvelle ligne de code pour repérer le manque de couverture, les répétitions, les failles de sécurité ou les mauvaises pratiques. Si le code ne respecte pas les critères exigés, la demande de fusion est automatiquement bloquée avant de pouvoir atteindre la branche principale. Quand on travaille tout seul sur un projet, cette automatisation remplace avantageusement un relecteur humain. Elle apporte un regard neutre et évite de devoir juger soi-même de la qualité de son propre travail.

11.3 - Orchestration conteneurisée et infrastructure as code

11.3.1 - Conteneurisation et cluster Swarm

Toutes les briques tournent en conteneurs orchestrés par Docker Swarm. Le fichier « stack.yml » décrit de façon déclarative l'état souhaité : services, nombre de réplicas, politiques de redémarrage, configuration de mise à jour, réseau overlay et montage des secrets. Il joue le même rôle qu'un ensemble de manifests Kubernetes.

11.3.2 - Infrastructure as Code avec Ansible

L'installation du serveur et la gestion des secrets se font de manière totalement automatique grâce à Ansible. Ce choix repose principalement sur son principe d'idempotence, ce qui signifie que chaque tâche vérifie d'abord l'état actuel de la machine avant d'agir. Ainsi, exécuter à nouveau le script sur un serveur déjà configuré ne risque pas de perturber ou d'endommager le système existant. Toute cette infrastructure s'organise ainsi autour de trois scripts de configuration principaux (cf Annexe 22 : Playbooks Ansible).

Playbook	Rôle	Contenu
playbook-provision.yml	Durcissement OS + installation Docker	Utilisateur deploy non-root, SSH par clé uniquement, UFW deny-all, fail2ban
playbook-swarm.yml	Initialisation du cluster	Init Swarm, réseau overlay, labels de nœuds
playbook-secrets.yml	Création des Docker Secrets	À chaque déploiement, idempotent, no_log

Tableau 11.3.2 - Playbooks Ansible d'infrastructure as code.

11.3.3 - Gestion des secrets en défense en profondeur

La gestion des secrets représente l'élément central de la sécurisation du système. Tout le processus est pensé pour qu'aucune donnée sensible ne soit jamais enregistrée en clair sur le disque du serveur.

Pour y parvenir, le parcours d'un mot de passe suit un cheminement très strict. La valeur est d'abord conservée de façon chiffrée dans les secrets de GitHub. Lors du déploiement, Ansible récupère cette information pour l'envoyer directement en mémoire vive sans jamais l'écrire dans un fichier intermédiaire, en masquant l'action dans ses propres rapports d'activité. Le secret est ensuite stocké au cœur du cluster Swarm, au sein d'une base de données distribuée et chiffrée avec l'algorithme AES-256. Enfin, au moment de lancer l'application, ce secret est mis à disposition du conteneur sous la forme d'un fichier temporaire virtuel, présent uniquement dans la mémoire vive, qui est lu par le programme puis chargé dans l'environnement de travail.

Du côté du code, une option spécifique indique au programme où se trouve le fichier contenant le secret. L'application lit ce fichier dès son démarrage et configure l'environnement juste avant de lancer ses outils principaux. Pour simplifier le travail au quotidien, si l'application tourne sur l'ordinateur du développeur et qu'aucun fichier sécurisé n'est disponible, elle récupère simplement les valeurs depuis un fichier de configuration classique. Cela permet d'utiliser exactement le même code en phase de développement et en production.

Sécurité des secrets - Les secrets sont stockés dans le raft Swarm, base de données distribuée et chiffrée native de Docker : pas de service tiers à sécuriser, pas de base externe. Docker gère seul la cohérence et le chiffrement, et la valeur en clair n'existe qu'en mémoire vive du conteneur.

11.3.4 - Clé SSH de déploiement et rotation des secrets

Pour que le déploiement se fasse de manière automatique, le système a besoin d'accéder au serveur et utilise donc une clé de connexion. Au lieu de me servir de ma propre clé, ce qui poserait un risque majeur en cas de vol, une clé exclusive a été créée pour cette tâche. Conçue avec un algorithme moderne et très robuste, cette clé fonctionne uniquement sur les serveurs de production et peut être désactivée à tout moment sans perturber le reste des accès. Sa partie privée est conservée à l'abri et sous forme chiffrée dans les outils de sécurité de GitHub, tandis que la partie publique est recréée automatiquement lors du déploiement.

Comme les secrets gérés par Docker ne peuvent pas être modifiés une fois créés, le renouvellement d'un mot de passe ou d'une clé sensible suit une méthode bien précise qui s'appuie sur un système de versions. Cela permet d'effectuer le changement sans jamais couper le service pour les utilisateurs.

Le processus se déroule en quelques étapes simples. On commence par enregistrer la nouvelle valeur dans les paramètres de GitHub, puis on crée une deuxième version du secret sur le serveur. Il suffit ensuite de modifier le fichier de configuration du déploiement pour qu'il utilise cette nouvelle version. Lors de la mise à jour, les serveurs se relancent progressivement les uns après les autres de manière totalement transparente. Enfin, dès que le système est stabilisé, l'ancienne version du secret est définitivement supprimée.

Toute cette organisation, qui repose sur l'utilisation d'une clé dédiée, de secrets versionnés et d'une procédure de renouvellement, montre bien comment les règles de sécurité et de protection sont appliquées au fonctionnement des serveurs, et pas seulement à la structure du code.

11.3.5 - Évolutivité horizontale

L'architecture a été pensée dès le départ pour pouvoir grandir facilement. S'il est nécessaire d'ajouter un nouveau serveur au cluster, la procédure est très simple. Il suffit de provisionner un nouveau VPS, de l'inscrire dans le répertoire de configuration d'Ansible, puis de relancer les scripts d'installation automatique et de gestion du cluster. Comme les clés de sécurité et le réseau interne partagé sont déjà en place au sein du cluster, cette nouvelle machine y a accès de manière instantanée, sans qu'il soit nécessaire de modifier ou de reparamétrer le code de l'application.

11.4 - Stratégie de monitoring et d'observabilité

L'observabilité est abordée ici sous l'angle DevOps, avec pour objectifs principaux d'anticiper les pannes et de piloter les performances du système. L'utilisation conjointe de Prometheus, Loki et Grafana, associée à cAdvisor, permet de rassembler les métriques, les journaux d'activité et la corrélation des données au sein d'une seule et même interface.

Pilier	Outils	Usage
Métriques applicatives	Prometheus (pull /metrics)	Latence, débit, taux d'erreur, 429
Métriques infrastructure	cAdvisor + Prometheus	CPU, mémoire
Logs structurés	Pino → Promtail → Loki	Logs JSON corrélés par identifiant de requête
Visualisation	Grafana	Tableaux de bord, corrélation métrique/log

Tableau 11.4 - Dispositif d'observabilité DevOps.

Concrètement, juste après un déploiement, les tableaux de bord permettent de vérifier immédiatement que les réplicas fonctionnent correctement et que le taux d'erreur n'augmente pas. Si un problème survient, le système permet de lier directement un pic de latence aux journaux d'erreur générés au même moment grâce à un identifiant de requête unique, ce qui

accélère grandement le diagnostic. C'est justement cette capacité à croiser les informations qui fait la force d'une suite d'outils unifiée par rapport à des solutions séparées qui ne communiquent pas entre elles.

11.5 - Sobriété énergétique et usage de l'intelligence artificielle

11.5.1 - Sobriété énergétique : de la démarche à la mesure

L'éco-conception a servi de fil conducteur pour l'ensemble des choix techniques du projet. C'est ce qui a motivé l'utilisation de Docker Swarm à la place de Kubernetes, de Loki plutôt que de la suite ELK, ou encore l'adoption des React Server Components, des images au format WebP, du cache et d'un CDN. Pour le moment, cette démarche reste purement qualitative. Cela signifie que l'on sélectionne des outils reconnus pour leur sobriété, mais que l'on ne mesure pas encore précisément leur empreinte carbone réelle.

Élément	État	Détail / évolution
Choix de briques sobres	En place	Swarm, Loki, RSC, WebP, cache Redis, diffusion CDN
Métriques de ressources	En place	CPU/mémoire par conteneur (cAdvisor + Prometheus)
Tableaux de bord	En place	Deux dashboards Grafana (métriques et logs d'API)
Mesure carbone dédiée	À implémenter	Instrumenter le worker (ex. CodeCarbon) et exposer la métrique
Alerte de consommation	À implémenter	Règle d'alerte sur CPU/énergie soutenus (la stack le permet)
KPIs de sobriété	À implémenter	Poids moyen des pages, octets/session, taux de hit CDN

Tableau 11.5.1 - Sobriété énergétique : état des lieux et axes de mesure.

La prochaine étape est de quantifier précisément la consommation d'énergie, en ciblant tout particulièrement le script d'optimisation des images qui représente la tâche la plus gourmande en ressources processeur. Il s'agira également de mettre en place une alerte en cas de surconsommation prolongée et de surveiller des indicateurs de sobriété sur la partie visible du site. Toutes ces améliorations pourront se faire directement en utilisant les outils de suivi déjà installés, sans avoir besoin de tout réorganiser.

11.5.2 - Usage de l'intelligence artificielle

Le référentiel met en avant l'usage de l'intelligence artificielle pour optimiser le travail et aider au développement. Dans ce projet, l'IA a joué un rôle bien précis qu'il convient de détailler en toute transparence.

Elle a servi uniquement d'assistante pour le développement. Des outils comme Claude et ChatGPT ont ainsi été utilisés au quotidien pour accélérer l'écriture du code, aider à la création des tests, rédiger la documentation et relire le travail effectué. À ce titre, l'IA intervient comme un simple outil de productivité et ne prend jamais la moindre décision. Chaque suggestion ou ligne de code proposée a été systématiquement vérifiée, corrigée et validée manuellement avant d'être intégrée au projet.

11.6 - Retour d'expérience et plan d'amélioration continue

11.6.1 - Difficultés réelles rencontrées en production

Le déploiement initial sur un serveur vierge a mis en évidence des ajustements techniques nécessaires pour faire cohabiter Docker Swarm et Ansible. Plutôt que de masquer ces imprévus, cette section documente les solutions réelles apportées lors de la mise en production.

Problème	Symptôme / cause	Résolution
Priorité des variables Ansible	ansible_user de l'inventaire écrasait l'option --user	Déplacer le réglage dans ansible.cfg (priorité la plus basse, surchargeable)
Clé publique absente du runner	authorized_key avait besoin du .pub, seule la clé privée existe	Régénérer la clé publique depuis la privée (ssh-keygen -y) dans le workflow
Nom de nœud Swarm ≠ alias Ansible	Label appliqué à un nœud introuvable	Utiliser ansible_hostname (hostname réel) et non l'alias d'inventaire
pgbouncer et convention _FILE	Image tierce n'implémentant pas _FILE pour le mot de passe	Générer dynamiquement un userlist.txt natif pgbouncer via un Docker Secret
Déploiements concurrents	api.yml et web.yml déployant le même stack simultanément	Groupe de concurrence GitHub Actions (sérialisation, sans annulation)

Tableau 11.6.1 - Retour d'expérience : incidents de déploiement et résolutions.

11.6.2 - Limites connues et axes d'évolution

Limite	Nature	Évolution envisagée
Manager Swarm unique	Point unique de défaillance pour le raft	Passer à 3 managers (quorum)
PostgreSQL nœud unique	Pas de réplication	Réplication primaire/secondaire
Migrations multi-réplicas	Sûr mais sous-optimal	Job Swarm one-shot dédié
Rotation de secrets manuelle	Secrets immuables	HashiCorp Vault (rotation automatique)
Pas d'environnement de staging	Déploiement direct en production	Ajouter un étage de pré-production

Tableau 11.6.2 - Limites assumées et plan d'amélioration continue.

Ces limites découlent de choix cohérents à l'échelle d'un travail en solo et ne sont en aucun cas des oublis. Pour chacune d'entre elles, une piste d'évolution a été clairement identifiée, et toutes pourront être mises en place sans avoir à reconstruire l'architecture du projet. C'est tout le principe d'une démarche d'amélioration continue, qui consiste à livrer d'abord un système parfaitement fonctionnel et surveillé, pour ensuite le faire grandir par étapes successives et maîtrisées.

Synthèse DevOps - La chaîne DevOps réalisée n'est pas un assemblage d'outils mais une propriété du système : depuis un dépôt cloné, il est possible de reconstruire l'infrastructure, déployer en une commande, et observer le résultat, le tout sans jamais manipuler un secret en clair.

12 - Analyses et perspectives

12.1 - Analyse des résultats et des performances

12.1.1 - Évaluation des objectifs atteints

Confronté aux quatre objectifs fixés dans la note de cadrage, le projet présente un bilan globalement conforme, avec des réussites nettes et des limites assumées.

Objectif	Niveau	Constat
Souveraineté des données	Atteint	Données identifiantes exclusivement sur VPS UE, RGPD (droit à l'oubli par cascade), binaires sur S3 région Paris seulement pour le MVP
Expérience moderne & accessible	Atteint	UI shadcn/ui accessible (Radix), galerie fluide, accessibilité AA
Valeur différenciante	Atteint	Exploration chromatique fonctionnelle (palette OKLab + atlas chromatique fixe), absente du marché
Industrialisation & fiabilité	Atteint	CI/CD complet, IaC Ansible, observabilité, tests bloquants à 5 niveaux

Tableau 12.1.1 - Évaluation des objectifs du cahier des charges.

12.1.2 - Analyse des performances

Les choix d'architecture ont des effets mesurables sur la performance perçue et sur la robustesse : optimisation des images en WebP et diffusion CDN (galerie fluide), pagination et

défilement infini (pas de chargement massif), cache Redis partagé (réduction de la charge PostgreSQL), traitement d'image asynchrone (réponse immédiate à l'upload). Sur le plan de la fiabilité, la suite de tests pluri-niveaux, le seuil de couverture bloquant (≥ 80 %) et le quality gate SonarQube en CI préviennent les régressions, tandis que le rolling update et les health checks assurent une continuité de service lors des déploiements.

12.2 - Retour d'expérience et leçons apprises

12.2.1 - Réussites et difficultés

Le premier point positif concerne la rigueur, qui a clairement porté ses fruits. Le fait de s'imposer une discipline stricte sur les tests et la documentation, notamment avec les comptes-rendus de décisions architecturales et les notes de développement, a permis de transformer ce projet solo ambitieux en un système facile à maintenir et à transmettre. À cela s'ajoute une vraie réussite autour de la sobriété. En choisissant systématiquement l'outil le plus simple et le plus adapté aux besoins réels, comme Docker Swarm, Loki ou une gestion des accès simplifiée, les délais ont pu être respectés sans jamais faire l'impasse sur la qualité.

Du côté des difficultés, l'écart entre la théorie et la réalité a été un vrai sujet. Le passage à la mise en production a mis en lumière de petites subtilités impossibles à deviner en lisant simplement la documentation, qu'il s'agisse de configurations incomplètes, de problèmes de noms de machines ou de caches corrompus. Heureusement, ces imprévus ont pu être réglés grâce à des tests concrets. Enfin, la principale complexité est restée l'ampleur générale du périmètre à gérer. Réussir à couvrir seul quatre grands blocs de compétences a imposé de faire des contrôles permanents. Dans ce contexte, l'utilisation de la méthode MoSCoW s'est révélée indispensable pour fixer les bonnes priorités et éviter de se disperser.

12.2.2 - Capitalisation des connaissances et couverture des compétences

Chaque sujet délicat a été capitalisé par écrit (ADR, notes de développement), garantissant la transmissibilité. Le tableau ci-dessous met en regard les blocs de compétences du référentiel.

Bloc de compétences	Éléments démontrés
BC1 - Définir une stratégie de SI	Veille, benchmark, cartographie, analyse de risques, stratégie
BC2 - Piloter des projets informatiques	Cadrage, WBS, phases, KPI, risques, RACI, communication
BC3 - Concevoir et développer une application	Architecture, modèle de données, patterns, sécurité, tests, livraison
BC4C - Infrastructures DevOps automatisées	CI/CD, conteneurisation, IaC, secrets, observabilité, amélioration continue

Tableau 12.2.2 - Couverture des blocs de compétences par le dossier.

12.3 - Perspectives et recommandations

Les évolutions identifiées se répartissent en trois horizons.

Horizon	Évolutions recommandées
Moyen terme	Tri des photos d'une cellule par poids de couleur Expiration des liens de partage Trois managers Swarm (quorum) Environnement de pré-production
Long terme	Stockage objet souverain (S3-compatible UE) pour lever la dépendance Cloud Act Application mobile Recherche avancée (date, tags, favoris)

Tableau 12.3 - Feuille de route d'évolution.

13 - Conclusion et remerciements

Ce projet de fin d'études avait pour ambition de démontrer, à travers un produit entièrement finalisé et mis en production, la maîtrise des compétences attendues d'un expert en informatique et systèmes d'information. L'application Kroma est née d'un constat simple sur le marché actuel, à savoir le manque d'une plateforme d'hébergement de photos qui soit à la fois respectueuse de la vie privée et agréable à utiliser au quotidien. Pour y répondre, le projet couvre l'ensemble de la chaîne technique, depuis la stratégie globale et la veille technologique jusqu'au développement sécurisé, en passant par l'industrialisation DevOps, le suivi des performances et une stratégie de tests totalement éliminatoire.

Les trois hypothèses posées au départ se confirment à l'arrivée. Tout d'abord, une petite structure est tout à fait capable de proposer une expérience utilisateur moderne sans pour autant sacrifier la simplicité d'utilisation et la conformité au RGPD. Ensuite, l'organisation des photos par code couleur apporte une vraie valeur ajoutée et une manière inédite de naviguer dans ses fichiers. Enfin, un développeur travaillant seul peut parfaitement industrialiser un déploiement fiable et facile à surveiller. Au-delà de la réussite technique du produit, ce travail valide une véritable posture d'ingénieur, qui consiste à justifier la moindre décision, à assumer et noter les compromis nécessaires, et à transformer les difficultés concrètes du terrain en compétences acquises.

L'impact de ce travail va bien au-delà du simple cadre académique. Le projet apporte une réponse concrète aux enjeux actuels de souveraineté numérique grâce à une démarche sobre et respectueuse de l'environnement. Il laisse derrière lui une base technique saine, entièrement testée, documentée et prête à être déployée, sur laquelle il sera facile de construire les prochaines améliorations. Sur le plan personnel, cette expérience confirme un profil polyvalent,

à la croisée du développement complet et du DevOps, capable de piloter un système d'information de l'idée de départ jusqu'à sa gestion quotidienne en production.

14 - Bibliographie et webographie

Les ressources ci-dessous ont fondé la veille et les choix techniques. Les documentations officielles ont systématiquement été privilégiées comme sources faisant autorité.

14.1 - Documentations techniques officielles

NestJS - Documentation officielle. https://docs.nestjs.com

Next.js - Documentation officielle (App Router, RSC). https://nextjs.org/docs

TypeORM - Documentation officielle. https://typeorm.io

PostgreSQL - Documentation officielle. https://www.postgresql.org/docs/

Docker - Swarm mode & Secrets. https://docs.docker.com/engine/swarm/

Ansible - Documentation officielle. https://docs.ansible.com

Redis & BullMQ. https://redis.io/docs/ - https://docs.bullmq.io

Sharp - Traitement d'images haute performance. https://sharp.pixelplumbing.com

Oklab (Björn Ottosson) : Spécifications et formules de l'espace colorimétrique. https://bottosson.github.io/posts/oklab/

Prometheus / Loki / Grafana. https://grafana.com/docs/

Tailwind CSS v4 & shadcn/ui. https://tailwindcss.com - https://ui.shadcn.com

TanStack Query v5. https://tanstack.com/query

Zod - Validation TypeScript-first. https://zod.dev

14.2 - Sécurité et réglementation

OWASP - Cheat Sheet Series (Authentication, Password Storage, CSRF). https://cheatsheetseries.owasp.org

ANSSI - Recommandations de sécurité. https://cyber.gouv.fr

CNIL - Guides RGPD pour les développeurs. https://www.cnil.fr

Règlement (UE) 2016/679 (RGPD). https://eur-lex.europa.eu

Règlement (UE) 2024/1689 (IA Act). https://eur-lex.europa.eu

RGAA 4.1 / WCAG 2.1 - Accessibilité. https://accessibilite.numerique.gouv.fr

- https://www.w3.org/TR/WCAG21/

14.3 - Documentation produite dans le cadre du projet

Le dépôt contient une documentation interne, citée tout au long du dossier :

Pilotage de projet : étude d'opportunité, note de cadrage, budget estimatif, WBS & planning (Gantt), **RACI & gouvernance :** docs/etude-opportunite.md, note-de-cadrage.md, budget-estimatif.md, wbs-planning.md, raci-gouvernance.md

15 ADR (Architecture Decision Records) : docs/adr/

Dossier de veille technologique et note Green IT - docs/veille-technologique.md, green-it.md

Cartographie du SI en 4 niveaux : métier, fonctionnel, applicatif, infrastructure, risques EBIOS, accessibilité - docs/cartographie-si.md

Modélisation des données, diagramme de classes et diagrammes de flux/séquence : docs/modelisation-donnees.md, diagramme-de-flux.md

Cas d'utilisation UML : docs/use_cases/*.puml

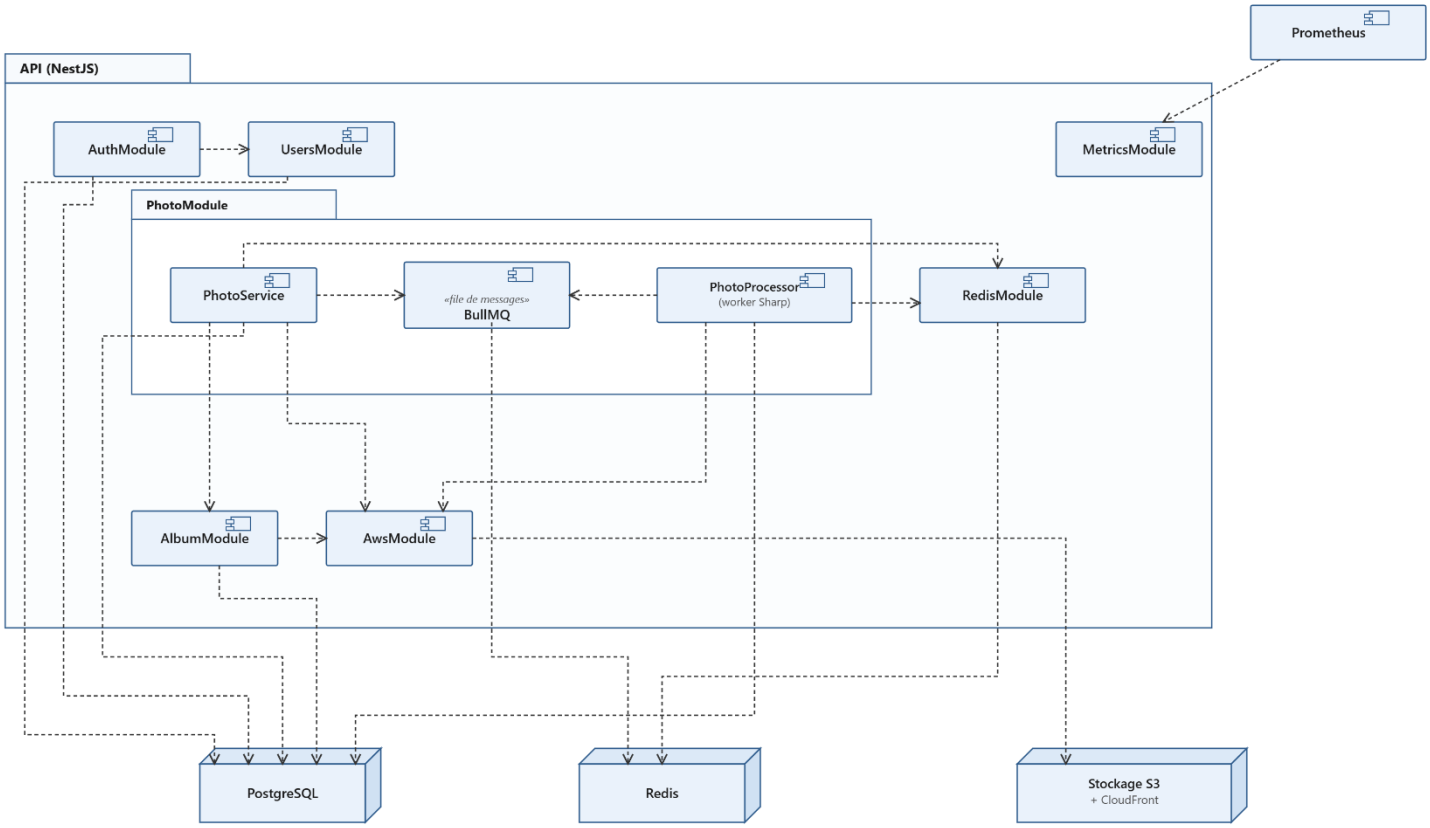
Benchmark concurrentiel, personas, user stories : docs/

15 - Annexes

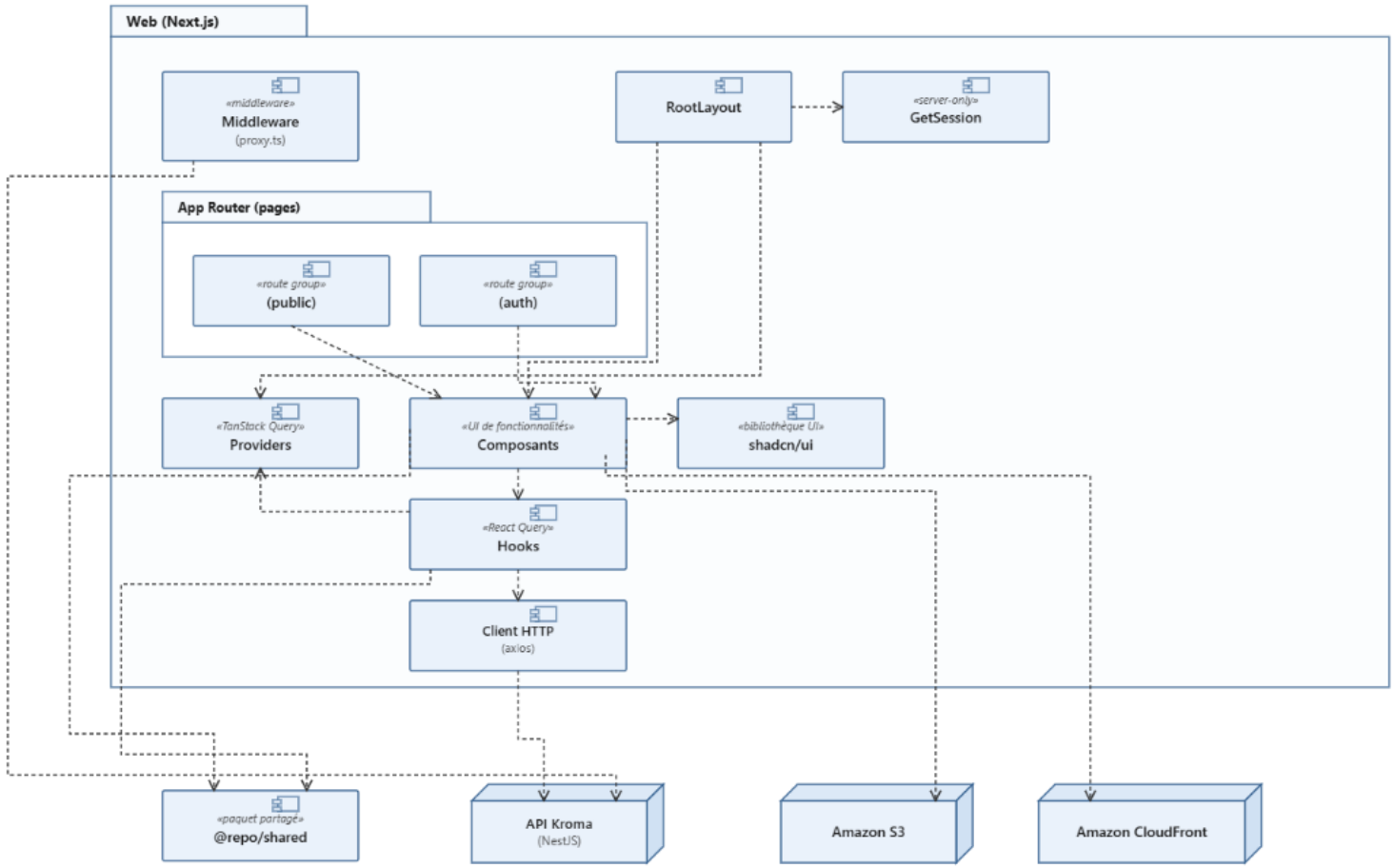
Les annexes regroupent les documents trop détaillés pour le corps du dossier mais utiles à l'argumentation et à la vérification. Elles sont disponibles dans le dépôt du projet.

Annexe 1 - Diagrammes de composants

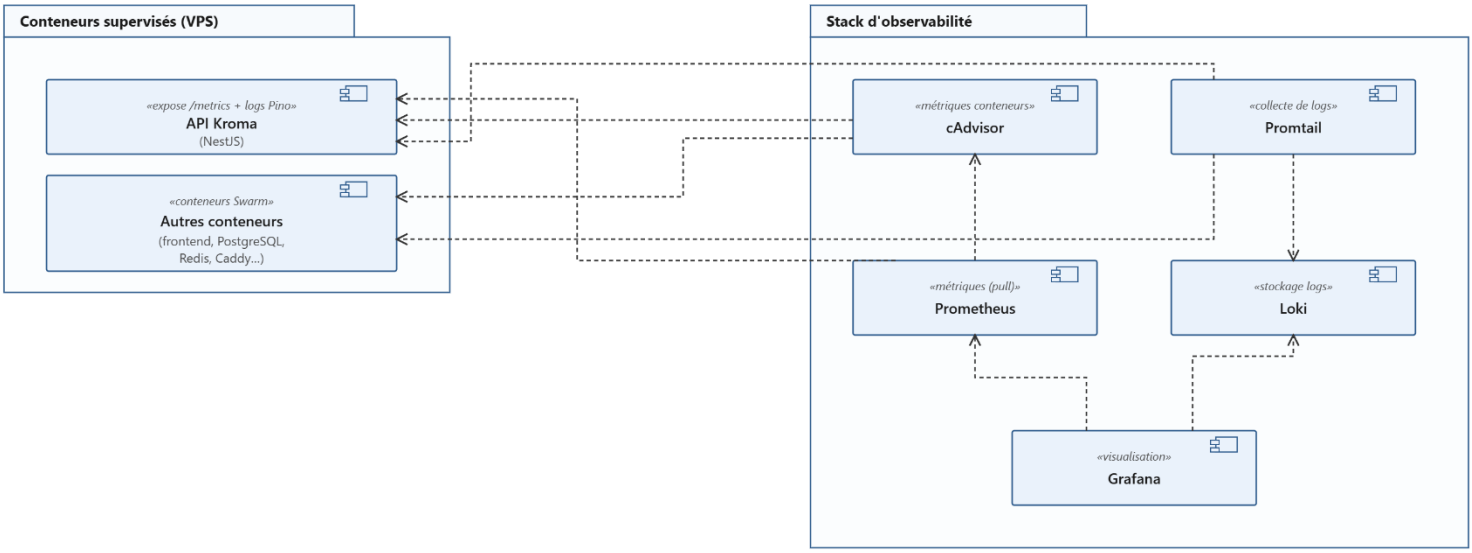
1.A - API



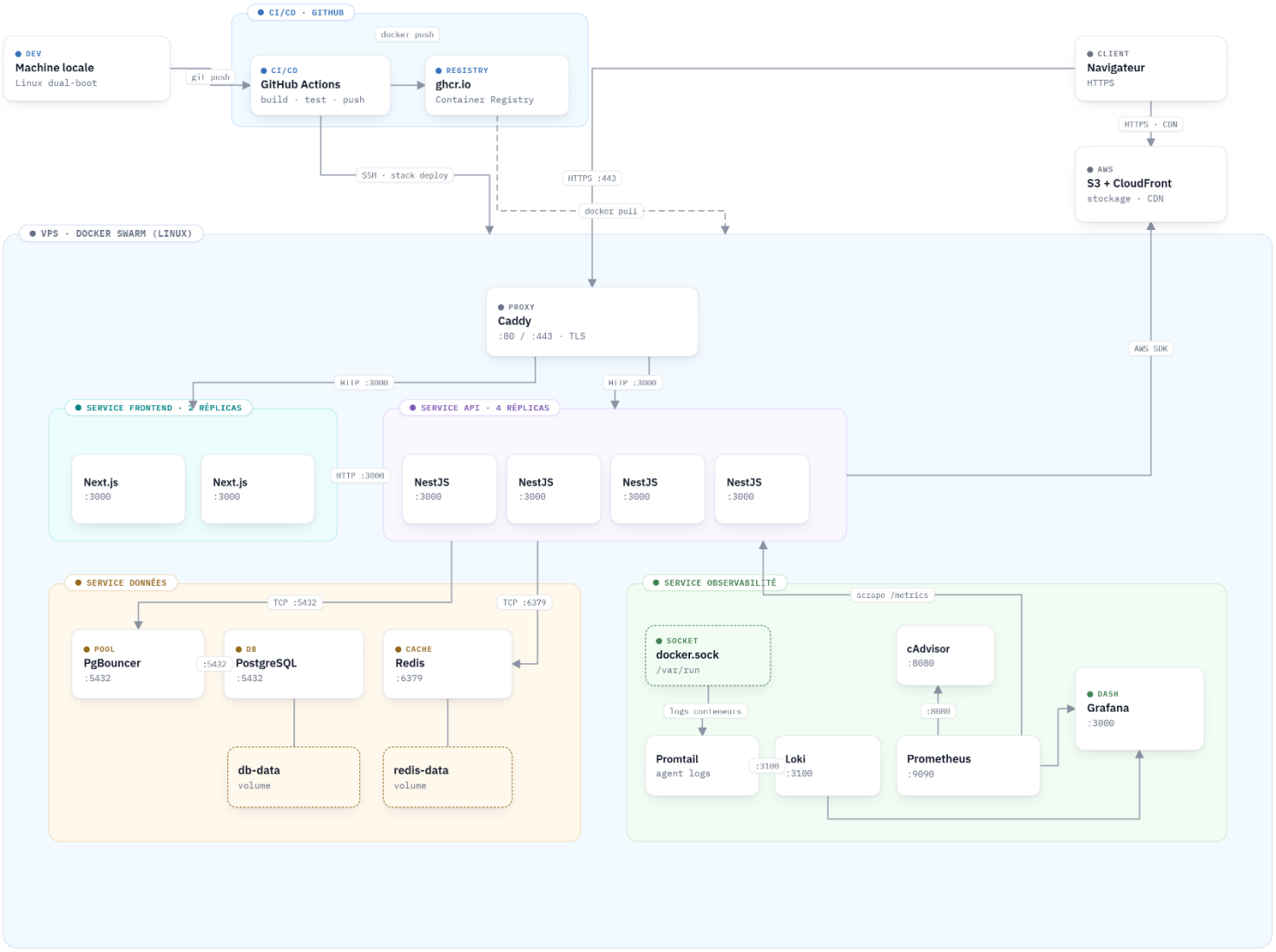
1.B - Frontend



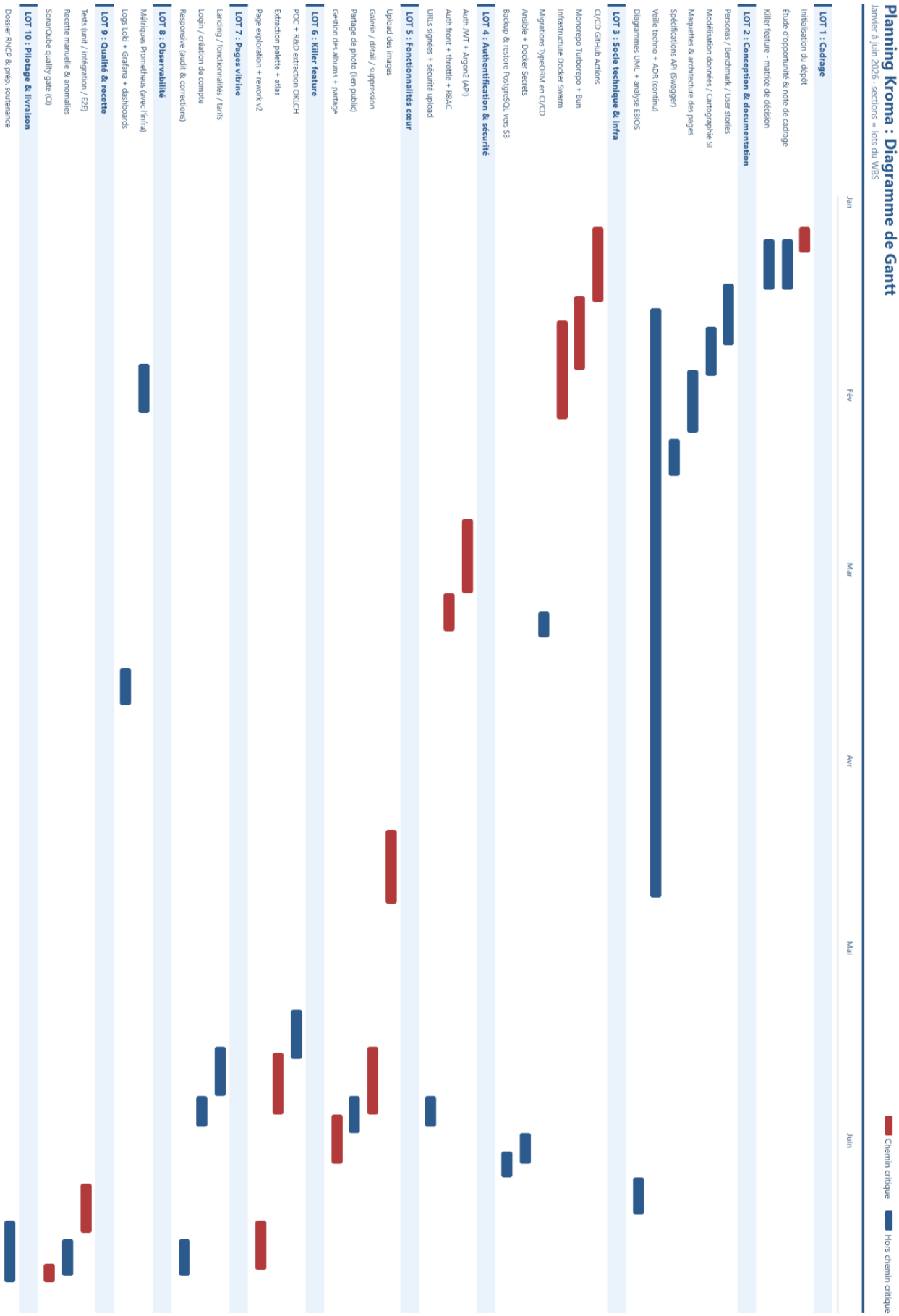
1.C - Observabilité



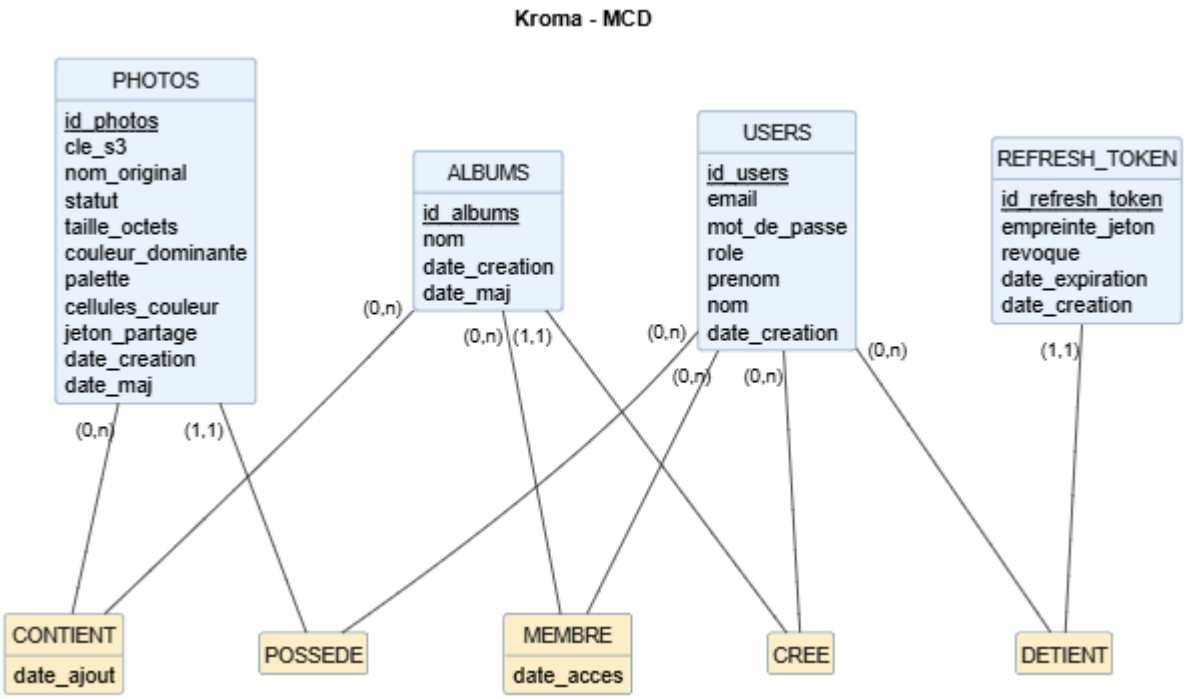
Annexe 2 - Diagramme de déploiement



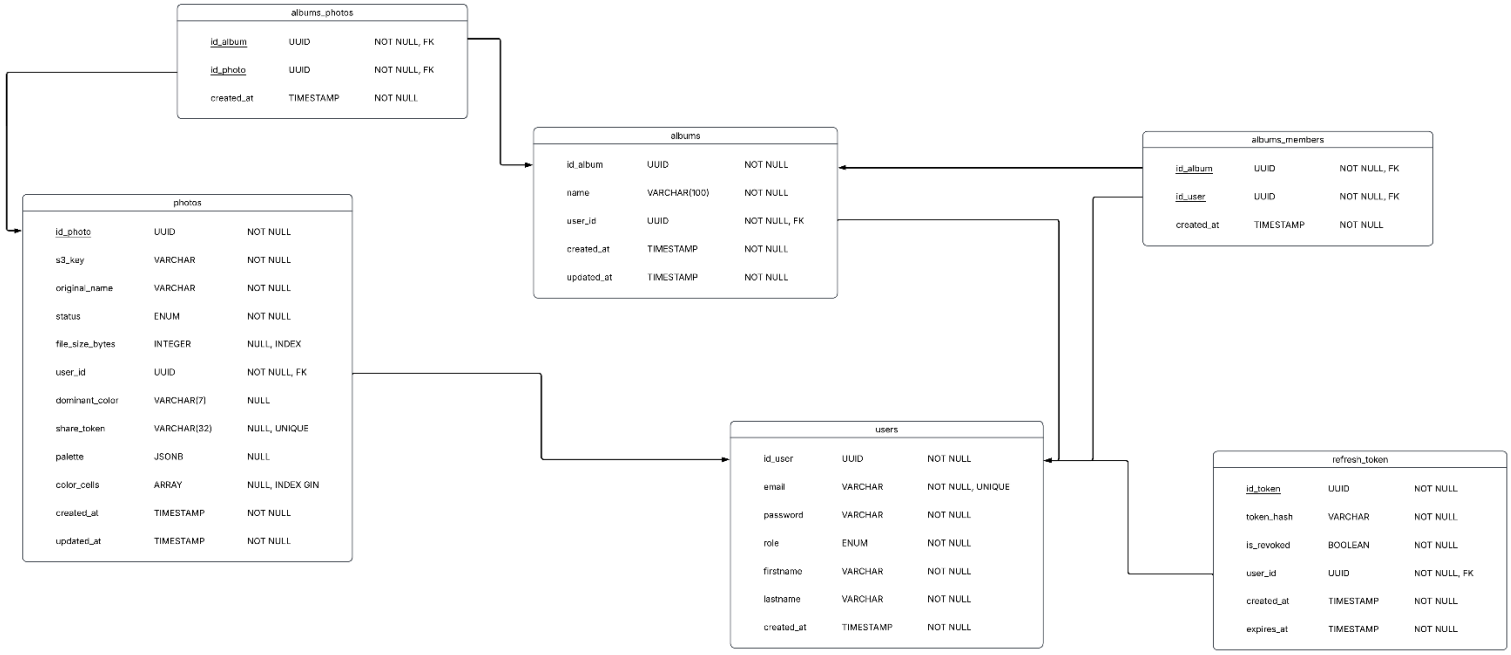
Annexe 3 - Diagramme de Gantt



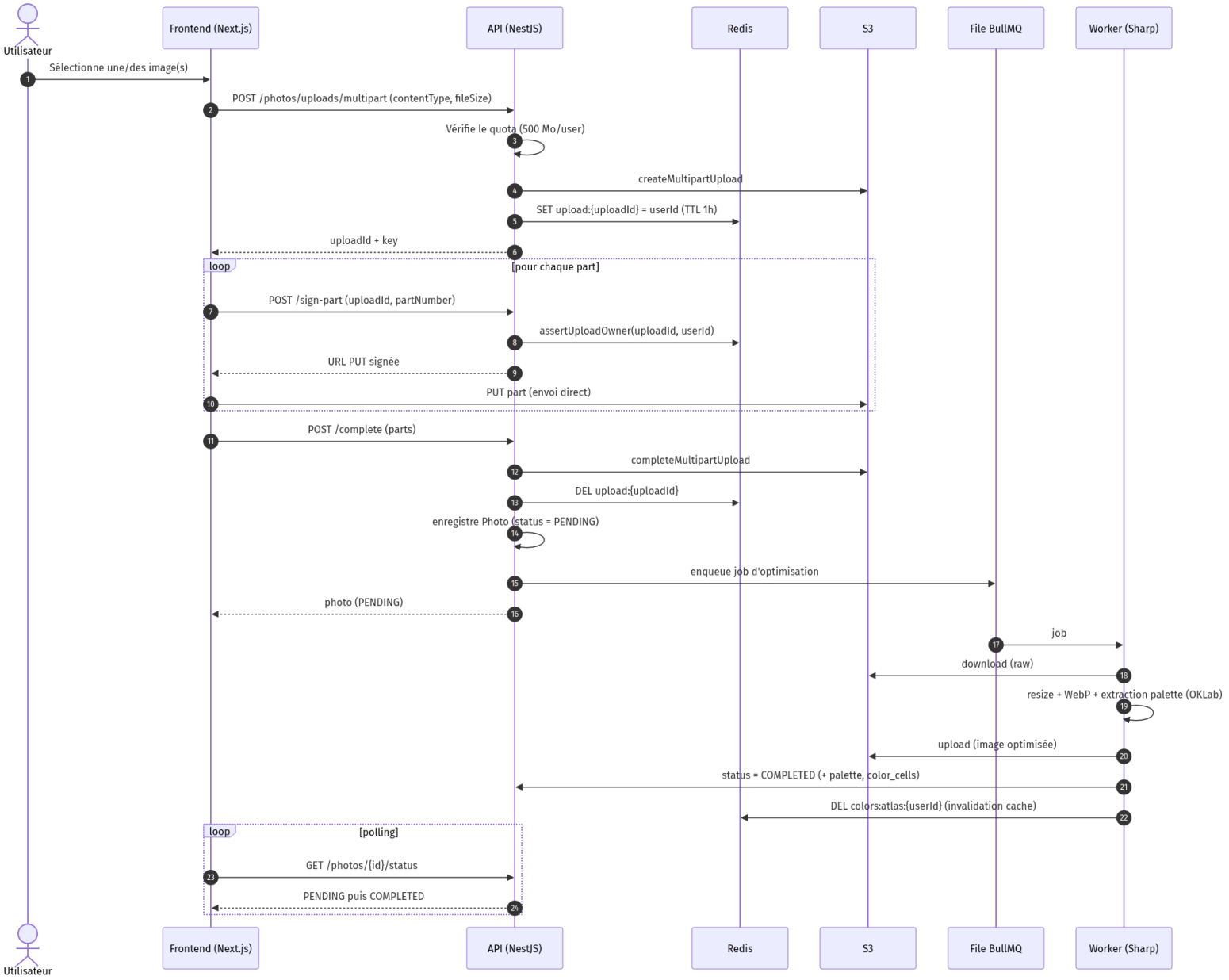
Annexe 4 - Modèle conceptuel de données



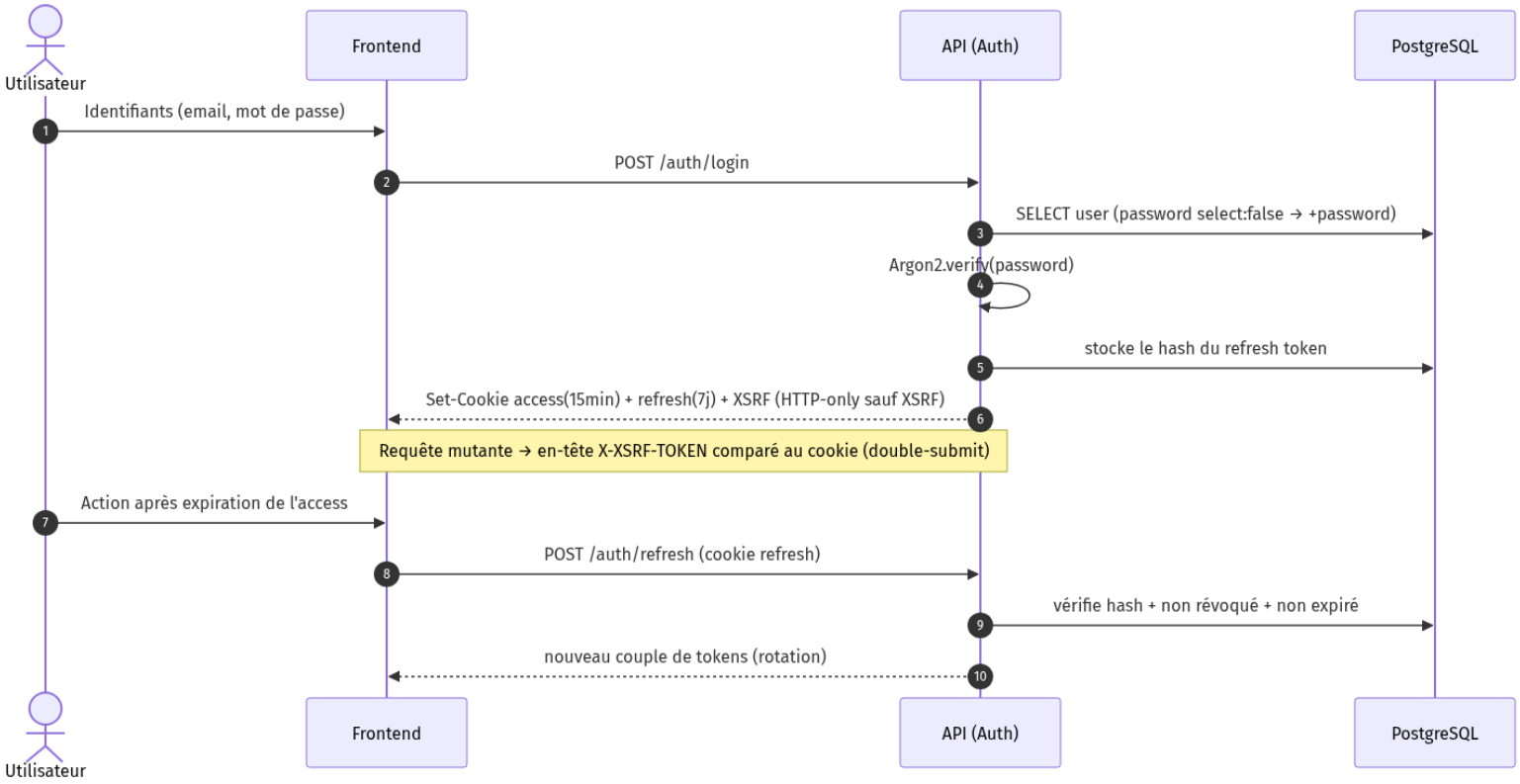
Annexe 5 - Modèle physique de données



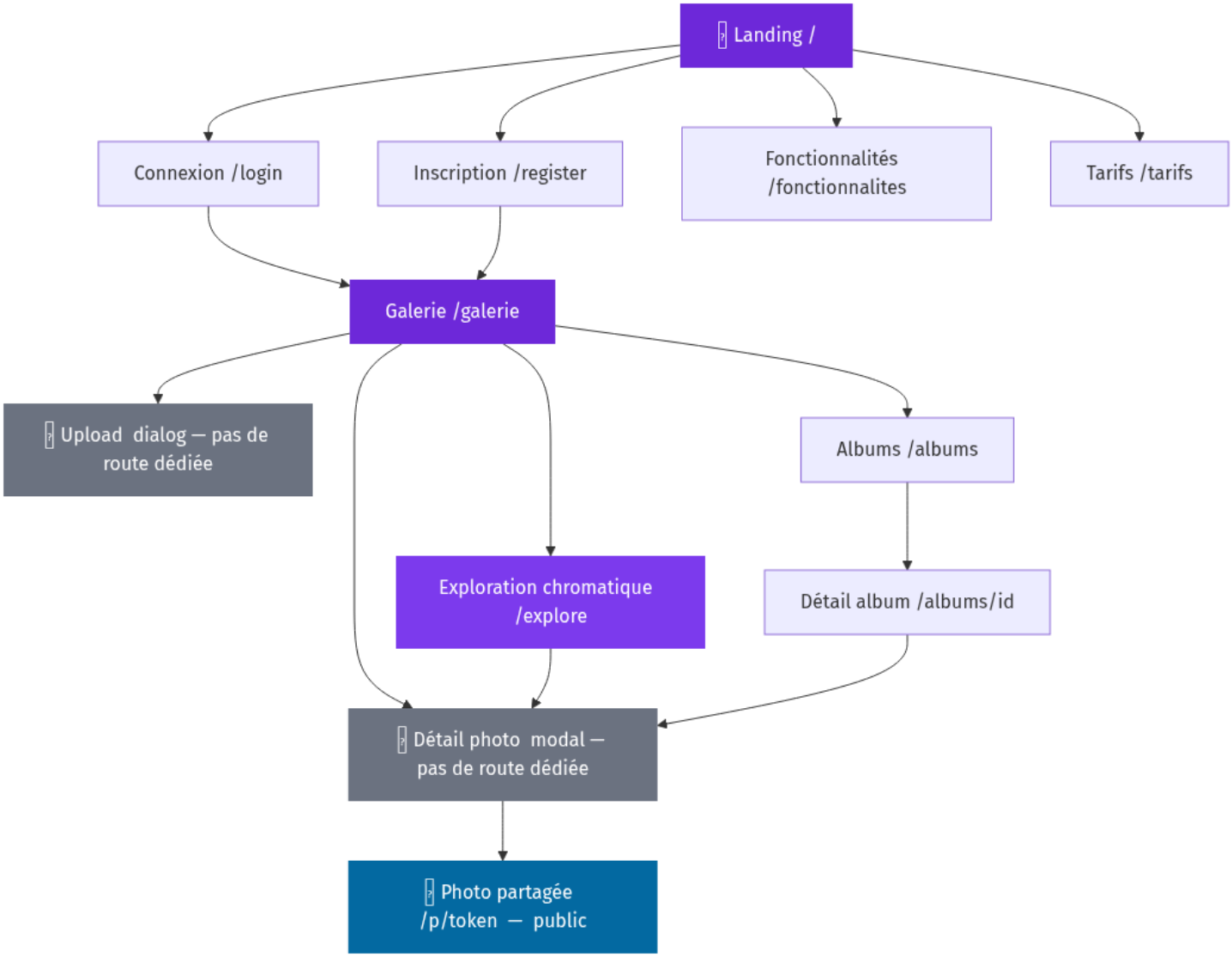
Annexe 6 - Diagramme de séquence de l’upload d’une image



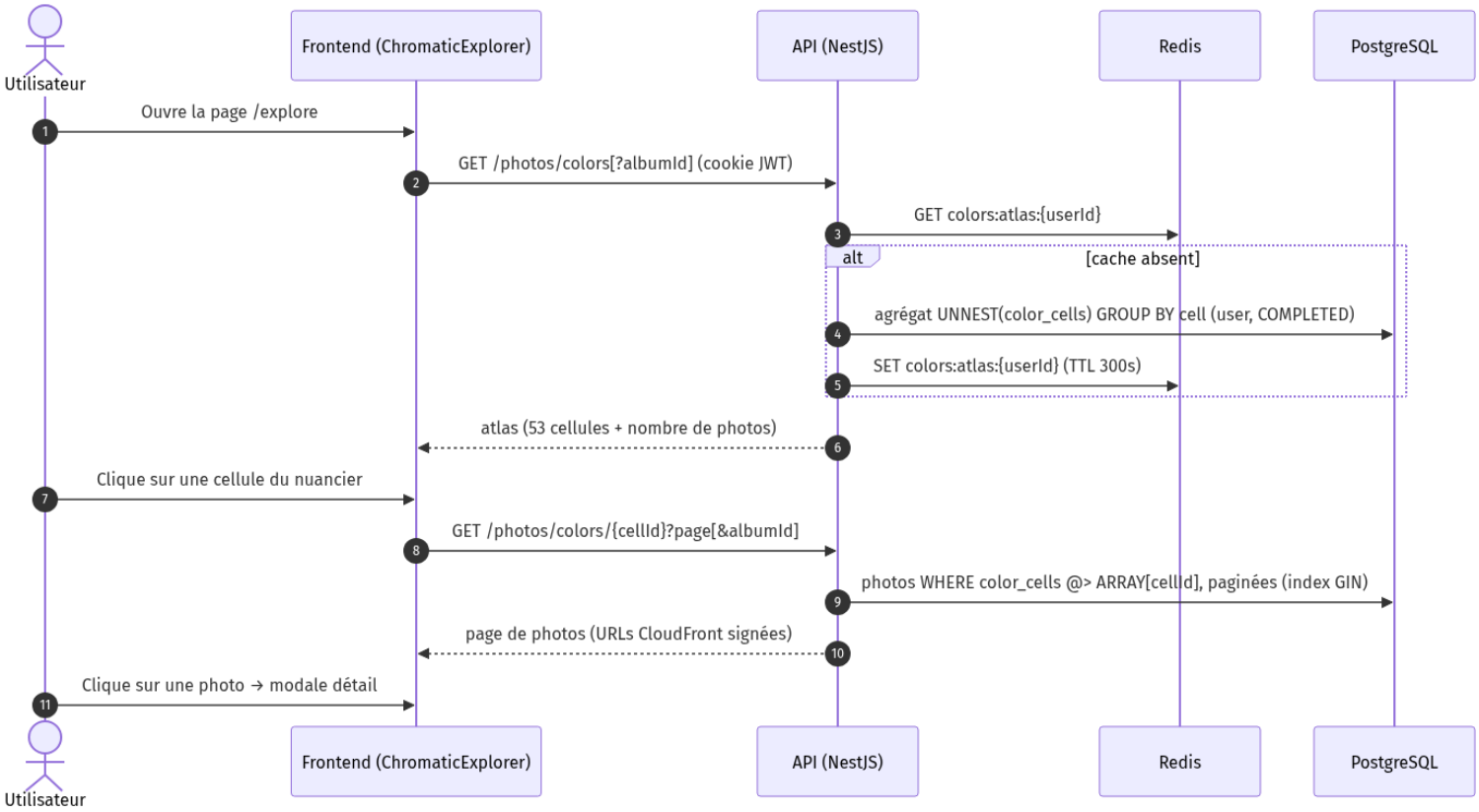
Annexe 7 - Diagramme de séquence de l'authentification



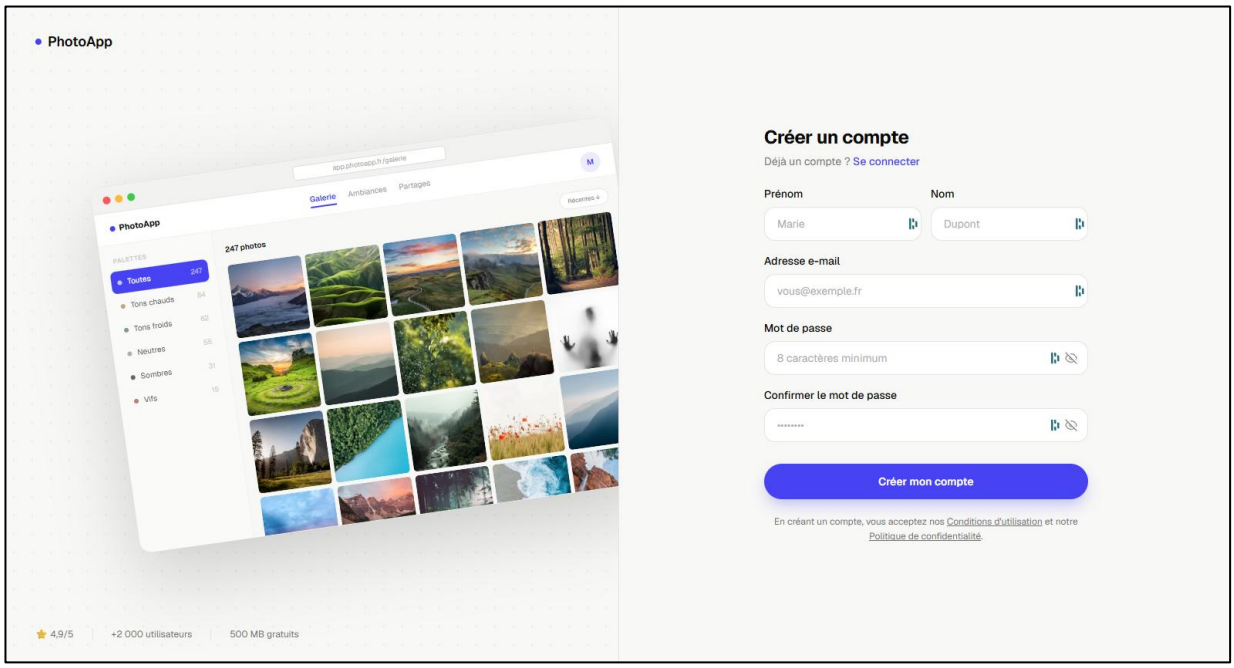
Annexe 8 - Arborescence des pages



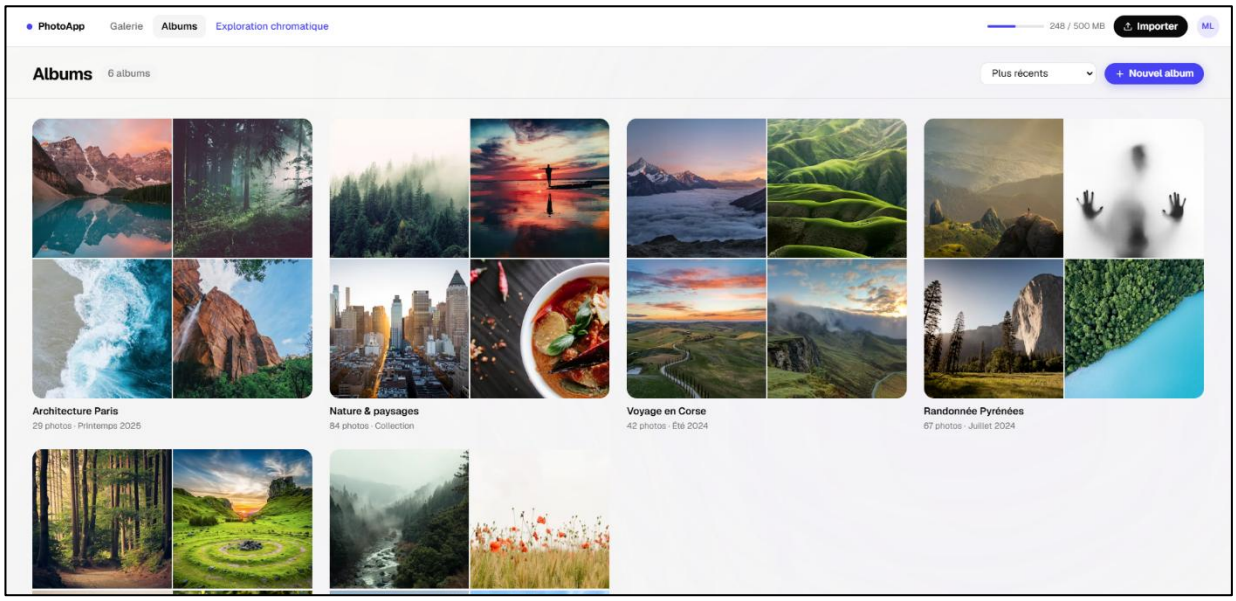
Annexe 9 - Diagramme de séquence de l'exploration chromatique



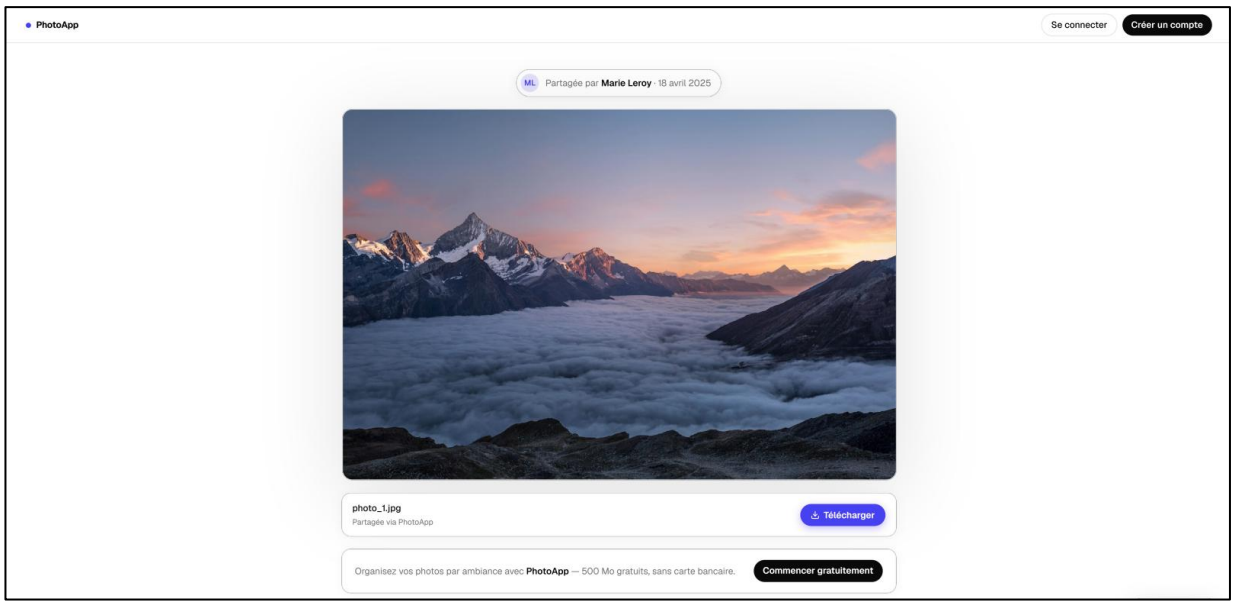
Annexe 10 - Maquettes

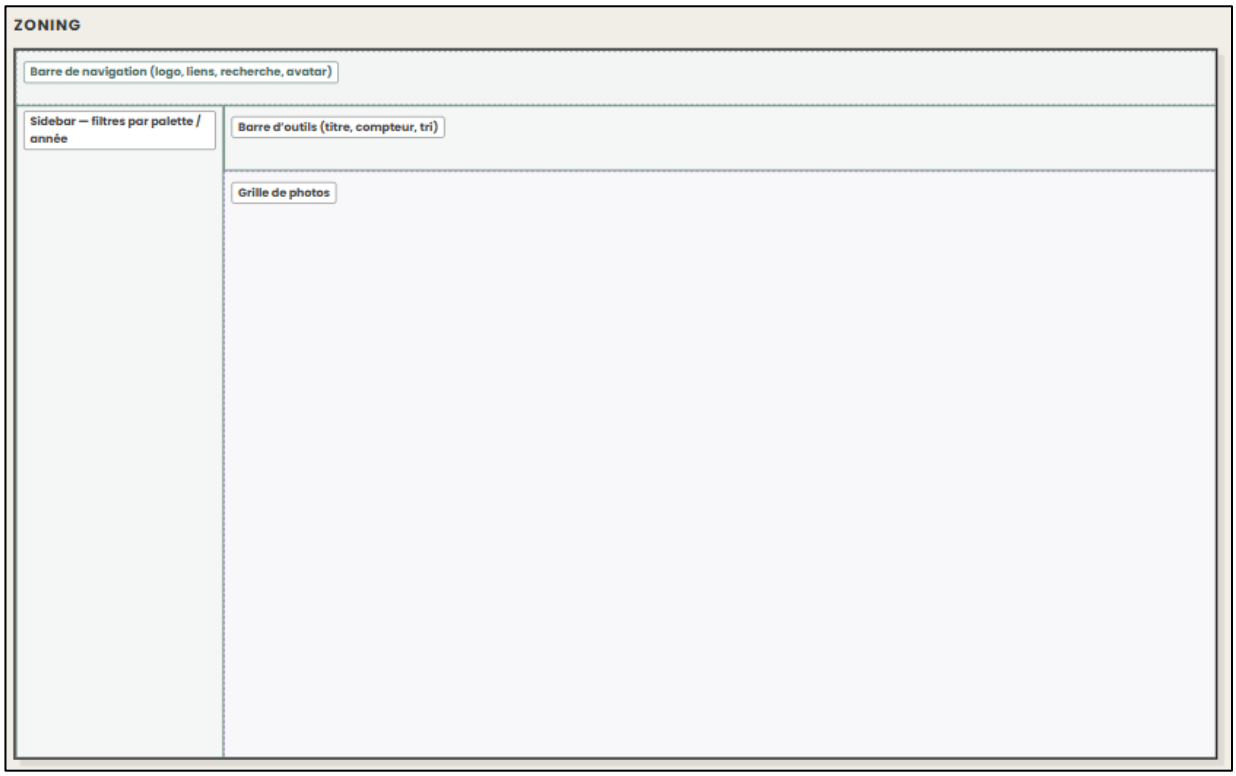


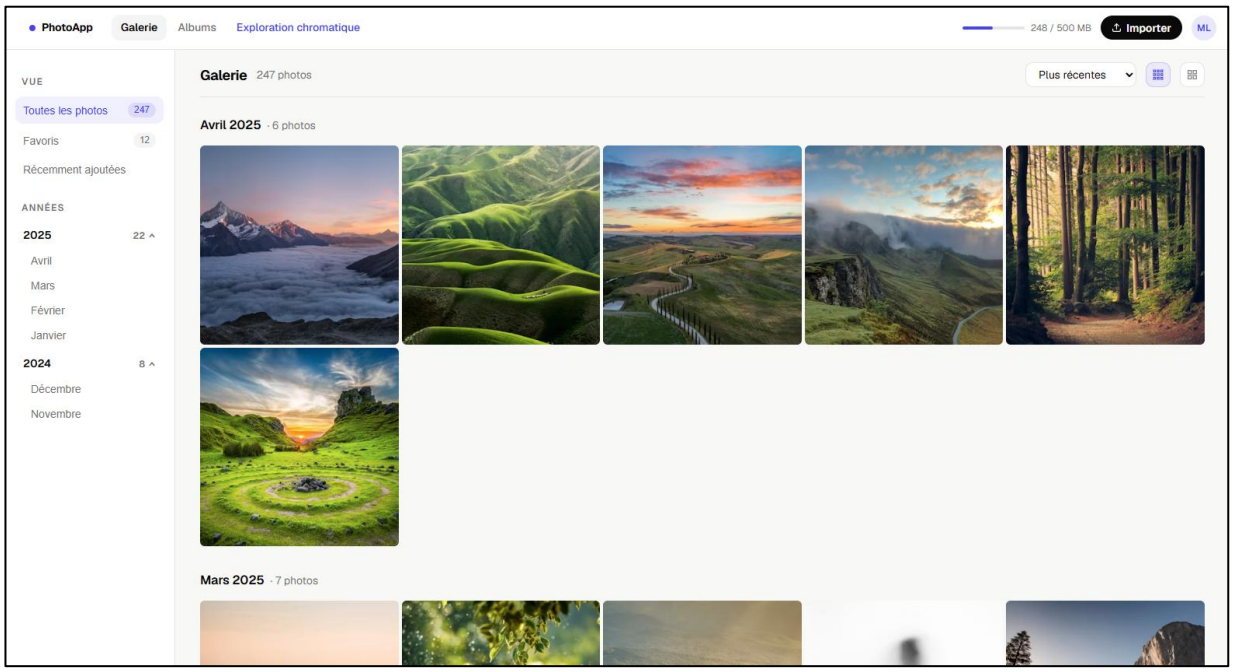



PhotoApp
Galerie
Albums
Exploration chromatique
248 / 500 MB
Importer
ML
Albums
6 albums
Plus récents
+ Nouvel album
Architecture Paris
29 photos · Printemps 2025
Nature & paysages
84 photos · Collection
Voyage en Corse
42 photos · Été 2024
Randonnée Pyrénées
67 photos · Juillet 2024

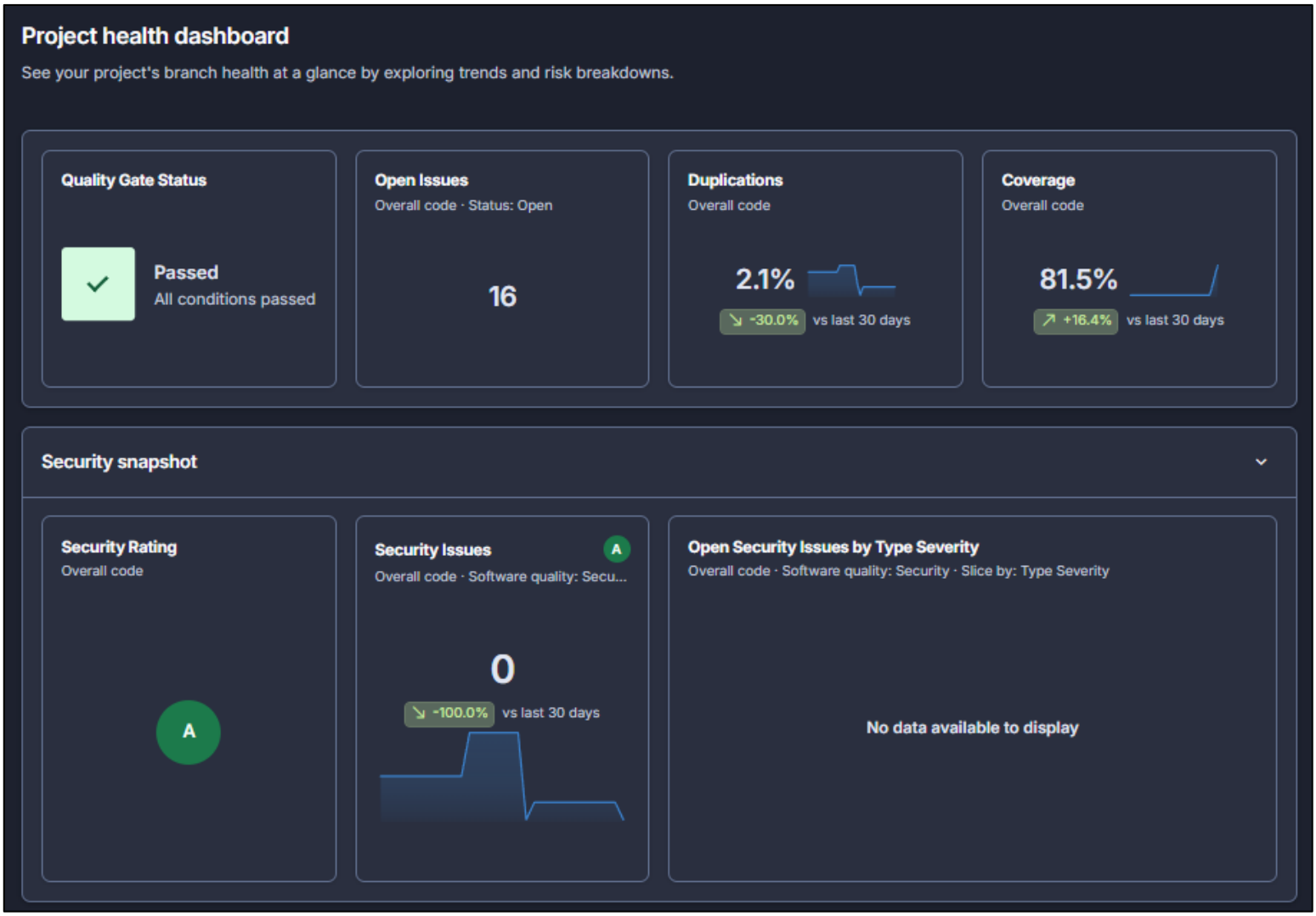

Importer des photos
Glissez vos photos ici
ou cliquez pour parcourir
JPG · PNG · HEIC · WebP
kahovsky171200043.jpg
47 KB
Stockage
248 MB → 248 MB / 500 MB
Annuler
Importer (1)


PhotoApp
Se connecter
Créer un compte
ML
Partagée par Marie Leroy · 18 avril 2025
photo_1.jpg
Partagée via PhotoApp
Télécharger
Organisez vos photos par ambiance avec PhotoApp — 500 Mo gratuits, sans carte bancaire.
Commencer gratuitement


ZONING
Barre de navigation (logo, liens, recherche, avatar)
Sidebar — filtres par palette / année
Barre d'outils (titre, compteur, tri)
Grille de photos


PhotoApp
Galerie
Albums
Exploration chromatique
248 / 500 MB
Importer
ML
VUE
Toutes les photos 247
Favoris 12
Récemment ajoutées
ANNÉES
2025 22
Avril
Mars
Février
Janvier
2024 8
Décembre
Novembre
Galerie 247 photos
Plus récentes
Avril 2025 · 6 photos
Mars 2025 · 7 photos

Annexe 11 - Rapports d'analyse SonarQube



Annexe 12 - Analyse des risques

#	Actif	Menace	Source de risque	Prob.	Impact	Score brut	Mitigation	Score résiduel
R1	Service Auth	Brute force sur login	Bot automatisé	3	3	**9**	Throttler (5 req/60s), Argon2id, refresh token JWT révocable en base	**2**
R2	PostgreSQL	Panne / indisponibilité (SPOF)	Panne matérielle	3	3	**9**	Backups quotidiens automatisés, restart policy Swarm, supervision Grafana (métriques)	**6**
R3	Photos	Accès non autorisé à des photos privées	Hacker externe	2	3	**6**	Ownership guard (`photo.userId`), JWT HTTP-only, URL S3 signées	**2**
R4	Tokens JWT	Vol de token via XSS	Hacker externe	2	3	**6**	Cookies HTTP-only (inaccessibles par JS), access token 15 min, HTTPS	**2**
R5	VPS	Indisponibilité totale (panne matérielle)	Panne matérielle	2	3	**6**	Rolling updates Swarm, snapshot VPS régulier, supervision Grafana (métriques)	**3**
R6	Redis	Indisponibilité (SPOF)	Panne matérielle	2	2	**4**	Restart policy Swarm, graceful degradation (cache miss → DB)	**2**
R7	API NestJS	Injection SQL	Hacker externe	1	3	**3**	TypeORM (requêtes paramétrées), validation Zod sur toutes les entrées	**1**
R8	S3 / AWS	Cloud Act — accès données par autorités US	Fournisseur défaillant	1	2	**2**	Région EU (eu-west-3 Paris), aucune donnée personnelle stockée sur S3	**1**
R9	Logs (Loki)	Fuite de données sensibles dans les logs	Erreur humaine	1	2	**2**	Pino sans MDP ni token ; email loggé uniquement à la connexion (audit intentionnel)	**1**
R10	Images Docker	Supply chain attack	Hacker externe	1	2	**2**	Build CI contrôlé, ghcr.io privé, GITHUB_TOKEN éphémère	**1**

Annexe 13 - Extrait de la note de cadrage

2. Objectifs

#	Objectif	Indicateur de succès
O1	Permettre à un particulier de stocker, organiser et partager ses photos sans abonnement croissant	Parcours upload → album → partage fonctionnel de bout en bout
O2	Offrir au photographe amateur une organisation **par la couleur** inédite	Nuancier (atlas) peuplé automatiquement à l'upload, navigable par teinte
O3	Garantir la confidentialité (hébergement EU, données personnelles non externalisées)	Métadonnées et comptes sur VPS EU ; binaires sur S3 région Paris
O4	Industrialiser le déploiement et l'exploitation	CI/CD automatisé, observabilité (métriques + logs), infra reproductible

3. Périmètre

3.1 Inclus (livré ou en cours — cf. user-stories.md)

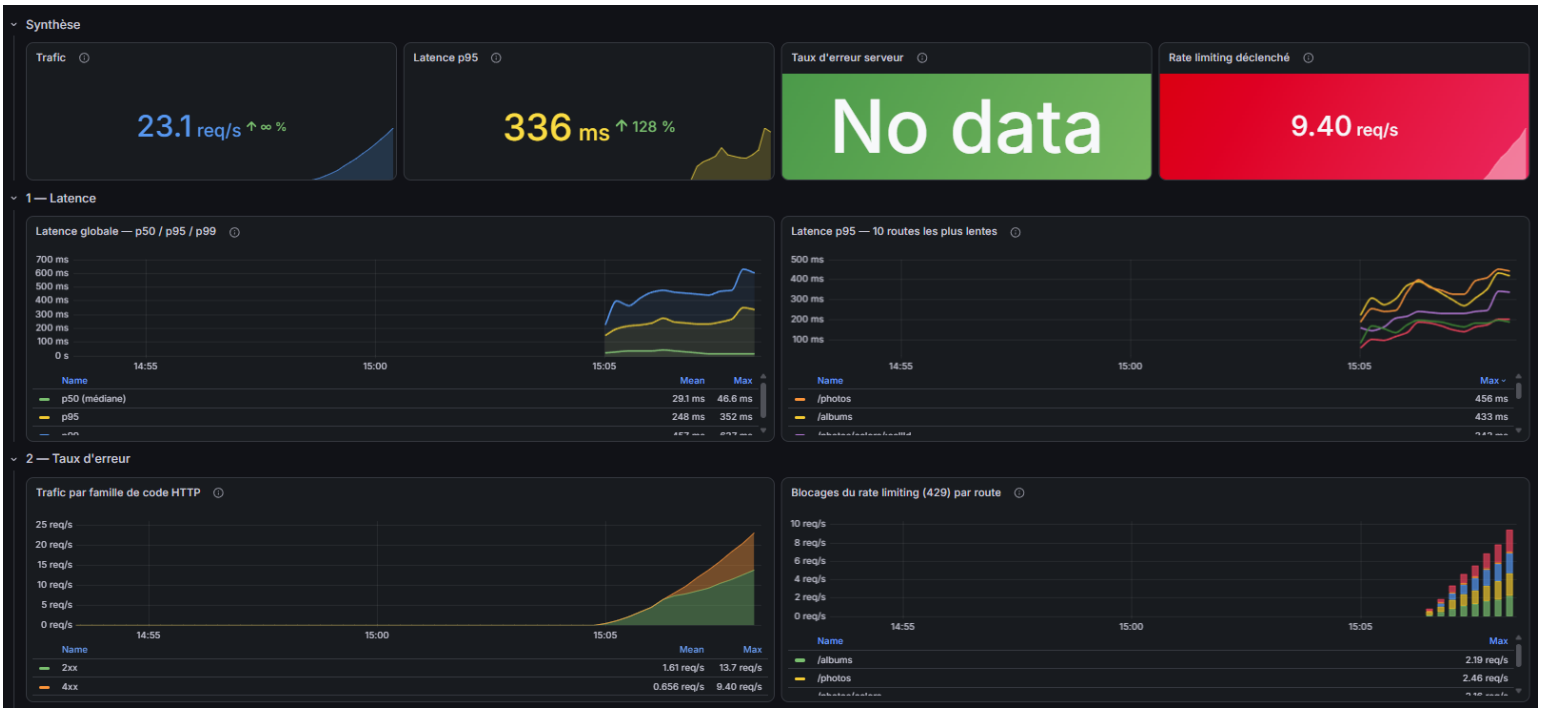
- **Authentification & compte :** inscription, connexion, déconnexion (JWT + Argon2).
- **Photos :** upload multiple, galerie chronologique, suppression, détail (date, nom).
- **Albums :** création, ajout/retrait de photos, liste, renommage/suppression.
- **Partage :** lien public par photo (activable/désactivable), album partagé entre utilisateurs (par email).
- **Exploration Chromatique** (killer feature) : extraction de palette OKLCH à l'upload, atlas fixe de 53 cellules, navigation par teinte, restriction à un album.
- **Socle technique :** infrastructure Docker Swarm, CI/CD, observabilité, rate limiting, validation partagée.

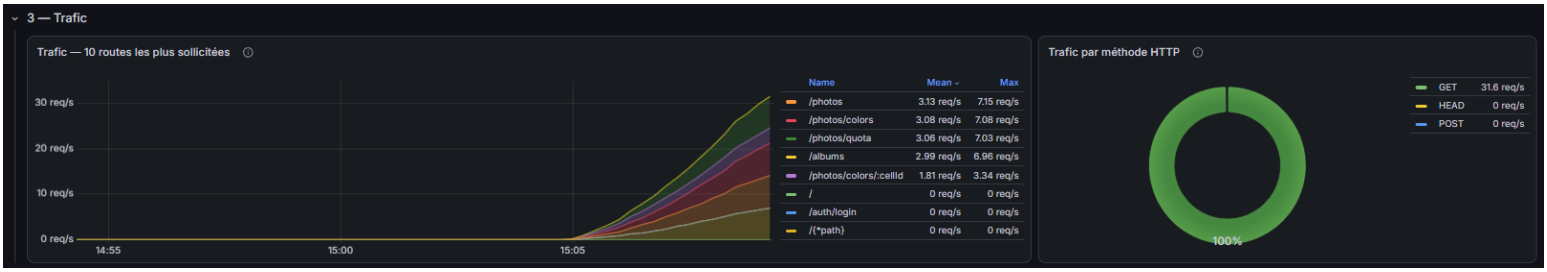
3.2 Exclu / reporté (roadmap)

- Modification et réinitialisation de mot de passe (US-04), suppression de compte / droit à l'oubli automatisé (US-05 — schéma prêt via cascades, pas de purge implémentée).
- Favoris, tags manuels, détail lieu/taille (US-09, US-11, US-10 partiel).
- Recherche et filtres (par date, favori, tag — US-27 à US-29).
- Création d'album directement depuis l'exploration chromatique (US-25), filtres avancés (US-26).
- Photo de couverture d'album (US-17), expiration des liens de partage (US-21).
- Application mobile native, édition photo intégrée, gestion des doublons.

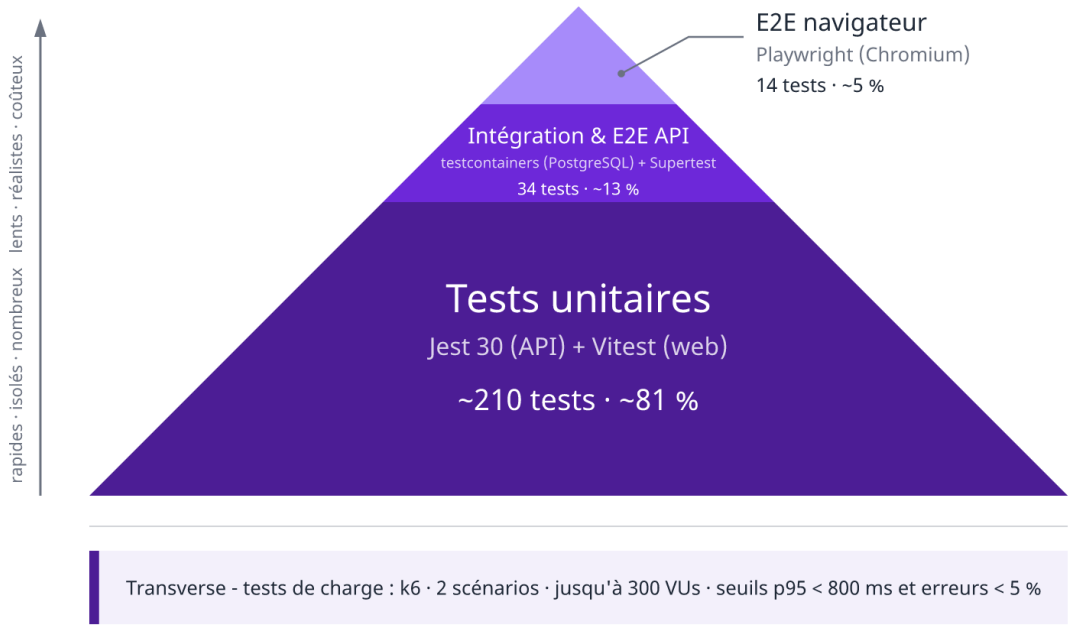
Annexe 14 - Test de charge





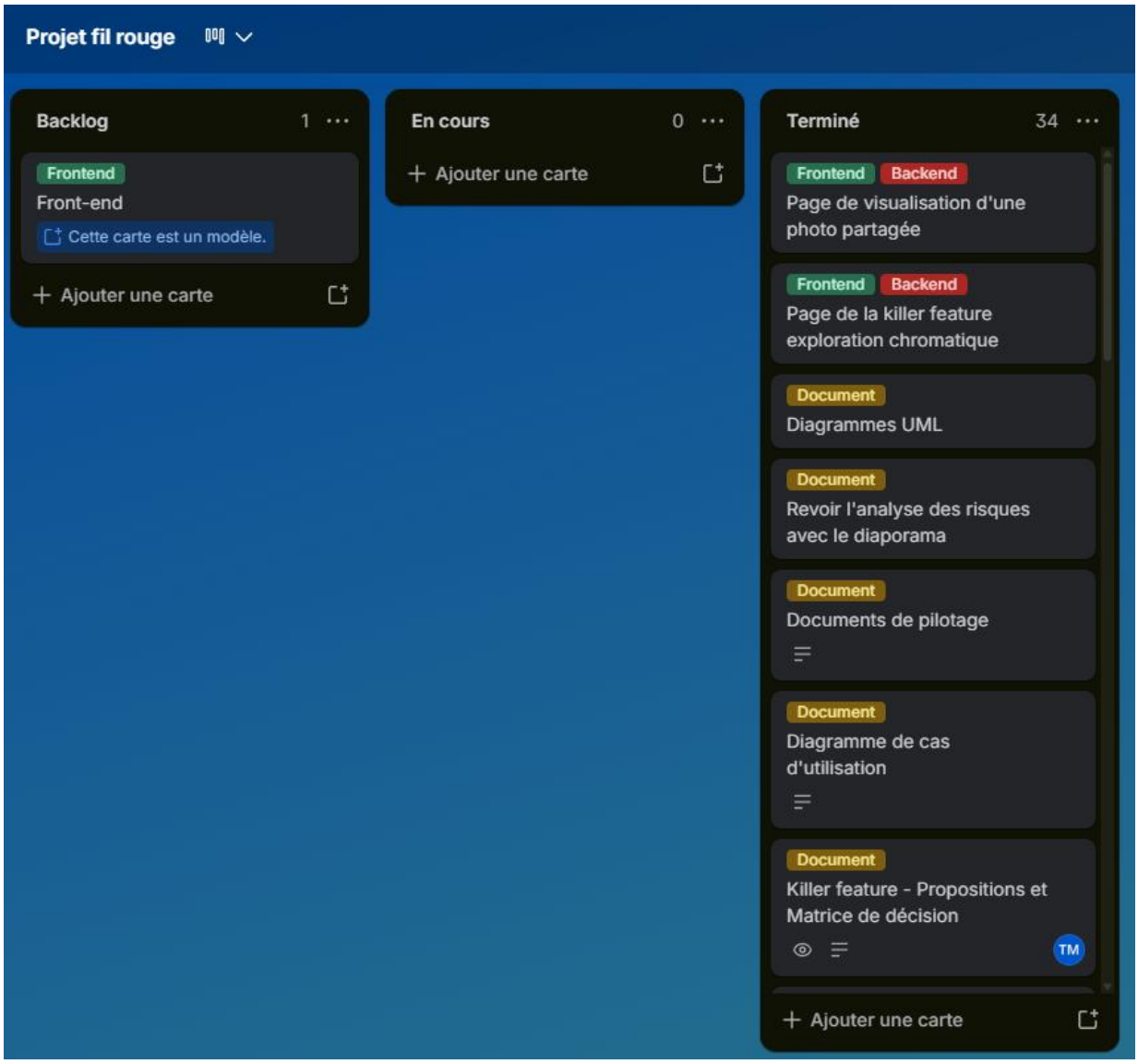


Annexe 15 - Pyramide des tests



Transverse - tests de charge : k6 · 2 scénarios · jusqu'à 300 VUs · seuils p95 < 800 ms et erreurs < 5 %

Annexe 16 - Trello



Annexe 17 - Architecture Decision Records

ADR-001 : Orchestration : Docker Swarm

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Orchestrer plusieurs réplicas (API, Frontend) avec HA, load balancing et rollouts sans coupure, sur un projet solo de 8 semaines.	**Docker Swarm comme orchestrateur de conteneurs.**	Kubernetes (Minikube/Kind) : complexité et charge opérationnelle disproportionnées pour la durée du projet.	Mise en place rapide, conso. ressources réduite (Green IT), pas d'exposition aux outils Kubernetes (Helm, CRDs).

ADR-002 : Base de données : PostgreSQL + PgBouncer

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Données relationnelles (users, photos, albums, partages) et plusieurs réplicas API ouvrant des connexions simultanées vers la base.	**PostgreSQL, avec PgBouncer en pooling de connexions.**	MySQL (moins riche en types), MongoDB (pas de FK natif), connexions directes sans pooler.	Intégrité forte, connexions mutualisées et stables, un seul nœud PostgreSQL, sans réplication.

ADR-003 : Backend : NestJS + TypeORM

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
API REST robuste (auth, CRUD, partage) tournant sur plusieurs réplicas, interagissant avec PostgreSQL, Redis et le stockage fichiers.	**NestJS comme framework, TypeORM comme ORM.**	Express.js (trop minimaliste), Spring Boot (rupture de stack TypeScript), Prisma pour l'ORM.	Stack TypeScript homogène, code modulaire, TypeORM moins performant et moins typé que Prisma.

ADR-004 : Cache : Redis

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Requêtes coûteuses (albums, métadonnées) et cache local non partagé entre réplicas Swarm.	**Redis comme cache centralisé.**	Memcached (structures limitées), cache en mémoire locale (non partagé entre réplicas).	Cache cohérent entre réplicas, charge DB réduite, Redis devient un point de défaillance unique (SPOF).

ADR-005 : Observabilité : Prometheus + Grafana + Loki

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Aucune visibilité sur l'état des services multi-réplicas (charge, latence, erreurs), monitoring exigé par la consigne.	**Prometheus (métriques) + Loki (logs) + Grafana (dashboards unifiés).**	Datadog (payant), ELK/Elasticsearch (lourd, JVM).	Visibilité complète, interface unique, rétention Prometheus limitée à 15 jours.

ADR-006 : Authentification : JWT + Argon2

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Protéger les ressources utilisateur sur une API stateless multi-réplicas, sans session en mémoire locale.	**JWT access (15 min) + refresh (7 j) en cookies HTTP-only, hachage Argon2.**	Sessions serveur Redis, OAuth2/OIDC externe (trop complexe pour un solo).	Sécurité élevée, compatible multi-réplicas, protection CSRF ajoute de la complexité côté client.

ADR-007 : Autorisation : RBAC + Ownership

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Distinguer « qui peut faire quoi » (rôle) et « qui peut accéder à quoi » (propriété de la ressource).	**RBAC statique (rôles en code) + Ownership-based Access Control, partage par token UUID.**	RBAC dynamique en base (surengineering), ACL par ressource dédiée.	Logique simple et performante, ajouter un rôle nécessite un redéploiement.

ADR-008 : Frontend : Next.js 16 + RSC

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Frontend moderne pour galeries photos, routage protégé et rendu optimisé, conteneurisé dans Swarm.	**Next.js App Router avec React Server Components par défaut, build standalone.**	Pages Router, Remix, Vite + React SPA.	Bundle JS réduit, images optimisées, paradigme RSC récent, support tiers partiel.

ADR-009 : Monorepo : Turborepo + Bun

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Deux applications (api, web) et un package partagé (shared) à builder et déployer efficacement.	**Turborepo comme orchestrateur de build, Bun comme package manager/runtime.**	Nx (trop lourd), Lerna, npm workspaces seuls.	Builds mis en cache, images Docker optimisées (turbo prune), Bun encore moins mature que Node.js.

ADR-010 : CI/CD : GitHub Actions + ghcr.io + SSH

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Pipeline automatisé pour builder, pousser et déployer les images sur un VPS via Docker Swarm, migrations incluses.	**GitHub Actions, registre ghcr.io, déploiement SSH (docker stack deploy).**	GitLab CI/CD, Jenkins, Ansible ou Kubernetes pour le déploiement.	Pipeline entièrement automatisé, rolling updates, VPS = point de défaillance unique, pas de staging.

ADR-011 : State & Fetching : React Query + Axios

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Interactions dynamiques côté client (upload, mutations) en complément des Server Components.	**TanStack React Query + Axios, utilisés uniquement dans les Client Components.**	SWR (moins riche), Apollo Client (GraphQL), fetch natif.	Cache et invalidation précis après mutation, deux systèmes de fetching coexistent (RSC + React Query).

ADR-012 : Validation : Zod + schémas partagés

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Valider les données API et les formulaires front sans dupliquer les règles de validation.	**Zod via nestjs-zod, schémas partagés dans packages/shared.**	class-validator (non partageable), Joi, Valibot.	Validation cohérente front/back, types inférés, nestjs-zod est un package communautaire.

ADR-013 : Rate Limiting : NestJS Throttler

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Routes publiques (login, partage) exposées au brute force et à l'abus de requêtes automatisées.	**@nestjs/throttler en guard global, deux buckets (court/long terme), override sur le login.**	Nginx rate limit (hors code), Redis + middleware custom (trop complexe à cette échelle).	Protection immédiate, configuration déclarative, compteurs en mémoire non partagés entre réplicas.

ADR-014 : Logging : Pino

Statut Accepté · 2025-03-13

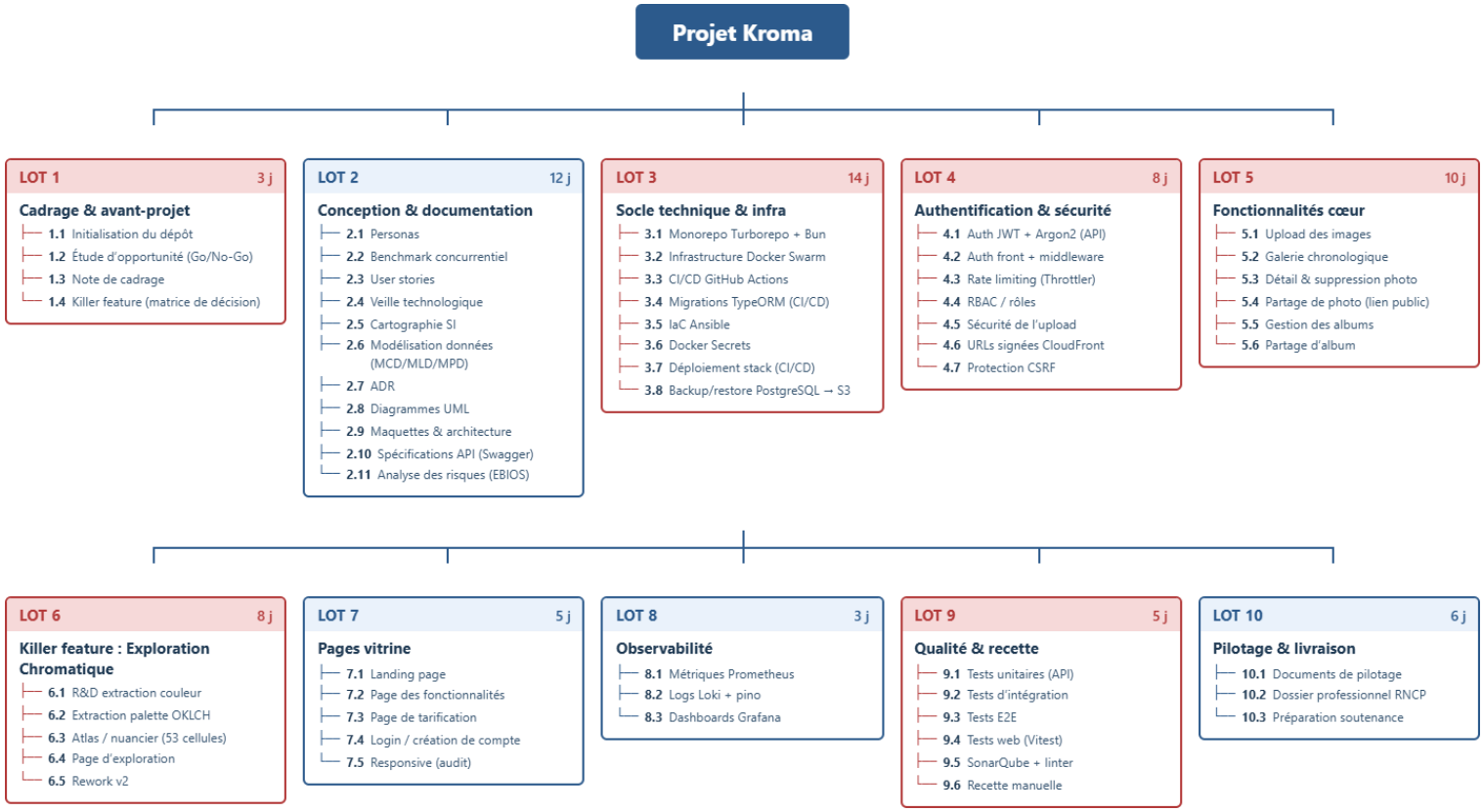
CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Logs exploitables pour le débogage, collectés par Promtail et indexés dans Loki (voir ADR-005).	**Pino via nestjs-pino, JSON en production, pretty en développement.**	Winston, Bunyan, console.log natif.	Logs structurés compatibles Loki, impact minimal sur les performances, package communautaire non officiel.

ADR-015 : UI : shadcn/ui + Radix + Tailwind

Statut Accepté · 2025-03-13

CONTEXTE	DÉCISION	ALTERNATIVES ÉCARTÉES	CONSÉQUENCES
Bibliothèque de composants cohérente, accessible et personnalisable pour React 19 / App Router.	**shadcn/ui (code owned) + Radix UI + Tailwind CSS v4.**	MUI (boîte noire, CSS-in-JS), Mantine, Chakra UI.	Personnalisation totale, accessibilité par défaut, mises à jour manuelles (pas de package npm).

Annexe 18 - WBS : Répartition en lots



Annexe 19 - Pipeline CI / CD et workflows

API

push main · apps/api, packages, infra, stack.yml

job: test

runs-on: ubuntu-latest

- bun install (+ build @repo/shared)
- Tests unitaires avec couverture (bun run test:cov)
- Tests E2E Supertest (bun run test:e2e)

job: test-integration

needs: test

- bun install (+ build @repo/shared)
- Tests d'intégration testcontainers PostgreSQL (bun run test:integration)

job: build-push

needs: test, test-integration

- permissions: contents:read, packages:write
- docker/login-action → ghcr.io
- docker/build-push-action (apps/api/Dockerfile)
- tags: ghcr.io/.../fil-rouge-api:latest

job: secrets

needs: build-push

- pip install ansible
- Configurer la clé SSH (secrets.SSH_PRIVATE_KEY)
- ansible-playbook playbook-secrets.yml (db, jwt, aws, cloudfront)

job: deploy

needs: secrets · concurrency: deploy-production

- scp stack.yml, prometheus.yml, promtail.yml, loki.yml, grafana/
- ssh: docker pull + docker stack deploy mon-app
- docker service update --force prometheus/loki/promtail
- docker image prune -f

job: migrate

needs: deploy

- ssh: sleep 20 (attente postgres)
- docker service create api-migrate (one-shot, restart-condition none)
- bun typeorm migration:run -d dist/database/data-source.js
- Boucle de polling (30×3s) puis docker service rm ; exit 1 si Failed/Rejected

WEB

push main · apps/web, packages

job: test

runs-on: ubuntu-latest

- bun install (+ build @repo/shared)
- Tests Vitest (bun run test)

job: playwright

needs: test

- bunx playwright install --with-deps chromium
- Tests E2E Playwright (bun run test:e2e)
- Upload du rapport si échec (retention 7 jours)

job: build-push

needs: test, playwright

- permissions: contents:read, packages:write
- docker/login-action → ghcr.io
- build-args: NEXT_PUBLIC_API_URL, NEXT_PUBLIC_CLOUDFRONT_DOMAIN
- tags: ghcr.io/.../fil-rouge-web:latest

job: secrets

needs: build-push

- pip install ansible
- Configurer la clé SSH (secrets.SSH_PRIVATE_KEY)
- ansible-playbook playbook-secrets.yml (idempotent)

job: deploy

needs: secrets · concurrency: deploy-production

- scp stack.yml, prometheus.yml, promtail.yml
- ssh: docker pull + docker stack deploy mon-app
- docker image prune -f

Ce qu'il faut retenir

- sonarqube.yml : sonar.qualitygate.wait=true, le job échoue si le Quality Gate est rouge.
- api.yml et web.yml exécutent le même playbook-secrets.yml (idempotent) avant chaque déploiement.
- deploy utilise concurrency deploy-production (cancel-in-progress: false) : un déploiement à la fois.
- migrate, backup et restore sont des services Swarm one-shot (restart-condition: none), supprimés après exécution.

Les autres workflows

SonarQube — PR + push + manu

Décl. pull_request/push main, workflow_dispatch

- checkout fetch-depth: 0 (historique complet)
- Tests + couverture (shared, api, web)
- sonarqube-scan-action, qualitygate.wait=true

Infrastructure — manuel

Décl. workflow_dispatch (playbook, remote_user)

- input playbook: provision / swarm
- input remote_user: deploy (défaut) / root
- Configure clé SSH + exécute le playbook choisi

Backup — cron 02h00 + manuel

Décl. schedule '0 2 * * *' + workflow_dispatch

- Service one-shot postgres:15-alpine
- pg_dump --data-only --exclude-table=migrations | gzip
- aws s3 cp → s3://fil-rouge-backups/

Restore — manuel

Décl. workflow_dispatch (input backup_file)

- Service one-shot postgres:15-alpine
- aws s3 cp du fichier | gunzip
- psql -v ON_ERROR_STOP=1 (restauration)

Annexe 20 - Personas

LM

Léa Martin · 28 ans
Chargée de communication, Lyon
iPhone, MacBook

« J'ai des milliers de photos de voyage sur Google Photos mais je sais même plus ce qu'il y a dedans. »

CONTEXTE
Voyage 3-4 fois par an, partage ses souvenirs avec ses proches. Non technique, cherche la simplicité. Google Photos saturé et payant.

OBJECTIFS
- Retrouver ses photos par voyage
- Partager des albums en un clic
- Ne plus payer d'abonnement
- Garder ses photos privées

FRUSTRATIONS
- Stockage payant vite saturé
- Méfiance sur ses données
- Tri automatique décevant
- Photos éparpillées (iCloud/Google/WhatsApp)

OUTILS ACTUELS
Google Photos, iCloud Photos, WhatsApp

APPORT DE LA PLATEFORME
- Espace unique sans abonnement croissant
- Albums créés et partagés à sa façon
- Données hébergées en Europe

MR

Maxime Renard · 24 ans
Étudiant en design, Bordeaux
Sony Alpha 7C, NAS 4 To

« Je passe des heures à trier mes photos à la main. J'aurais besoin d'un outil qui comprend ma logique de travail. »

CONTEXTE
Photographie urbaine et portrait. Tri manuel sous Lightroom, organise mentalement ses photos par ambiance colorimétrique.

OBJECTIFS
- Trier selon sa logique colorimétrique
- Visualiser les photos par ambiance
- Construire des albums cohérents
- Réduire le temps de recherche

FRUSTRATIONS
- Tri entièrement manuel
- Aucune vision colorimétrique globale
- Rupture entre travail et partage
- Outils trop complexes ou trop basiques

OUTILS ACTUELS
Adobe Lightroom Classic, dossiers NAS, Google Photos/Flickr

APPORT DE LA PLATEFORME
- Exploration chromatique : nuancier automatique
- Clique une teinte, voit toutes les photos associées
- Cohérence colorielle de ses séries visible

Synthèse comparative

	Léa : Particulier	Maxime : Photographe
Motivation principale	Simplicité + confidentialité	Organisation colorimétrique
Killer feature utilisée	Albums + partage	Exploration chromatique
Niveau technique	Non technique	Non technique
Volume de photos	Quelques centaines / an	Plusieurs milliers / an
Partage	Famille et amis	Entourage et réseau
Douleur actuelle	Coût + vie privée	Temps de tri + manque de vision

Annexe 21 - Matrice MoSCoW

Must Have	15 stories
US-01	Créer un compte
US-02	Se connecter
US-03	Se déconnecter
US-06	Uploader des photos
US-07	Galerie chronologique
US-08	Supprimer une photo
US-12	Créer un album
US-13	Ajouter des photos à un album
US-14	Retirer une photo d'un album
US-15	Lister mes albums
US-18	Partager une photo (lien public)
US-19	Désactiver un lien de partage
US-22	Nuancier chromatique automatique
US-23	Restreindre l'exploration à un album
US-24	Explorer les photos d'une teinte

Should Have	9 stories
US-04	Modifier le mot de passe
US-05	Supprimer compte (RGPD)
US-09	Ajouter une photo en favori
US-10	Voir les détails d'une photo
US-16	Renommer / supprimer un album
US-20	Partager un album par email
US-25	Créer un album depuis l'exploration
US-27	Rechercher par date/période
US-28	Filtrer par favori

Could Have	5 stories
US-11	Ajouter des tags manuels
US-17	Photo de couverture d'album
US-21	Expiration d'un lien de partage
US-26	Filtrer l'exploration (album/période)
US-29	Rechercher par tag

Annexe 22 - Playbooks Ansible

playbook-provision.yml

hosts: all · become: yes

rôle: common

1er rôle exécuté

- Mise à jour système (apt update + dist-upgrade, cache 3600s)
- Paquets de base : curl, ufw, fail2ban, ca-certificates, gnupg, python3-pip, netcat-openbsd
- Création user deploy (groupe sudo, shell bash, home) + sudoers NOPASSWD + clé publique id_deploy.pub
- Durcissement SSH : PermitRootLogin no, PasswordAuthentication no (notify restart sshd)
- UFW policy deny incoming ; allow 22/80/443 ; ports Swarm 2377/7946/4789 limités à src 10.0.0.0/24 ; ufw enabled

rôle: docker

needs : rôle common

- Clé GPG + dépôt officiel Docker dans /etc/apt/keyrings
- apt install docker-ce, docker-ce-cli, containerd.io, docker-buildx-plugin, docker-compose-plugin
- Ajout de deploy au groupe docker (append)
- systemd docker enabled + started

playbook-swarm.yml

hosts: swarm · become: yes

rôle: swarm

needs : —

- docker swarm init sur le 1er manager (advertise-addr/listen-addr = ansible_private_host, port 2377)
- Génération tokens worker/manager (delegate_to 1er manager, run_once)
- Join des workers et managers supplémentaires si swarm_state == inactive
- Réseau overlay mon-app_app-net (attachable, création idempotente)
- Dossier /var/www/html/fil_rouge (owner deploy) + prometheus.yml/promtail.yml/loki.yml vides si absents + dossiers grafana/provisioning
- Label node type=manager (utilisé par stack.yml pour épingler postgres/redis)

playbook-secrets.yml

hosts: managers · run_once: true

Création des Docker Secrets

needs : —

- Boucle docker secret create sur 10 secrets
- db_user, db_password, db_name, jwt_access_secret, jwt_refresh_secret
- aws_access_key_id, aws_secret_access_key, cloudfront_key_pair_id, cloudfront_private_key_b64
- pgbouncer_userlist (format natif "user" "password")
- Idempotent : "already exists" ignoré (changed_when/failed_when) · no_log: true

inventory.ini

[managers] manager-1 · 109.123.247.178 (ansible_host = ansible_private_host)

[workers] vide (worker-1 en exemple, commenté)

[swarm:children] managers + workers

[all:vars]
ansible_ssh_private_key_file=~/.ssh/id_deploy · ansible_python_interpreter=/usr/bin/python3

ansible.cfg

remote_user = deploy

host_key_checking = False · retry_files_enabled = False

roles_path = roles · interpreter_python = auto_silent

Ce qu'il faut retenir

- provision (common + docker) tourne sur hosts: all ; swarm ne tourne que sur le groupe swarm (managers + workers).
- secrets.yml cible uniquement les managers et ne s'exécute qu'une fois (run_once: true).
- Les ports Swarm 2377/7946/4789 sont restreints au CIDR 10.0.0.0/24, à mettre à jour avec le vrai réseau privé Contabo.
- Un seul manager actif aujourd'hui (manager-1) ; le groupe workers est vide dans l'inventaire.